ΣΚΟΠΟΣ

Σκοπός της εργασίας είναι η σχεδίαση, κατασκευή και επίδειξη οχήματος σε λειτουργία αυτόνομης κίνησης εντός διαγραμμισμένης πίστας λαμβάνοντας υπόψη τις επικρατούσες συνθήκες της πίστας.

Η κεντρική μονάδα ελέγχου του συστήματος βασίζεται σε μικροελεγκτή (microcontroller), ενώ για την επίτευξη της πλοήγησης και την αναγνώριση συμβάντων έχουν ενσωματωθεί τεχνολογίες Μηχανικής Όρασης (Computer Vision) και Τεχνητής Νοημοσύνης (AI). Παράλληλα, για την απομακρυσμένη διαχείριση και εποπτεία των δεδομένων του οχήματος, αναπτύχθηκε εξειδικευμένη εφαρμογή παρακολούθησης (Graphical User Interface), η σχεδίαση και υλοποίηση της οποίας πραγματοποιήθηκε με τη χρήση του περιβάλλοντος Qt Creator. Για τη διαχείριση το κώδικα δημιουργήθηκε Github Repository.

ΣΕΝΑΡΙΟ ΛΕΙΤΟΥΡΓΙΑΣ

Η διάταξη λειτουργεί σε 3 καταστάσεις (modes):

1. Κατάσταση αναμονής (mode 1)
2. Κατάσταση κίνησης (mode 2)
4. Κατάσταση στάσης (mode 3)

Η διάταξη αρχικά βρίσκεται σε *κατάσταση ηρεμίας* πριν την εκκίνηση λειτουργίας του οχήματος, μέσω πατήματος του κουμπιού D14 στα πλάγια του μικροελεγκτή που ενεργοποιεί τη τροφοδοσία από τη μπαταρία λιθίου. Κατά την εκκίνηση λειτουργίας ένας βομβητής (BUZZER) ηχεί με ένα σύντομο ήχο (beep) ενημερώνοντας τον χρήστη ότι η διάταξη προχωρά σε *κατάσταση αναμονής (mode 1).*

Κατά την κατάσταση αυτή το όχημα αναμένει το σήμα εκκίνησης από το χρήστη. Ο χρήστης εκκινεί το όχημα με τη χρήση τηλεχειρισμού (IR REMOTE) και το όχημα ανταποκρίνεται με διπλό σύντομο beep πριν ξεκινήσει για πρώτη φορά για να τον ενημερώσει ότι πλέον το όχημα βρίσκεται σε *κατάσταση κίνησης (mode 2)* κατά την οποία όλα τα LEDs είναι σβηστά.

Παράλληλα, γίνεται η σύνδεση στο διαδίκτυο και μέσω πρωτοκόλλου TCP/IP μεταδίδονται δεδομένα εφόσον ζητηθούν από την ειδική εφαρμογή (Application), ένα από αυτά είναι το σύστημα παρακολούθησης της ενσωματωμένης κάμερας. Η κάμερα μεταδίδει εικόνα σε πραγματικό χρόνο στην εφαρμογή , επιτρέποντας στους γονείς και τους υπεύθυνους του σχολείου να έχουν οπτική επαφή και να επιβλέπουν την ασφάλεια και την πορεία του οχήματος εξ αποστάσεως.

Περιπτώσεις:

A. Αν ανιχνευτεί εμπόδιο τότε το όχημα μεταβαίνει σε *κατάσταση στάσης (mode 3)*: ακινητοποιείται σε απόσταση 10cm από το προπορευόμενο όχημα ή εμπόδιο, ανάβει τα ενδεικτικά κόκκινα LEDs (stop) και παραμένει ακίνητο μέχρι το όχημα ή εμπόδιο να κινηθεί εκ νέου ή να βγει εκτός πίστας. Όταν αυτό συμβεί το όχημα σβήνει τα κόκκινα LEDs και ξεκινάει εκ νέου την κίνηση (*mode 2*).

   Αν για κάποιο λόγο το όχημα χρειαστεί να κινηθεί προς τα πίσω τότε ανάβουν ενδεικτικά λευκά LED's (όπισθεν) και ο βομβητής ηχεί το χαρακτηριστικό διακοπτόμενο ήχο της οπισθοπορείας.

B. Αν το όχημα ανιχνεύσει πινακίδες σήμανσης του ΚΟΚ συμπεριφέρεται αναλόγως:

   Πινακίδα υποχρεωτικής διακοπής πορείας (STOP): το όχημα κάνει μια προσωρινή παύση της κίνησης του για 3s και μετά το πέρας του χρόνου συνεχίζει την πορεία του.

Πινακίδα απαγόρευσης εισόδου (NO ENTRY): το όχημα σταματά τη λειτουργία του και τίθεται σε *κατάσταση αναμονής (mode 1)*.

C. Αν το όχημα ανιχνεύσει ανθρώπινες φιγούρες ακινητοποιείται σε απόσταση 10cm, αναβοσβήνει τα ενδεικτικά πορτοκαλί LEDs (alarm) και ο βομβητής ηχεί το χαρακτηριστικό επαναλαμβανόμενο διακοπτόμενο ήχο του alarm. Το όχημα αναμένει πότε η φιγούρα θα βγει εκτός πίστας για να συνεχίσει την πορεία του.



Εικόνα 1: Πινακίδα απαγόρευσης εισόδου (NO ENTRY)



Εικόνα 2: Πινακίδα υποχρεωτικής διακοπής πορείας (STOP)

D. Αν ενεργοποιηθούν οι αισθητήρες ασφαλείας (κραδασμών ή καπνού) στέλνεται αντίστοιχη ειδοποίηση ασφαλείας στους γονείς και τη διευθυνση του σχολείου μέσω της εφαρμογής.

ΑΡΧΙΤΕΚΤΟΝΙΚΗ ΣΥΣΤΗΜΑΤΟΣ

Προδιαγραφές Πίστας

Οι γραμμές χάραξης (track limits) της πίστας σχηματίζονται από πλαστική ταινία πλάτους 18mm, μαύρου χρώματος. Η ταινία είναι επικολλημένη σε λεία επιφάνεια μέσης αντακλαστικότητας (matte finish) για τη βελτιστοποίηση της αντίθεσης (contrast) κατά την ανίχνευση. Το σχήμα των γραμμών είναι ανοικτό (χωρίς διασταυρώσεις) και γενικά καμπυλόγραμμα, με ακτίνα καμπυλότητας μεγαλύτερη από 30mm. Η γεωμετρία της πίστας είναι κλειστού βρόχου (closed loop), το πλάτος του διαδρόμου της είναι 200mm και η απόσταση μεταξύ των διπλών γραμμών χάραξης 20mm.

ΦΩΤΟΓΡΑΦΙΕΣ ΠΡΑΓΜΑΤΙΚΗΣ ΠΙΣΤΑΣ

Προδιαγραφές Οχήματος

Το όχημα λειτουργεί αυτόνομα βάσει της παρακάτω λογικής ελέγχου:

i. <u>Εκκίνηση & Έλεγχος:</u>
Η ενεργοποίηση του συστήματος πραγματοποιείται ασύρματα μέσω πρωτοκόλλου υπερύθρων (IR Remote), δίνοντας την εντολή μετάβασης από την κατάσταση αναμονής σε κατάσταση κίνησης.

ii. <u>Πλοήγηση & Κίνηση:</u>

Το όχημα κινείται εντός των ορίων της διαγράμμισης χρησιμοποιώντας IR line sensors και διορθωνοντας την πορεία με PID βασισμένο στο γυροσκόπιο (yaw).

iii. Αποφυγή Εμποδίων (Obstacle Avoidance):

Μέσω του αισθητήρα υπερήχων (Ultrasonic Sensor HC-SR04), το όχημα πραγματοποιεί συνεχή μέτρηση της απόστασης από εμπόδια που βρίσκονται μπροστά του. Όταν ανιχνευτεί εμπόδιο σε απόσταση μικρότερη από το προκαθορισμένο όριο ασφαλείας, το όχημα διακόπτει την κίνησή του και εκτελεί ελιγμό αποφυγής, μεταβαίνοντας προσωρινά σε κατάσταση απομάκρυνσης από το εμπόδιο. Η κανονική αυτόνομη κίνηση επανεκκινείται αυτόματα μόλις το εμπόδιο απομακρυνθεί και η απόσταση επανέλθει σε ασφαλή επίπεδα.

iv. Αναγνώριση Σήμανσης:

Το σύστημα επεξεργάζεται εικόνες σε πραγματικό χρόνο για τον εντοπισμό πινακίδων ΚΟΚ μέσω κάμερας OV2640 και προσαρμόζει τη συμπεριφορά του οχήματος ανάλογα με την ένδειξη.

v. Σήμανση Κατάστασης (Mode Indication):

Οπτική: Χρήση ενδεικτικών LEDs για την οπτικοποίηση της κατάστασης λειτουργίας (mode) ή της πρόθεσης ελιγμού.

Ηχητική: Ο ενσωματωμένος βομβητής (buzzer) παρέχει ηχητικά σήματα ανάλογα με τα σφάλματα ή την τρέχουσα λειτουργία.

vii. Τηλεμετρία & Απομακρυσμένη Επίβλεψη (Remote Monitoring):

Το σύστημα υποστηρίζει αμφίδρομη επικοινωνία με ειδική εφαρμογή για γονείς και σχολικούς υπευθύνους:

Οπτική Επαφή: Μετάδοση εικόνας (Live Streaming) από την κάμερα του οχήματος.

Εντοπισμός Θέσης: Αποστολή γεωγραφικών συντεταγμένων μέσω του δέκτη GPS.

Ειδοποιήσεις**:** Αυτόματη αποστολή ειδοποιήσεων (Push Notifications) στην εφαρμογή σε περίπτωση ενεργοποίησης των αισθητήρων ασφαλείας (ατύχημα, φωτιά).

Εξαρτήματα – Hardware

Κατηγορία	Εξάρτημα (Hardware)	Περιγραφή / Χαρακτηριστικά	Ποσότητα (Τμχ)	Σημειώσεις
Επεξεργασία	DFRobot FireBeetle 2	ESP32-S3 (με WiFi/BT)	1	Κεντρικός Ελεγκτής
Αισθητήρες	OV2640 Camera	Αισθητήρας Εικόνας	1	Μηχανική Όραση (Signs)
	Ultrasonic Sensor HC-SR04	Ultrasonic Sensor (Υπέρηχος)	1	Ανίχνευση εμποδίων και ανθρώπινων φιγούρων
	IMU - MPU6050	Επιταχυνσιόμετρο & Γυροσκόπιο 3 Αξόνων	1	Σταθεροποίηση/Κλίση
	GPS Module NEO 6M V2	GPS Module	1	Εντοπισμός θέσης
	Fermion MEMS Smoke Gas Detection Sensor	Αισθητήρας Καπνού	1	Ασφάλεια (Safety)
	Gravity Digital Piezo Disk Vibration Sensor	Αισθητήρας Κραδασμών	1	Ανίχνευση πρόσκρουσης
	IR Receiver	Δέκτης Υπερύθρων	1	Για το τηλεχειριστήριο
	IR Sensor	Αισθητήρας εντοπισμού εμποδίων	1	Ανίχνευση διαγράμμισης

Κίνηση	DC Gear Motor TT - 130 RPM	DC Motor 3-6V, 130 RPM	4	Κινητήρες τροχών
	Rubber Wheels	Τροχοί 66x26mm	4	Ελαστικά
	Fermion TB6612FNG 2x1.2A DC Motor Driver	Dual DC Motor Driver	1	Οδηγός Μοτέρ (αντί L298N)
Ισχύς	Μπαταρία Λιθίου	Li-Po 600mAh	1	Τροφοδοσία Microcontroller και Κάμερας
	4 AA Battery Pack	6V	1	Τροφοδοσία Μοτέρ
	Button Switch	Διακόπτης ON/OFF	1	Κεντρικός Διακόπτης
	Fuse Holder (6A)	Θήκη Ασφάλειας	1	Προστασία κυκλώματος Μοτέρ
	Fuse 3A	Ασφάλεια τήξεως 3 Ampere	1	Προστασία κυκλώματος Μοτέρ
	Πυκνωτής	1000μF	1	Σταθεροποίηση τάσης μοτέρ
	Step-down Converter 5V			Προστασία
Ενδείξεις	Active Buzzer	Βομβητής	1	Ηχητικά σήματα
	LED Πορτοκαλί	Diffused 5mm	2	Φλας / Alarm

	LED Κόκκινα	Diffused 5mm	2	Stop lights
	LED Λευκά	Diffused 5mm	2	Φώτα Όπισθεν
	Basic 16x2 Character LCD	White on Blue 5V	1	Προβολή μηνυμάτων (SCL/SDA)
	I2C backpack		1	Μετατροπή πρωτοκόλλου επικοινωνίας σε I2C (SCL/SDA)
Διάφορα	Breadboard	830 tie Points	1	Για τις συνδέσεις
	Mini Breadboard	170 tie Points	1	Για τις συνδέσεις
	MCP23X17	16-bit I/O (Input/Output) expander chip	1	Port Expander
	Jumper Wires	Male-Male, Male-Female		Καλωδίωση
	Αντιστάσεις	68,150,220,4.7k Ohm	10	Για προστασία των LEDs, και για τα transistors
	Transistors	2N2222 NPN BJT	2	Για τον έλεγχο πολλαπλών LEDs με περιορισμένες ακίδες (pins)

	IR Remote	Τηλεχειριστήριο	1	Χειρισμός εκκίνησης
	Σασί (Chassis)	Custom 3D printed	1	Σκελετός οχήματος
	HC-SR04 Mounting Bracket	Βάση στήριξης HC-SR04	1	Στήριξη Υπερήχου
	Βίδες / Παξιμάδια	M3 διάφορα μήκη	1 Σετ	Συναρμολόγηση
	Screw Terminal	2-pin 3.5mm	1	Σύνδεση καλωδίων μπαταρίας

Επιλογή Μικροελεγκτή

Για την κεντρική μονάδα επεξεργασίας και ελέγχου του συστήματος επιλέχθηκε ο μικροελεγκτής FireBeetle 2 ESP32-S3 with Camera – 16MB Flash (N16R8) της DFRobot. Η συγκεκριμένη επιλογή προέκυψε κατόπιν αξιολόγησης των ιδιαίτερων απαιτήσεων του έργου για υπολογιστική ισχύ και διαχείριση ενέργειας.

Οι κύριοι λόγοι που οδήγησαν στην επιλογή του είναι οι εξής:

1. Επέκταση Μνήμης για Επεξεργασία Εικόνας (16MB Flash & 8MB PSRAM):

   Το κρισιμότερο πλεονέκτημα της συγκεκριμένης έκδοσης είναι η ενσωμάτωση 8MB PSRAM (Pseudo-Static RAM) επιπλέον της Flash μνήμης των 16MB.
   Η επεξεργασία εικόνας από την κάμερα απαιτεί μεγάλη μνήμη RAM για την αποθήκευση των καρέ (frame buffering) πριν την ανάλυση. Η μνήμη PSRAM επιτρέπει στο όχημα να αποθηκεύει και να επεξεργάζεται μεγαλύτερο όγκο οπτικών δεδομένων, εξασφαλίζοντας ομαλή αναγνώριση γραμμών χωρίς "κολλήματα" (bottlenecks). Οι απλοί μικροελεγκτές αδυνατούν να διαχειριστούν εικόνες υψηλής ανάλυσης λόγω περιορισμένης μνήμης.

2. Εξειδικευμένη Υποστήριξη Κάμερας (Dedicated Camera Circuit):

Η πλακέτα διαθέτει ενσωματωμένη διεπαφή για την κάμερα OV2640 , εξαλείφοντας την ανάγκη για πολύπλοκες εξωτερικές καλωδιώσεις.

Παρόλο που η κάμερα δεσμεύει σημαντικό αριθμό ακίδων (GPIOs), η διάταξη της πλακέτας εξασφαλίζει επαρκείς ελεύθερες θύρες για τους υπόλοιπους αισθητήρες. Επιπλέον, ο σχεδιασμός αυτός μειώνει δραστικά τις παρεμβολές (signal interference), προσφέροντας καθαρότερη εικόνα και συνεπώς πιο αξιόπιστη αναγνώριση αντικειμένων από τους αλγορίθμους.

3. Ολοκληρωμένη Διαχείριση Ενέργειας:

Ενσωματώνει κύκλωμα φόρτισης μπαταριών Λιθίου (Li-Ion) και δυνατότητα ελέγχου On/Off μέσω hardware. Αυτό επιτρέπει την υιοθέτηση αρχιτεκτονικής διπλής τροφοδοσίας: η κάμερα και ο μικροελεγκτής τροφοδοτούνται από σταθερή πηγή (Li-Po), ξεχωριστά από το κύκλωμα ισχύος των κινητήρων. Με τον τρόπο αυτό, απομονώνεται ο "εγκέφαλος" του οχήματος από τις πτώσεις τάσης που προκαλούν τα μοτέρ.

4. Συνδεσιμότητα και Προγραμματισμός:

Υποστηρίζει διπλή επικοινωνία WiFi και Bluetooth 5 (LE) καθώς και πρωτόκολλα Mesh διευκολύνοντας την τηλεμετρία. Παράλληλα, η συμβατότητα με περιβάλλοντα όπως το Arduino IDE διευκολύνει την ανάπτυξη κώδικα και τη χρήση έτοιμων βιβλιοθηκών για τους αισθητήρες και τα μοτέρ.

Επιλογή Motor Driver

Για τον έλεγχο των κινητήρων του οχήματος, επιλέχθηκε το module οδήγησης Fermion TB6612FNG. Η επιλογή αυτή έγινε κατόπιν συγκριτικής μελέτης με παλαιότερης τεχνολογίας οδηγούς (όπως το L298N), καθώς ο TB6612FNG προσφέρει ανώτερα χαρακτηριστικά απόδοσης που είναι κρίσιμα για μπαταριοκίνητες εφαρμογές.

Οι κύριοι λόγοι που οδήγησαν στην επιλογή του είναι οι εξής:

1. Ελάχιστη Πτώση Τάσης (Efficiency):

Καθώς ο TB6612FNG βασίζεται σε τεχνολογία MOSFET, η εσωτερική του αντίσταση είναι εξαιρετικά χαμηλή με αποτέλεσμα η πτώση τάσης του είναι αμελητέα. Αυτό σημαίνει ότι σχεδόν όλη η τάση της μπαταρίας φτάνει στους κινητήρες, προσφέροντας μεγαλύτερη ροπή και ταχύτητα με την ίδια πηγή ενέργειας. Αντίθετα, παλαιότεροι οδηγοί μετατρέπουν μέρος της ενέργειας σε θερμότητα, μειώνοντας την αυτονομία και διακυβεύοντας τη θερμική σταθερότητα του συστήματος.

2. Θερμική Συμπεριφορά και Μέγεθος:

   Λόγω της υψηλής αποδοτικότητάς του, ο TB6612FNG δεν μετατρέπει την ενέργεια σε θερμότητα. Έτσι, δεν απαιτεί ογκώδεις ψύκτρες (heatsinks) για να λειτουργήσει, γεγονός που μειώνει δραστικά τον όγκο και το βάρος του κυκλώματος. Αυτό μας επέτρεψε να εξοικονομήσουμε χώρο στο σασί του οχήματος

3. Ισχύς και Έλεγχος PWM:

   Το module μπορεί να παρέχει συνεχές ρεύμα 1.2A ανά κανάλι (με κορυφές στα 3.2A), το οποίο είναι υπεραρκετό για την οδήγηση των κινητήρων TT Gear Motors.

   Υποστηρίζει συχνότητες PWM έως και 100 kHz, επιτρέποντας πολύ ομαλό και ακριβή έλεγχο της ταχύτητας των κινητήρων, χωρίς τον ενοχλητικό θόρυβο (buzzing) που παρουσιάζεται σε χαμηλότερες συχνότητες.

4. Αρχιτεκτονική Ελέγχου 4WD:

Παρόλο που ο οδηγός διαθέτει δύο κανάλια εξόδου, επαρκεί για την κίνηση του τετρακίνητου οχήματος. Οι δύο κινητήρες της αριστερής πλευράς συνδέονται παράλληλα στο κανάλι A και της δεξιάς πλευράς στο κανάλι B. Με τον τρόπο αυτό επιτυγχάνεται συγχρονισμένη κίνηση των τροχών ανά πλευρά, επιτρέποντας την εκτέλεση στροφών τύπου "Differential Steering".

Επιλογή Κινητήρων Κίνησης (DC Gear Motors)

Για την κίνηση του οχήματος επιλέχθηκαν οι κινητήρες **DC Gear Motor TT - 130 RPM**. Πρόκειται για κινητήρες συνεχούς ρεύματος (DC) που ενσωματώνουν κιβώτιο υποπολλαπλασιασμού στροφών (gearbox), γνωστοί στη διεθνή βιβλιογραφία και ως "TT Motors" (Tactile/Touch motors) ή "Yellow Motors" λόγω του χαρακτηριστικού κίτρινου χρώματός τους.

Η συγκεκριμένη έκδοση έρχεται με προ-εγκατεστημένα καλώδια (With Wire), γεγονός που απλοποιεί τη διαδικασία συνδεσμολογίας εξαλείφοντας την ανάγκη για κολλήσεις στους ακροδέκτες του μοτέρ.

Χαρακτηριστικό	Τιμή / Περιγραφή

Τύπος	Κινητήρας DC με μειωτήρα (Gear Motor)
Ταχύτητα Περιστροφής (χωρίς φορτίο)	130 RPM (Στροφές ανά λεπτό)
Τάση Λειτουργίας	3V έως 6V DC (Τυπική: 6V)
Σχέση Μετάδοσης (Gear Ratio)	1:48 (48 στροφές εισόδου : 1 εξόδου)

Οι κύριοι λόγοι που οδήγησαν στην επιλογή τους είναι οι εξής:

1. Ροπή (Torque):

   Χάρη στον ενσωματωμένο μειωτήρα στροφών (1:48), ο κινητήρας προσφέρει ικανοποιητική ροπή για την κίνηση μικρών ρομποτικών οχημάτων (σαν το δικό μας), επιτρέποντας στο όχημα να μεταφέρει το βάρος των μπαταριών και των ηλεκτρονικών, αλλά και να κινείται σε ελαφρώς ανώμαλες επιφάνειες.

2. Ταχύτητα:

   Οι 130 στροφές ανά λεπτό προσφέρουν μια ισορροπημένη ταχύτητα που είναι διαχειρίσιμη για εφαρμογές αυτόνομης πλοήγησης και αποφυγής εμποδίων, χωρίς το όχημα να κινείται υπερβολικά γρήγορα ώστε να χάνεται ο έλεγχος.

3. Ευκολία Ενσωμάτωσης:

   Η ύπαρξη των καλωδίων μειώνει τον χρόνο συναρμολόγησης και ελαχιστοποιεί τον κίνδυνο κακής επαφής.

4. Συμβατότητα:

   Συνεργάζονται άψογα με τον Motor Driver που έχουμε επιλέξει (Fermion TB6612FNG), επιτρέποντας τον έλεγχο τόσο της ταχύτητας (μέσω PWM) όσο και της φοράς κίνησης.

Μηχανολογική Σχεδίαση και Κατασκευή

Η σχεδίαση του πλαισίου (chassis) του οχήματος υλοποιήθηκε μέσω του λογισμικού τρισδιάστατης μοντελοποίησης (CAD) Blender. Η διαδικασία σχεδιασμού καθοδηγήθηκε από δύο βασικούς περιορισμούς:

1. Γεωμετρική Συμβατότητα: Οι εξωτερικές διαστάσεις του οχήματος προσαρμόστηκαν ώστε να διασφαλίζεται η απρόσκοπτη κίνηση εντός των ορίων της πίστας (πλάτος διαδρόμου 200mm), όπως ορίζεται στις προδιαγραφές του έργου.

2. Χωροταξική Επάρκεια: Η επιφάνεια του σασί σχεδιάστηκε με γνώμονα την εργονομική και ασφαλή τοποθέτηση του συνόλου των ηλεκτρομηχανικών εξαρτημάτων (μπαταρίες, αισθητήρες, ελεγκτής).

3. Λειτουργική Ενσωμάτωση Εξαρτημάτων: Μετά την οριστικοποίηση του υλικού (τροχοί, κινητήρες, οπτικοί αισθητήρες), ενσωματώθηκαν στο σχεδιαστικό περιβάλλον τα ακριβή τρισδιάστατα μοντέλα τους. Η διαδικασία αυτή επέτρεψε τον σχεδιασμό εξειδικευμένων βάσεων στήριξης (mounts) και διασφάλισε την άρτια συναρμογή όλων των δομικών στοιχείων επί του πλαισίου πριν την εκτύπωση.

4. Στρατηγική Τοποθέτηση Αισθητήρων Ασφαλείας & Τηλεμετρίας: Ιδιαίτερη έμφαση δόθηκε στη θέση των αισθητήρων για τη μέγιστη απόδοση:

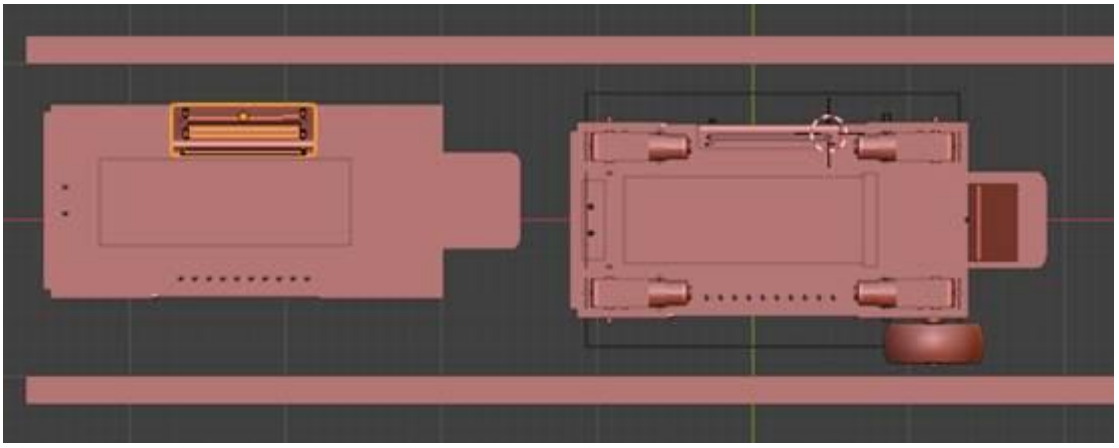
   Η Κάμερα (OV2640) τοποθετήθηκε στο μπροστινό τμήμα και σε κατάλληλο ύψος ώστε να έχει καθαρό οπτικό πεδίο για την αναγνώριση γραμμών και σημάτων.

   Ο Αισθητήρας Κραδασμών τοποθετήθηκε στο κάτω μέρος του πλαισίου (σασί) για την άμεση ανίχνευση μηχανικών κρούσεων.

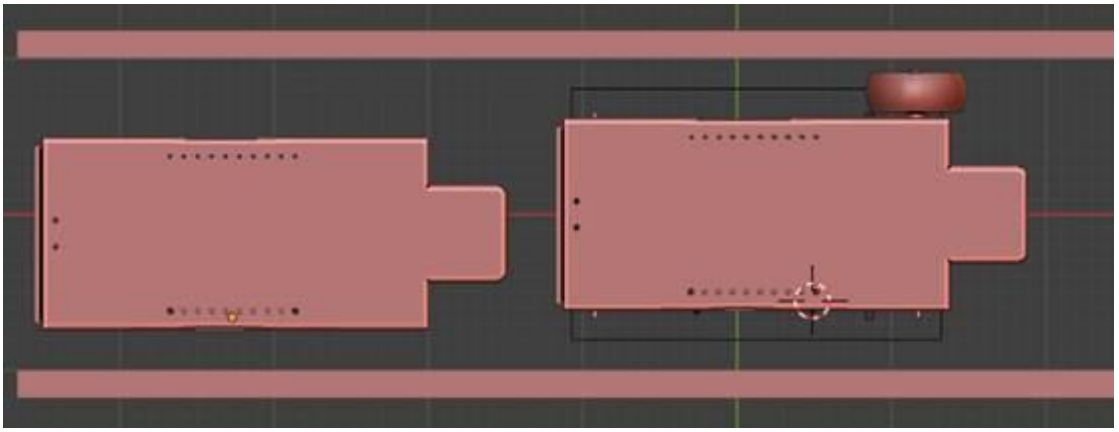
   Ο αισθητήρας καπνού και η κεραία GPS τοποθετήθηκαν σε πλευρικό στήριγμα του οχήματος, επιτρέποντας την ελεύθερη ροή αέρα και τη βέλτιστη λήψη σήματος αντίστοιχα.

   Οι αισθητήρες υπερύθρων (IR) τοποθετήθηκαν στη μπροστινή δεξιά πλευρά του οχήματος, κοντά στον τροχό, με κατάλληλη κλίση ώστε και οι

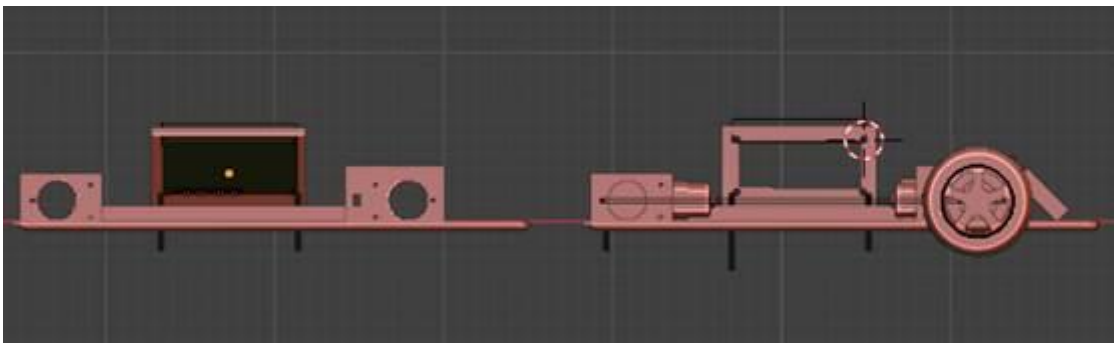
δύο να ανιχνεύουν τις μεταβολές της διαγράμμισης κατά την κίνηση του οχήματος και να συμβάλλουν στην καθοδήγησή του πάνω στην πίστα.



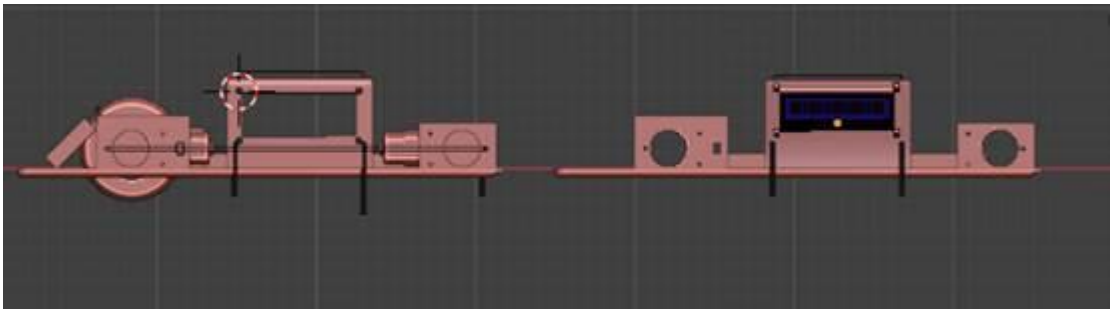
Εικόνα 3: Top View Αριστερά: Το τελικό πλαίσιο όπως κατασκευάστηκε. Δεξιά: Το ψηφιακό μοντέλο (CAD) με την πλήρη χωροθέτηση των εξαρτημάτων.-



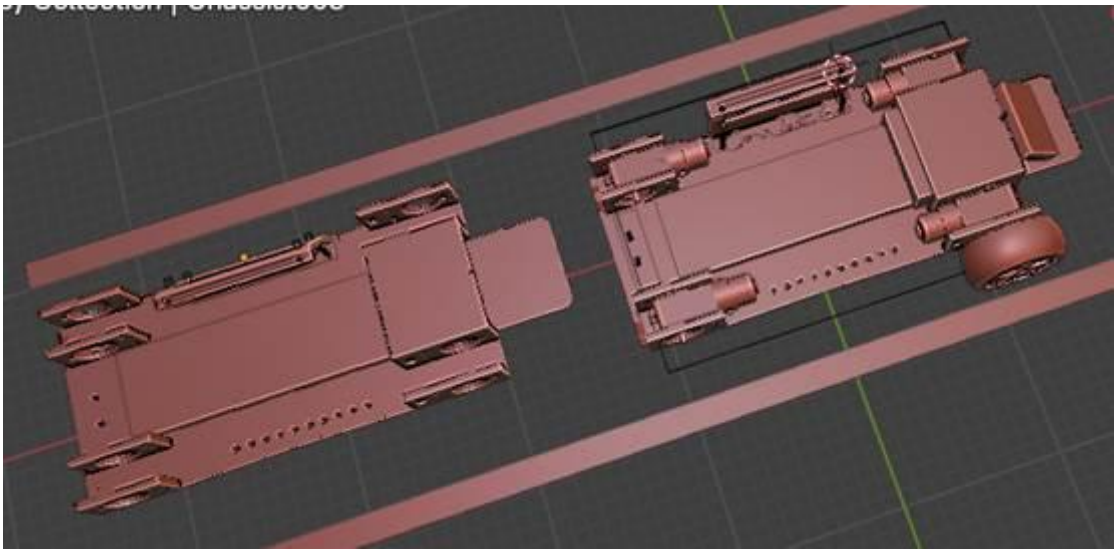
Εικόνα 4:Bottom View- Αριστερά: Το τελικό πλαίσιο όπως κατασκευάστηκε. Δεξιά: Το ψηφιακό μοντέλο (CAD) με την πλήρη χωροθέτηση των εξαρτημάτων.



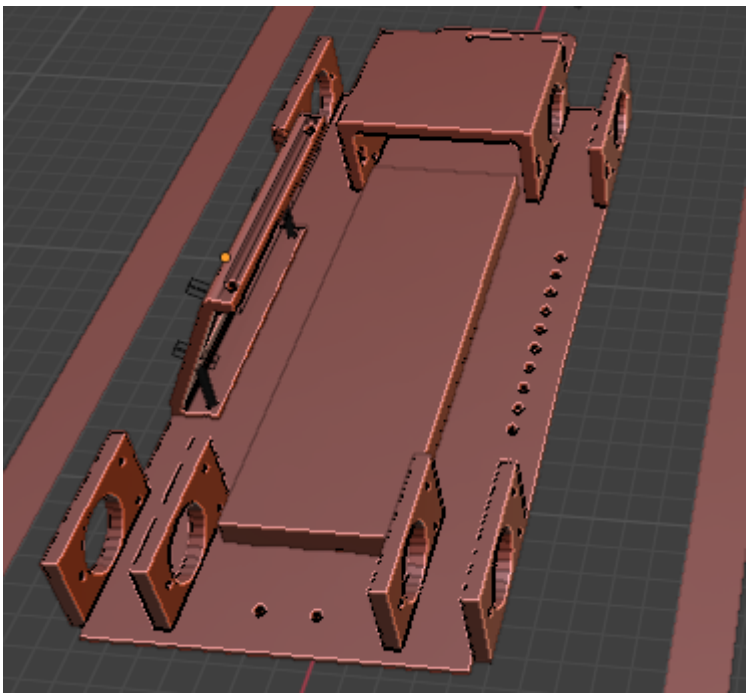
Εικόνα 5: Front View - Αριστερά: Το τελικό πλαίσιο όπως κατασκευάστηκε. Δεξιά: Το ψηφιακό μοντέλο (CAD) με την πλήρη χωροθέτηση των εξαρτημάτων.



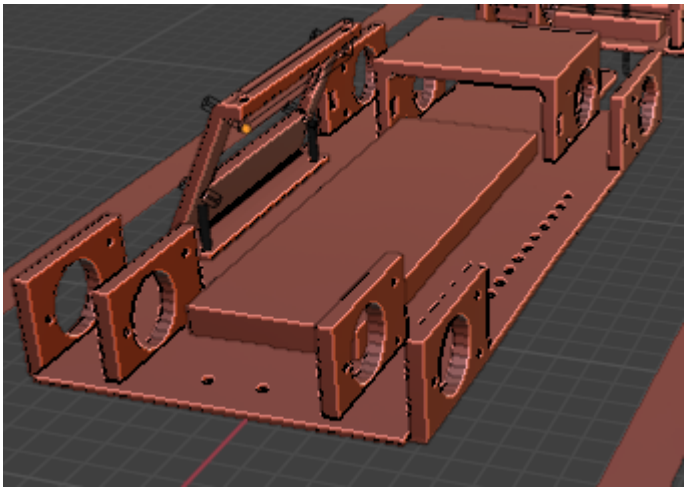
Εικόνα 6: Back View - Αριστερά: Το τελικό πλαίσιο όπως κατασκευάστηκε. Δεξιά: Το ψηφιακό μοντέλο (CAD) με την πλήρη χωροθέτηση των εξαρτημάτων.



Εικόνα 7: Different View - Αριστερά: Το τελικό πλαίσιο όπως κατασκευάστηκε. Δεξιά: Το ψηφιακό μοντέλο (CAD) με την πλήρη χωροθέτηση των εξαρτημάτων.



Εικόνα 8: Different View



Εικόνα 9: Different View

Η φυσική υλοποίηση του πλαισίου πραγματοποιήθηκε μέσω της τεχνολογίας τρισδιάστατης εκτύπωσης (3D Printing), χρησιμοποιώντας ως υλικό εκτύπωσης το PLA (Polylactic Acid), λόγω των κατάλληλων μηχανικών του ιδιοτήτων και της ευκολίας εκτύπωσης.

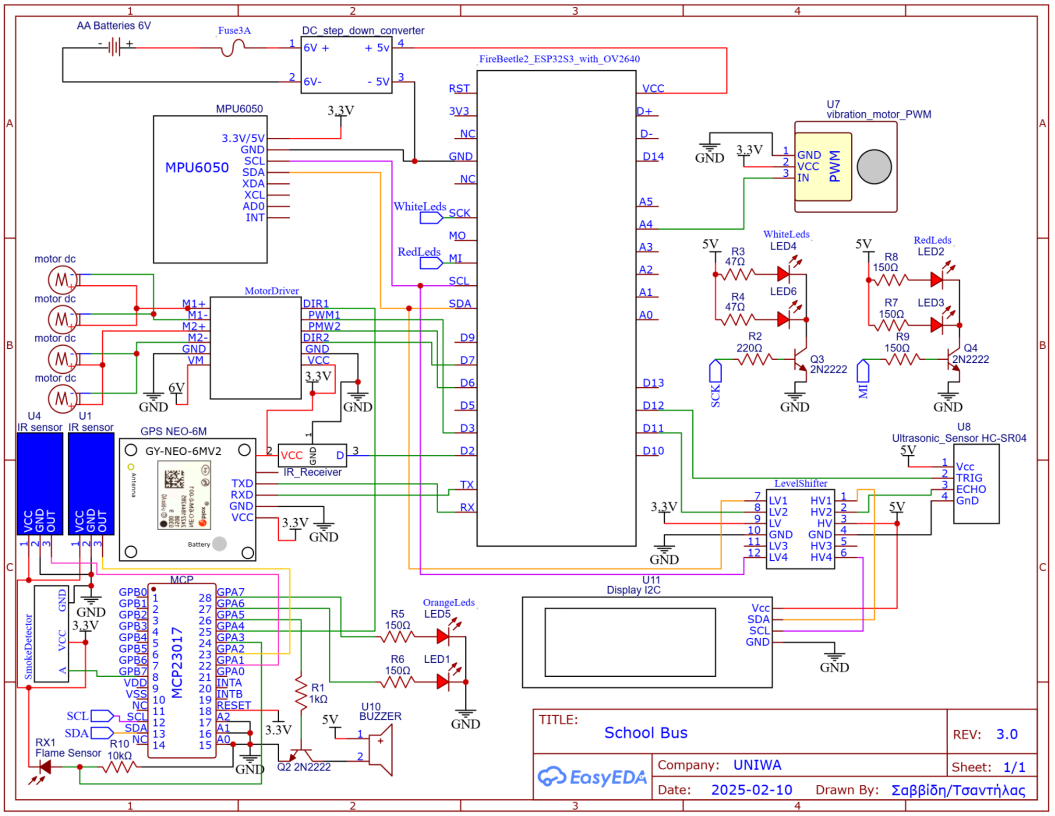
Χρήση Github Repository

Η πλατφόρμα GitHub χρησιμοποιήθηκε για την οργάνωση και τον διαμοιρασμό του κώδικα. Περιλαμβάνει όλο το λογισμικό ελέγχου του οχήματος και των αισθητήρων.

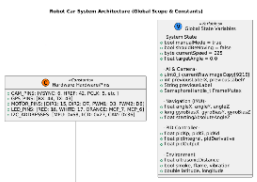
Github Repository: https://github.com/THP-dr/drosos/tree/main

ΛΟΓΙΣΜΙΚΟ ΜΕΡΟΣ

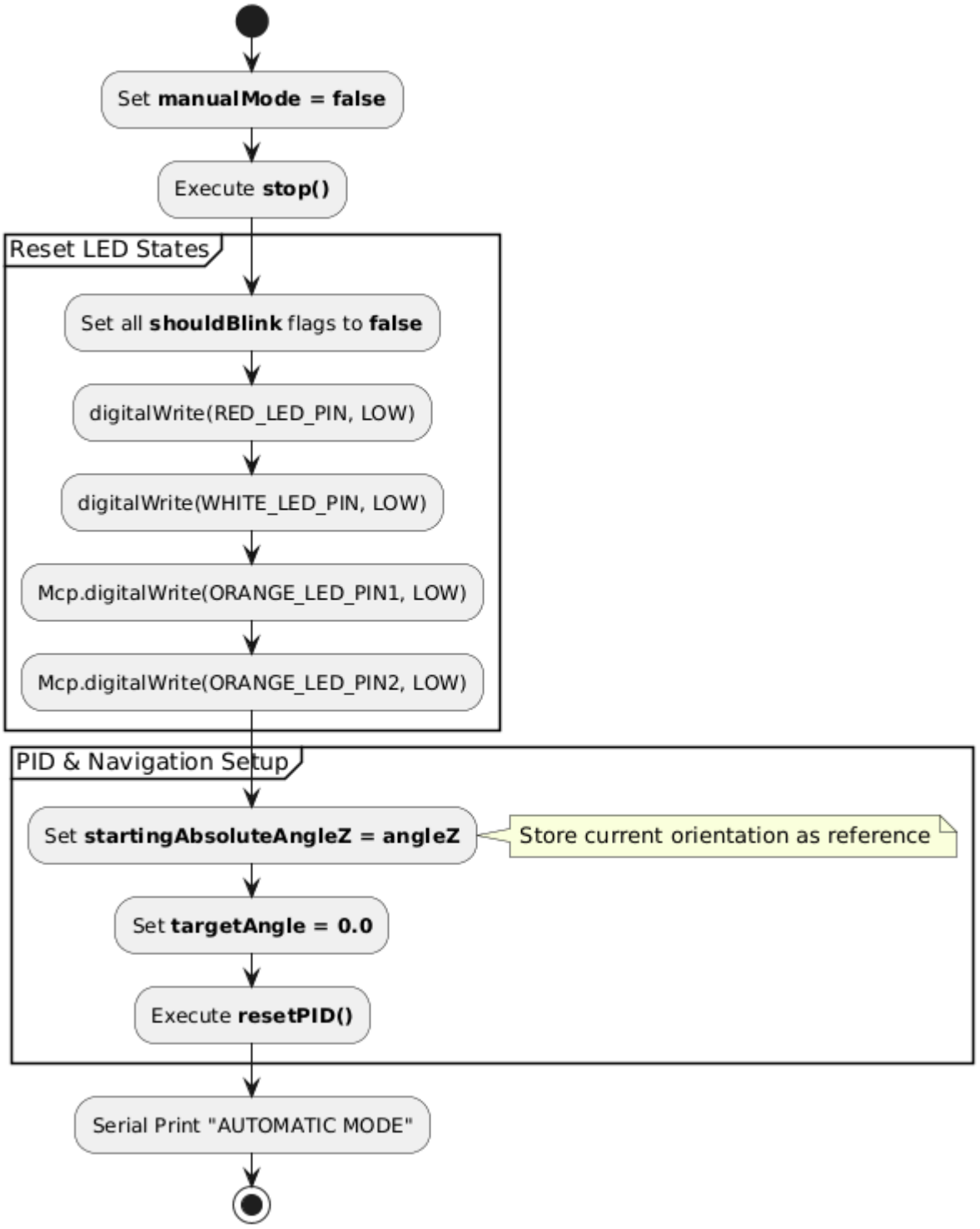
Ηλεκτρονικό Κύκλωμα – Schematic



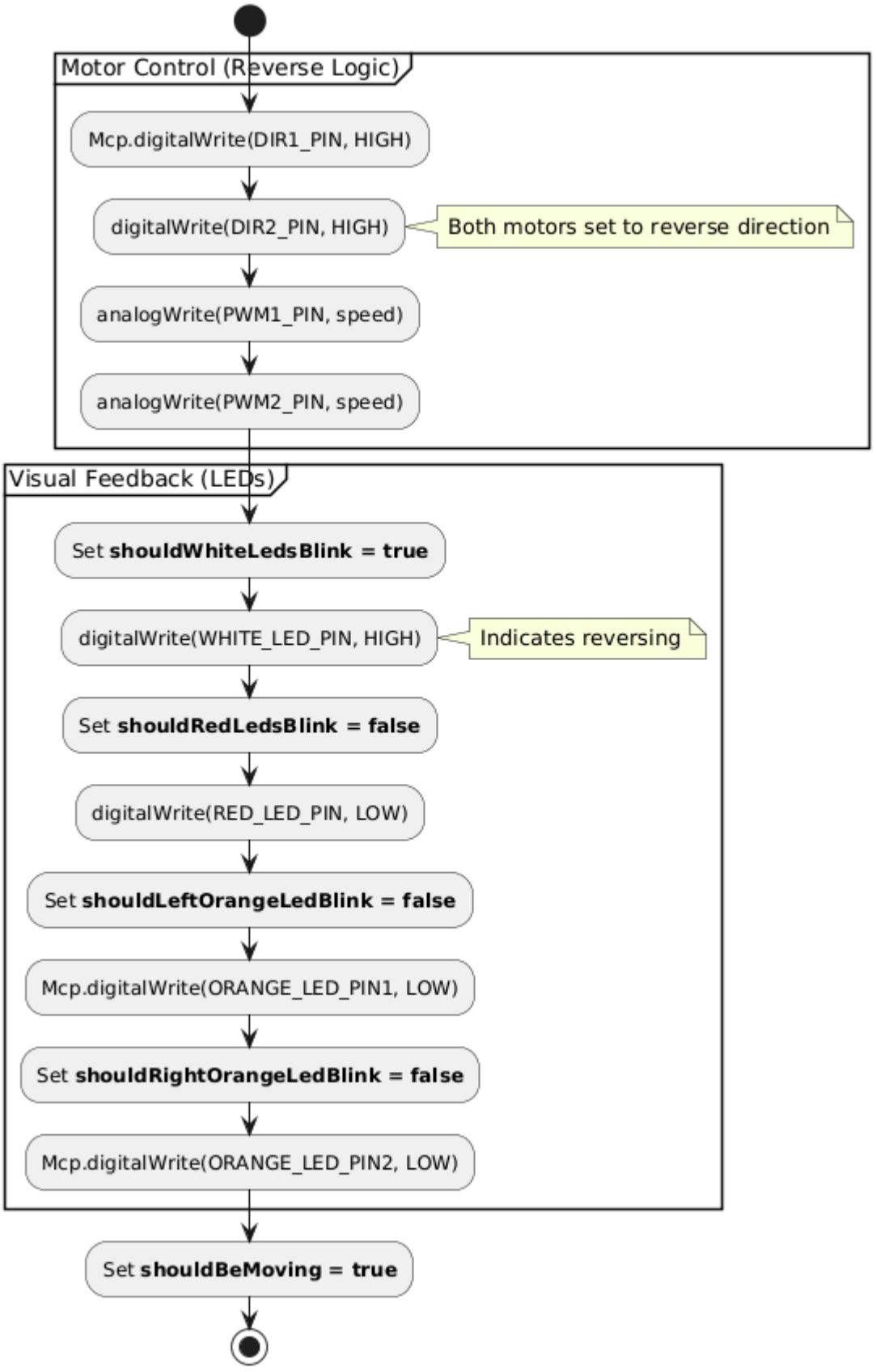
Flowcharts

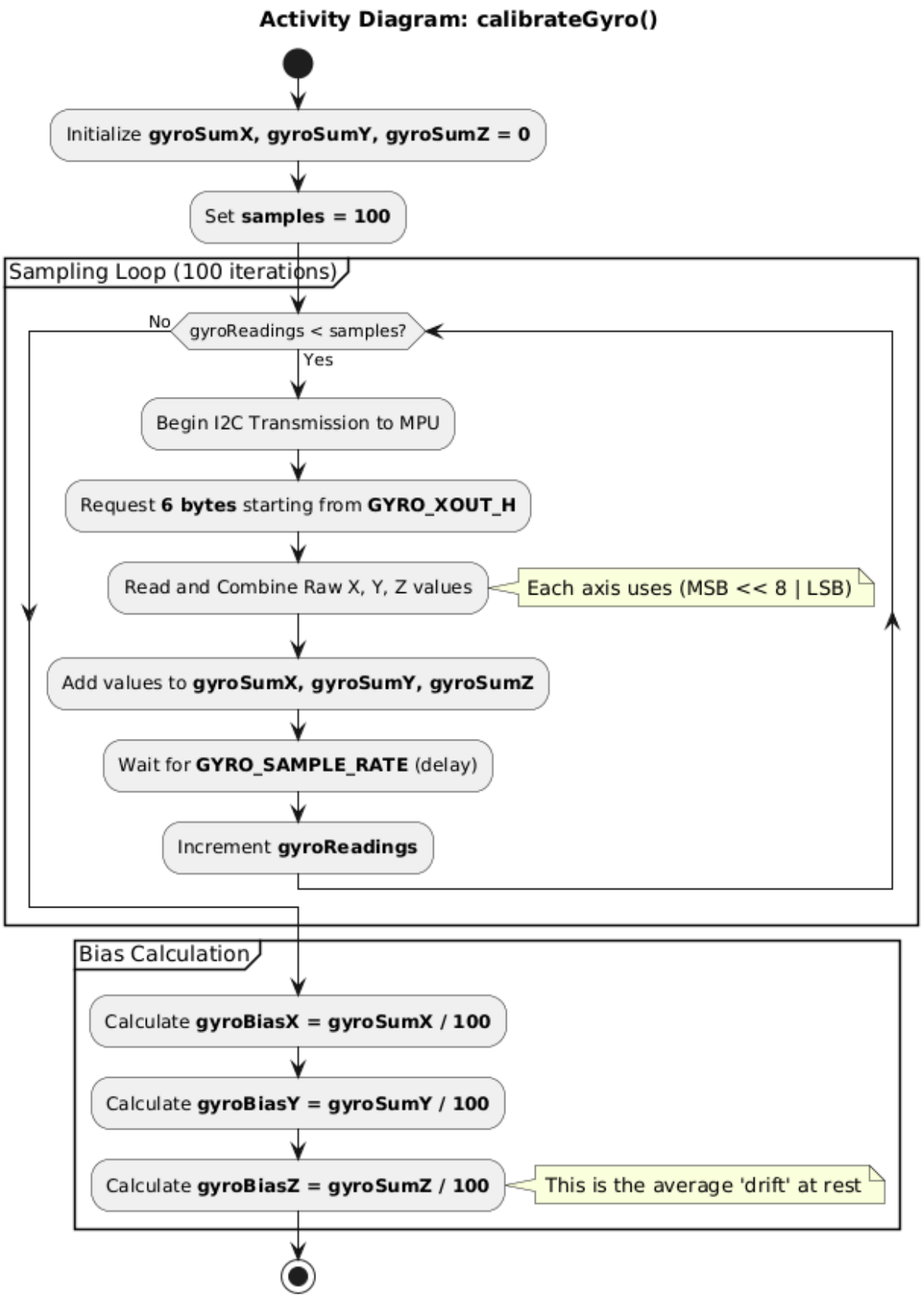

Robot Car System Architecture (Global Scope & Constants)
«Hardware»
Global State variables
System State
bool manualMode = true
float motorSpeed = 255
float targetAngle = 0.0
IR & Camera
uint8_t currentRaw, maxRaw, minRaw
String lastCommand = ""
String previousLine
SensorFrontLeft, SensorFrontRight
Hardware (PWM)
float angleX, angleY, angleZ
long gyroRateX, gyroZRaw, gyroBiasZ
float filteredAngle, prevFiltered
float pidOutput
Environment
float distanceLeft, distanceRight
float prevDistanceLeft, prevDistanceRight
bool isTurning, isBraking, isReversing
double lastLoopTime, loopDuration
«Hardware»
Pin Assignments (Constants)
CAM_PINS: XCLK=10, D0=17, VSYNC=5, HREF=42, PCLK=3, SIOC=1, SIOD=2, PWDN=-1, RESET=-1
MOTOR_PINS: IN1=15, IN2=16, IN3=7, IN4=8, ENA=9, ENB=6
LED_PINS: FRONT_RED=38, FRONT_YELLOW=39, FRONT_GREEN=40, LEFT_BLINKER=41, RIGHT_BLINKER=42, REAR_LIGHT=2
I2C_SDA=48, I2C_SCL=47
«Hardware»
Pin Assignments
Front Ultrasonic Sensor: TRIG=14, ECHO=21
Left Ultrasonic Sensor: TRIG=45, ECHO=35
Right Ultrasonic Sensor: TRIG=46, ECHO=36
«Hardware»
External Libraries / Objects
Adafruit_MCP23X17 mcp
LiquidCrystal_I2C lcd
ESP32QRCodeReader reader
WebSocketsServer webSocket(81)
Activity Diagram: resetPID()
Activity Diagram: pidCorrection (PID Control Loop)
Activity Diagram: setup() Initialization Sequence
Activity Diagram: automaticMode
Activity Diagram: manual_mode
Activity Diagram: readAccelZ()
Activity Diagram: isPickerSign()
LED Control Flow (Non-blocking Blink)
Activity Diagram: forward(speed)
Activity Diagram: backward(speed)
Activity Diagram: stop()
Activity Diagram: turnLeft(speed)
Activity Diagram: turnRight(speed)
Activity Diagram: loop() Main Executive Flow
Activity Diagram: calibrateGyro()
Activity Diagram: readAxis() (Angle Integration)
Inference Task Flowchart
Activity Diagram: gyroBegin() (Wake up MPU)
Activity Diagram: handleServerData
Activity Diagram: handleServerMessage & Web UI Logic
Activity Diagram: handleServerImageCapture
Activity Diagram: convertRawFrame()
Activity Diagram: handleMotorControl
Activity Diagram: handleCommunication
Autonomous Navigation
Emergency Stop
Camera Capture & Classification
Web Server Update
Main Loop Timing

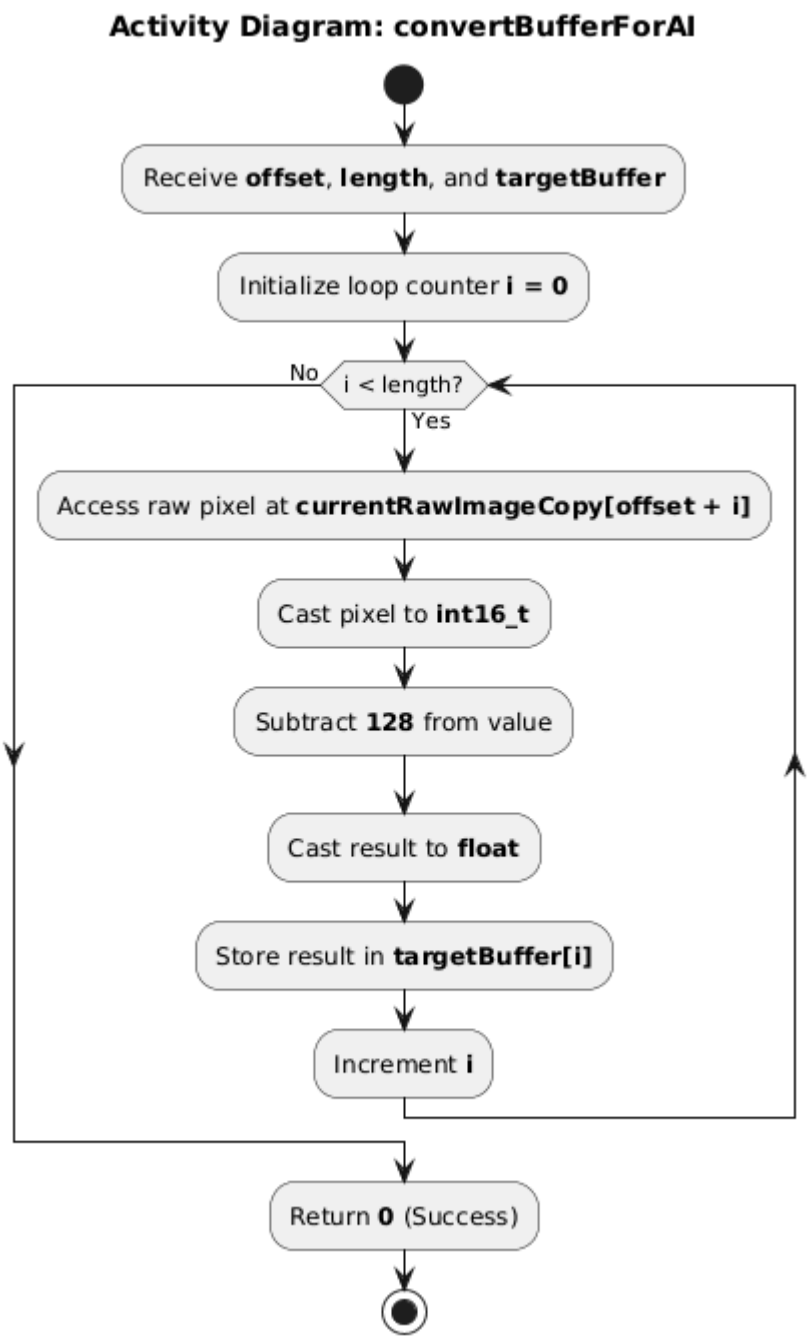
Activity Diagram: automaticMode



Activity Diagram: backward(speed)



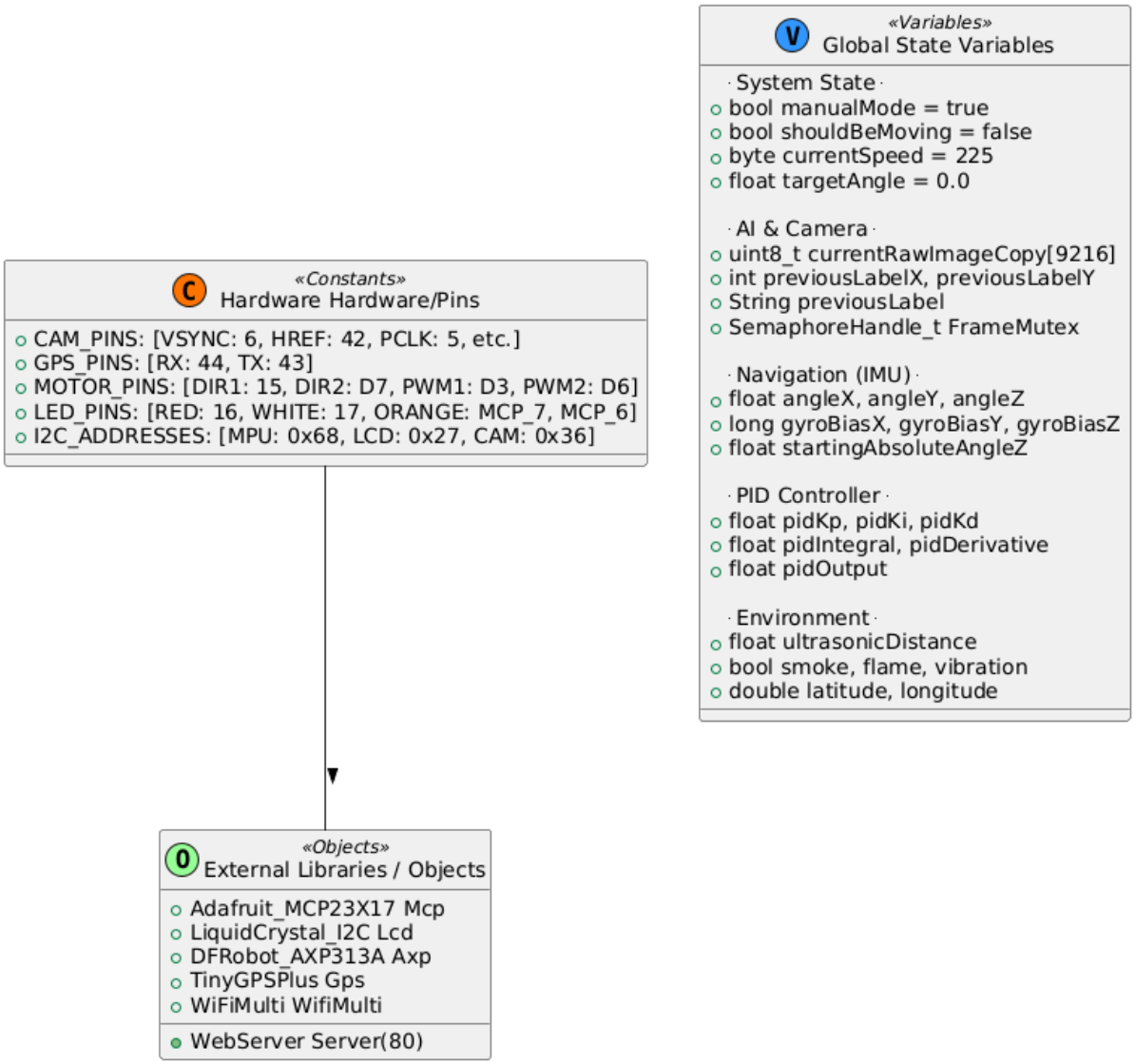

Activity Diagram: calibrateGyro()
Initialize gyroSumX, gyroSumY, gyroSumZ = 0
Set samples = 100
Sampling Loop (100 iterations)
No
gyroReadings < samples?
Yes
Begin I2C Transmission to MPU
Request 6 bytes starting from GYRO_XOUT_H
Read and Combine Raw X, Y, Z values
Each axis uses (MSB << 8 | LSB)
Add values to gyroSumX, gyroSumY, gyroSumZ
Wait for GYRO_SAMPLE_RATE (delay)
Increment gyroReadings
Bias Calculation
Calculate gyroBiasX = gyroSumX / 100
Calculate gyroBiasY = gyroSumY / 100
Calculate gyroBiasZ = gyroSumZ / 100
This is the average 'drift' at rest


Activity Diagram: convertBufferForAI
Receive offset, length, and targetBuffer
Initialize loop counter i = 0
No
i < length?
Yes
Access raw pixel at currentRawImageCopy[offset + i]
Cast pixel to int16_t
Subtract 128 from value
Cast result to float
Store result in targetBuffer[i]
Increment i
Return 0 (Success)

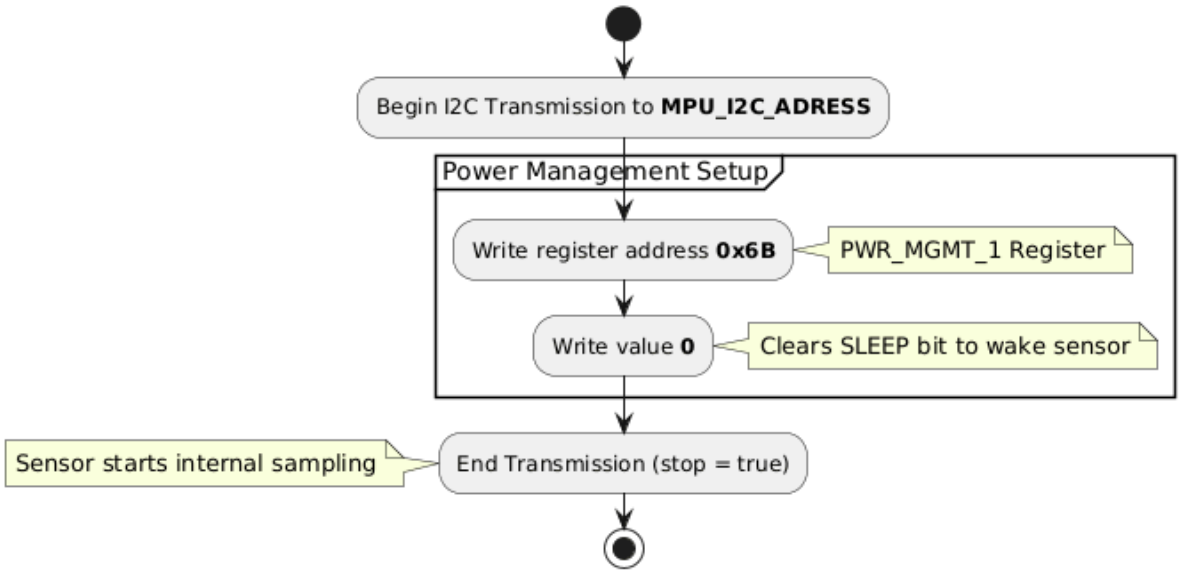
Activity Diagram: forward(speed)



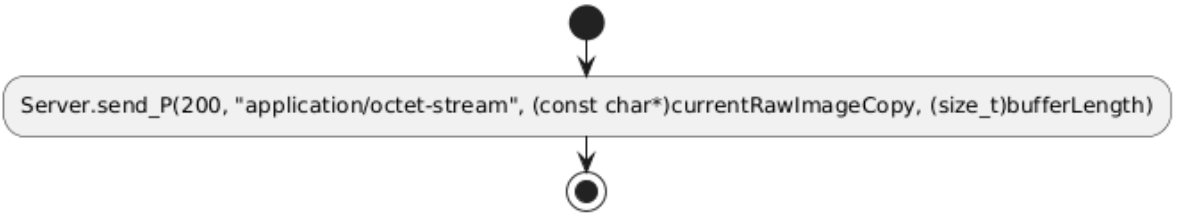
Robot Car System Architecture (Global Scope & Constants)

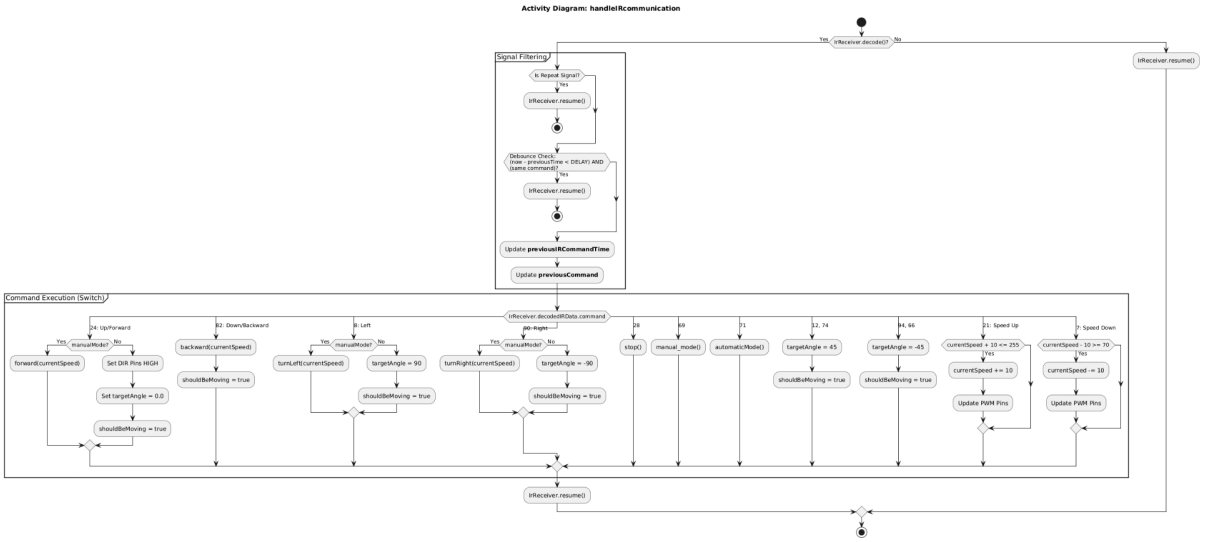


Activity Diagram: gyroBegin() (Wake up MPU)

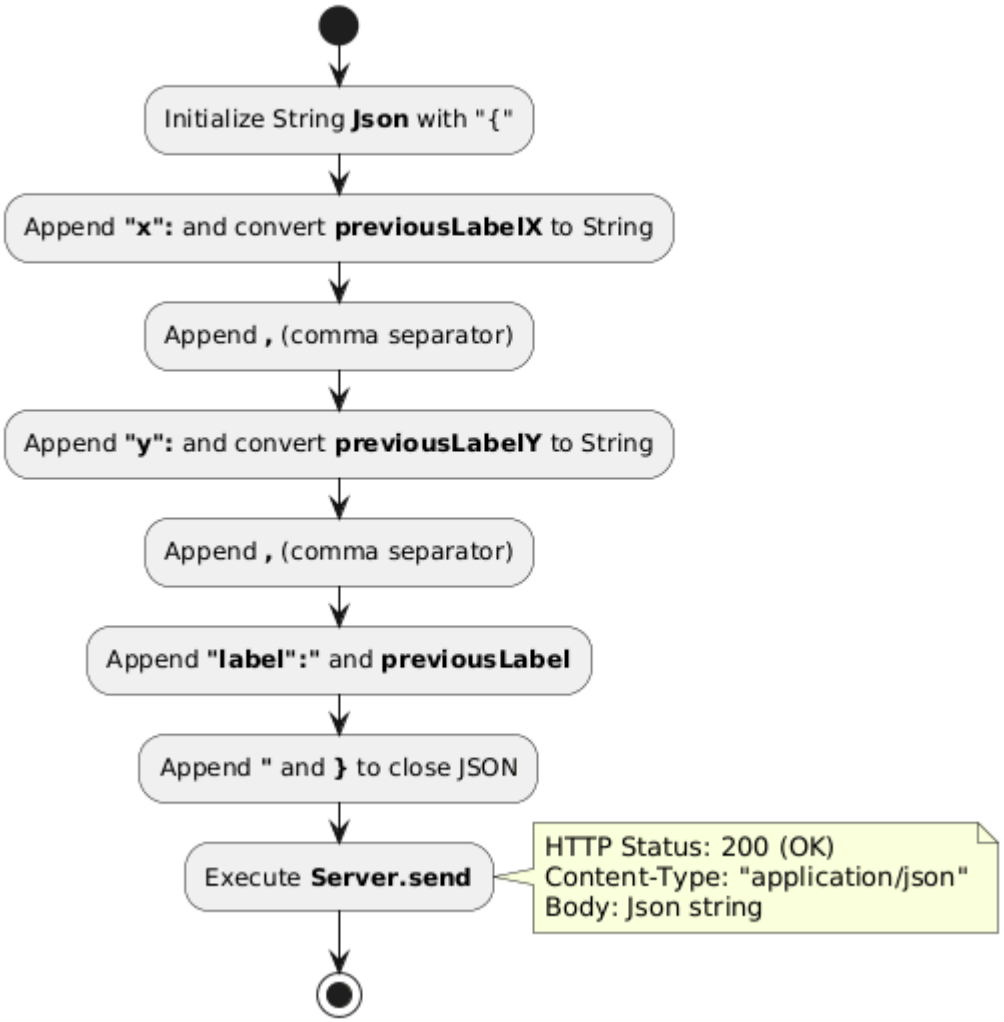


Activity Diagram: handleServerImageCapture

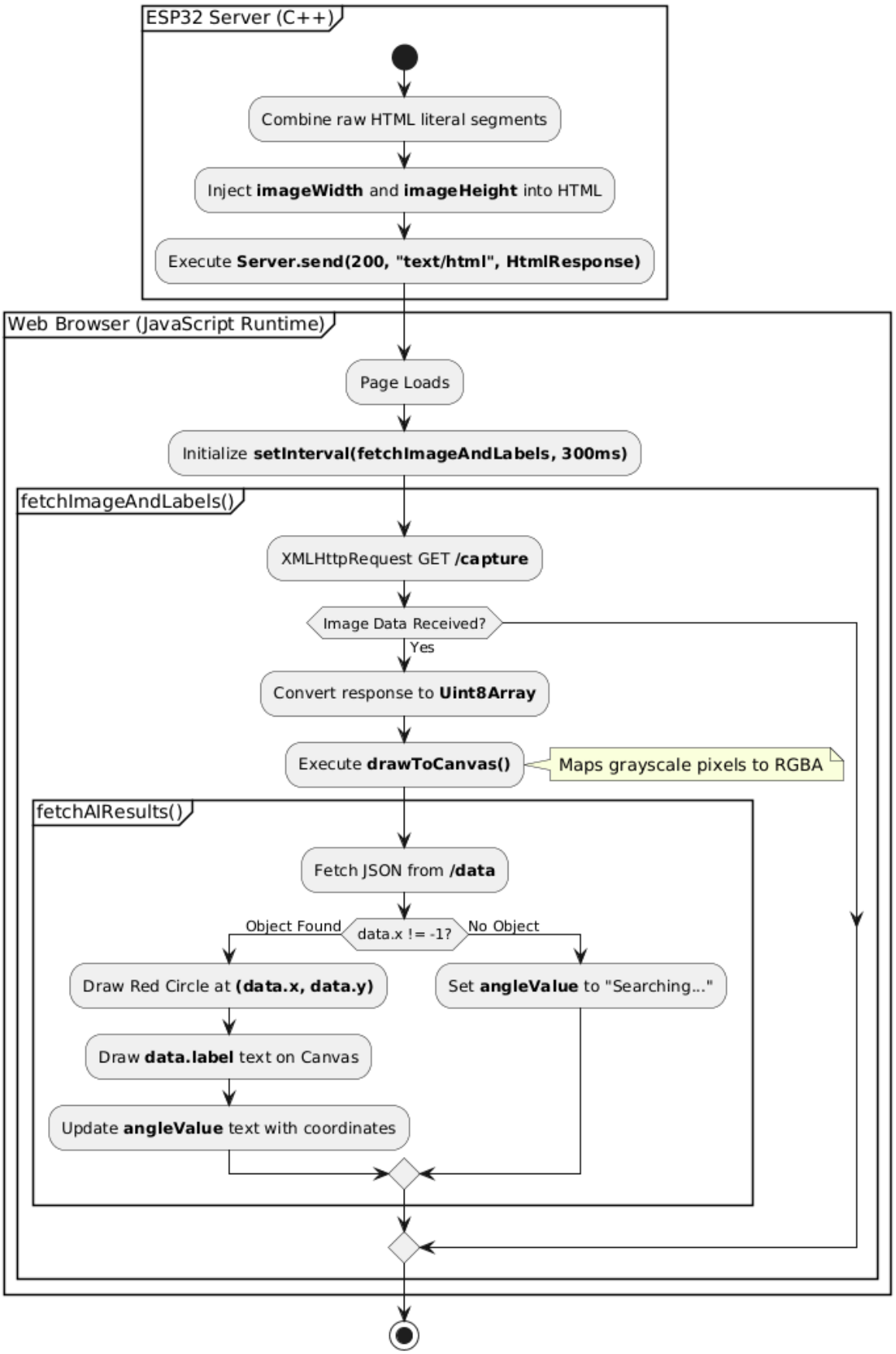


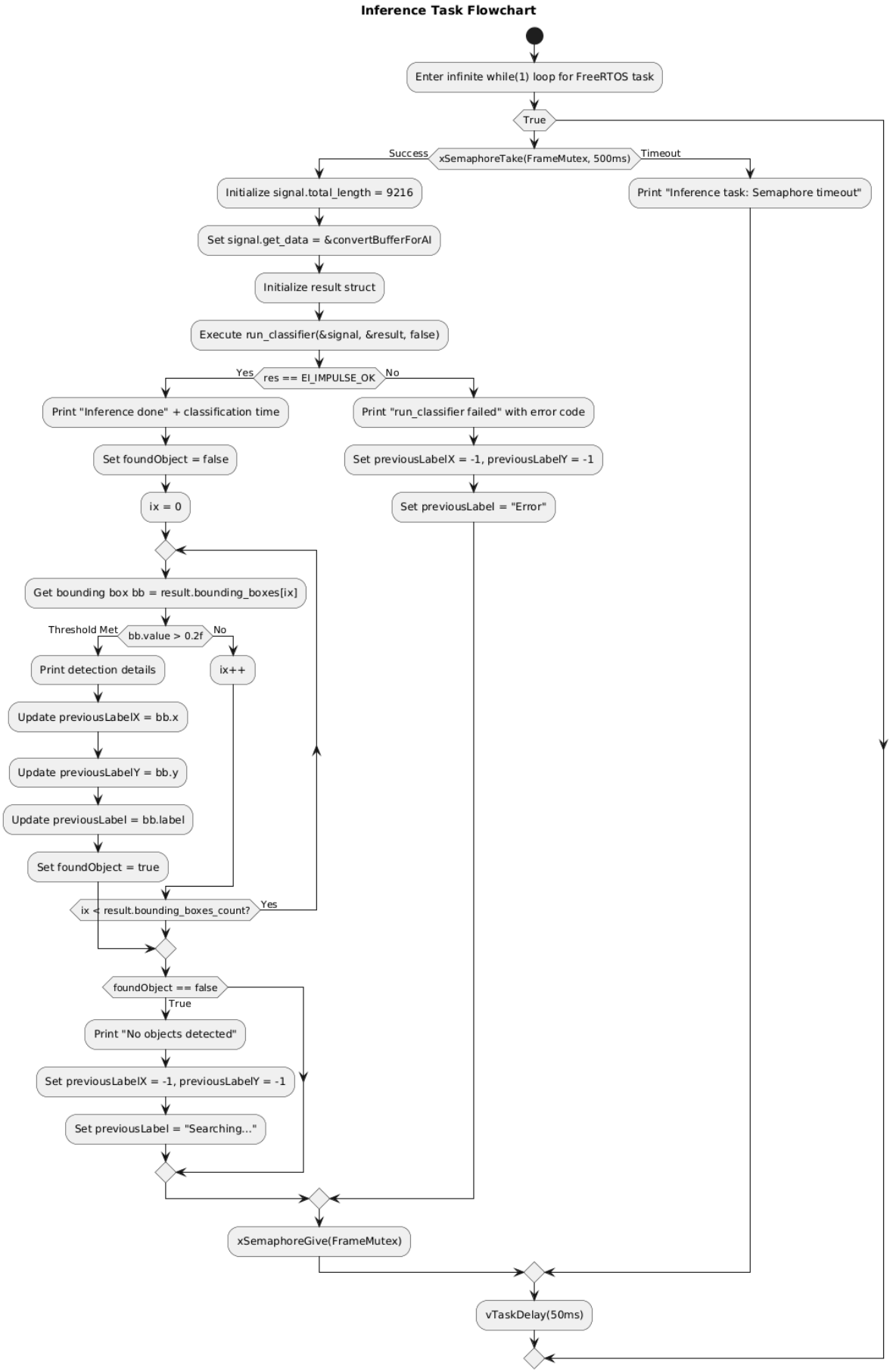


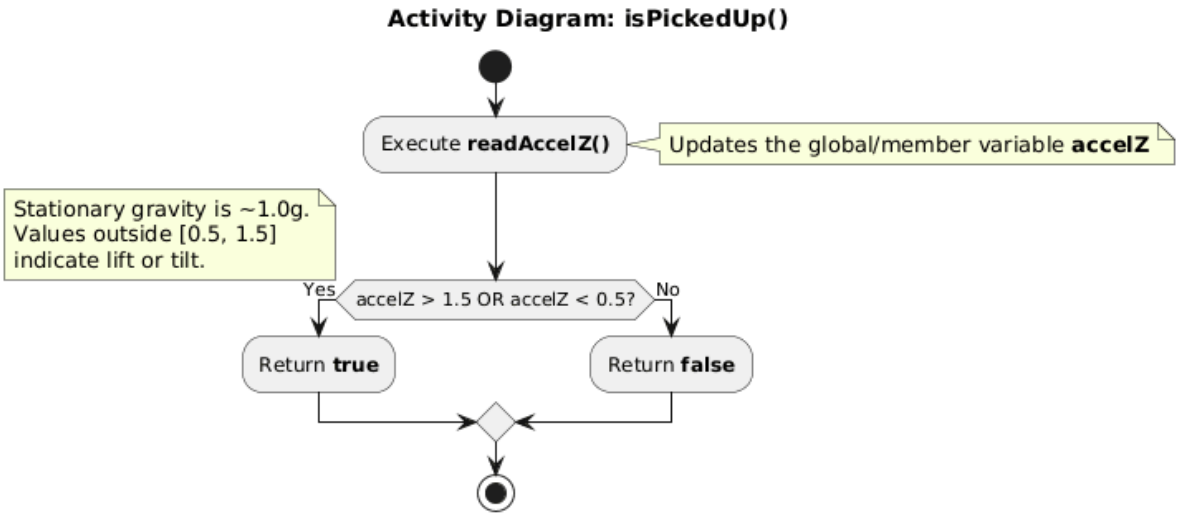
Activity Diagram: handleServerData

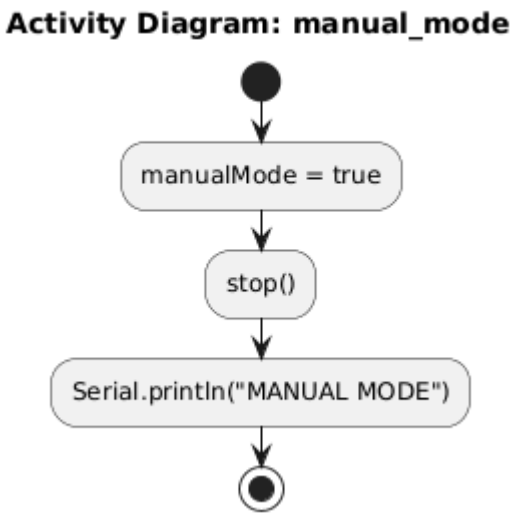


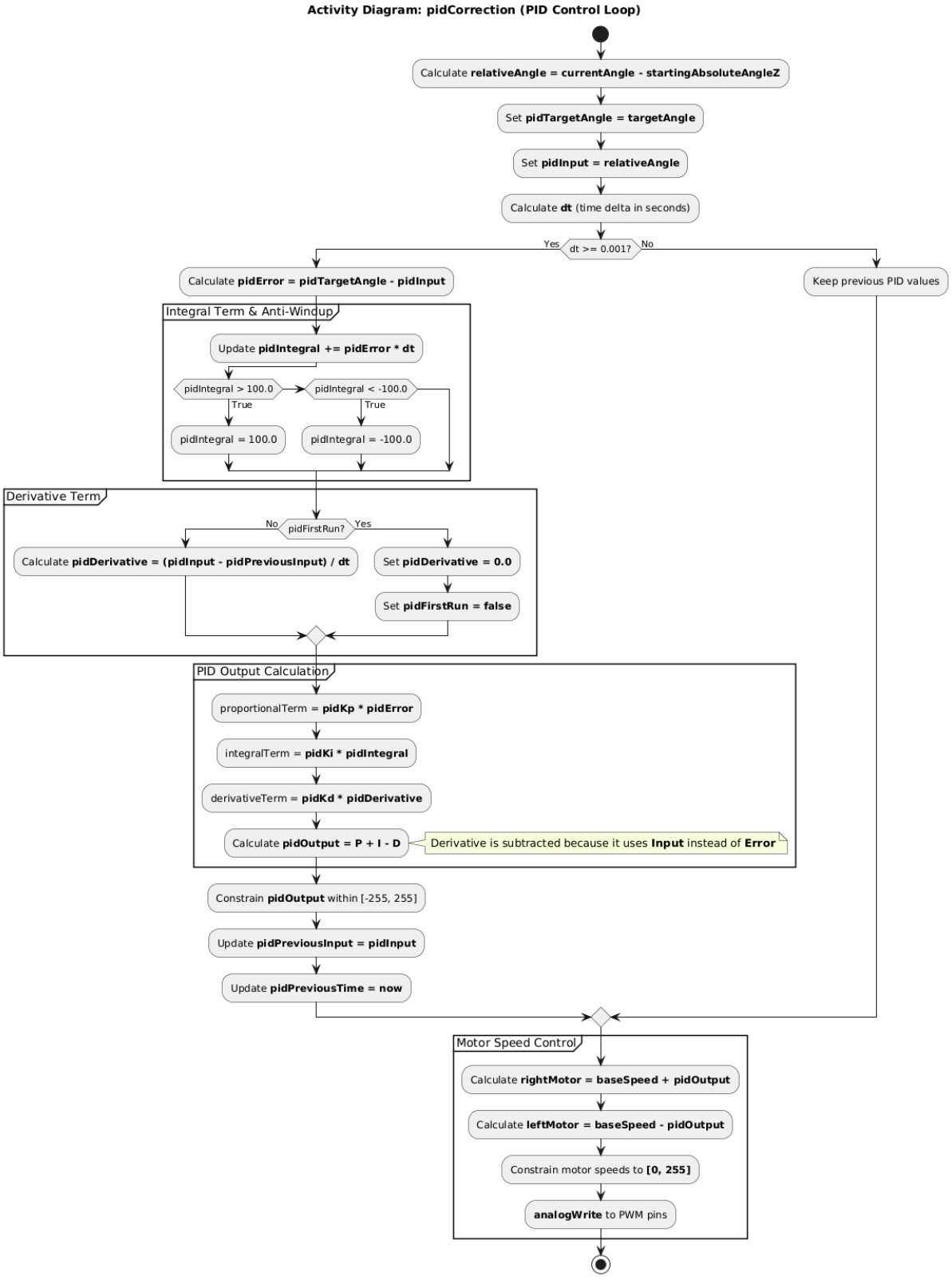
Activity Diagram: handleServerRoot & Web UI Logic



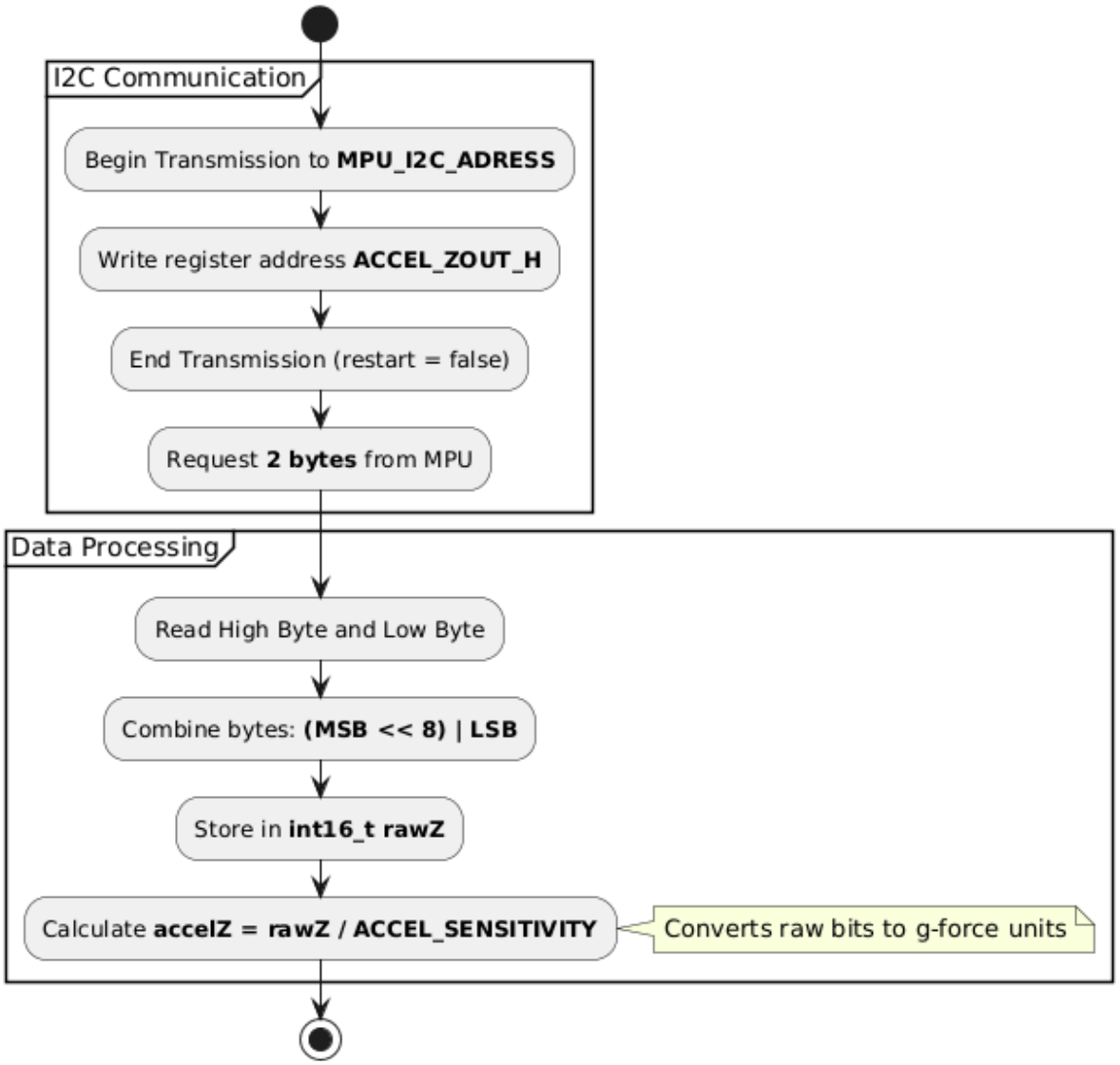

Inference Task Flowchart
Enter infinite while(1) loop for FreeRTOS task
True
xSemaphoreTake(FrameMutex, 500ms)
Success
Timeout
Initialize signal.total_length = 9216
Print "Inference task: Semaphore timeout"
Set signal.get_data = &convertBufferForAI
Initialize result struct
Execute run_classifier(&signal, &result, false)
res == EI_IMPULSE_OK
Yes
No
Print "Inference done" + classification time
Print "run_classifier failed" with error code
Set foundObject = false
Set previousLabelX = -1, previousLabelY = -1
ix = 0
Set previousLabel = "Error"
Get bounding box bb = result.bounding_boxes[ix]
bb.value > 0.2f
Threshold Met
No
Print detection details
ix++
Update previousLabelX = bb.x
Update previousLabelY = bb.y
Update previousLabel = bb.label
Set foundObject = true
ix < result.bounding_boxes_count?
Yes
foundObject == false
True
Print "No objects detected"
Set previousLabelX = -1, previousLabelY = -1
Set previousLabel = "Searching..."
xSemaphoreGive(FrameMutex)
vTaskDelay(50ms)

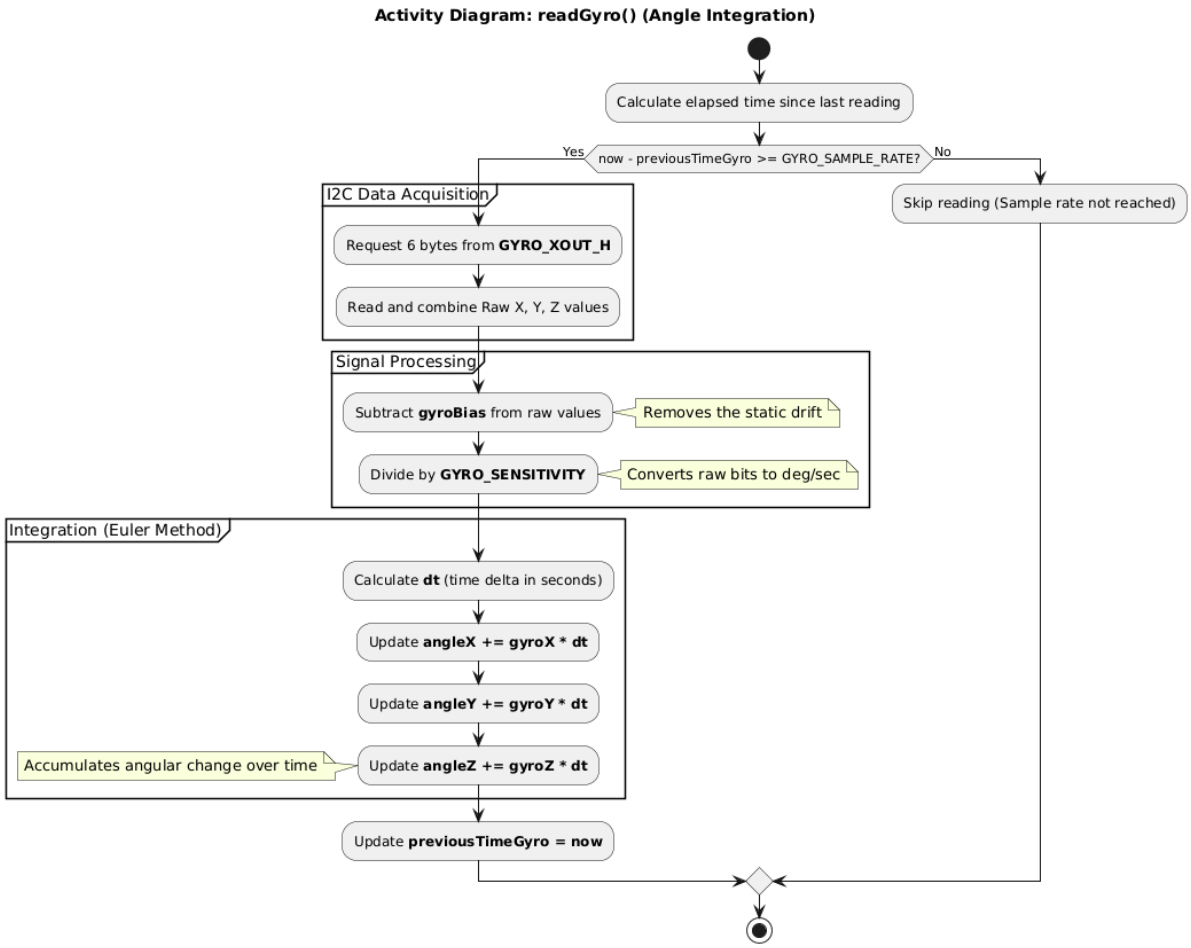

Activity Diagram: isPickedUp()
Execute readAccelZ()
Updates the global/member variable accelZ
Stationary gravity is ~1.0g.
Values outside [0.5, 1.5]
indicate lift or tilt.
Yes
accelZ > 1.5 OR accelZ < 0.5?
No
Return true
Return false

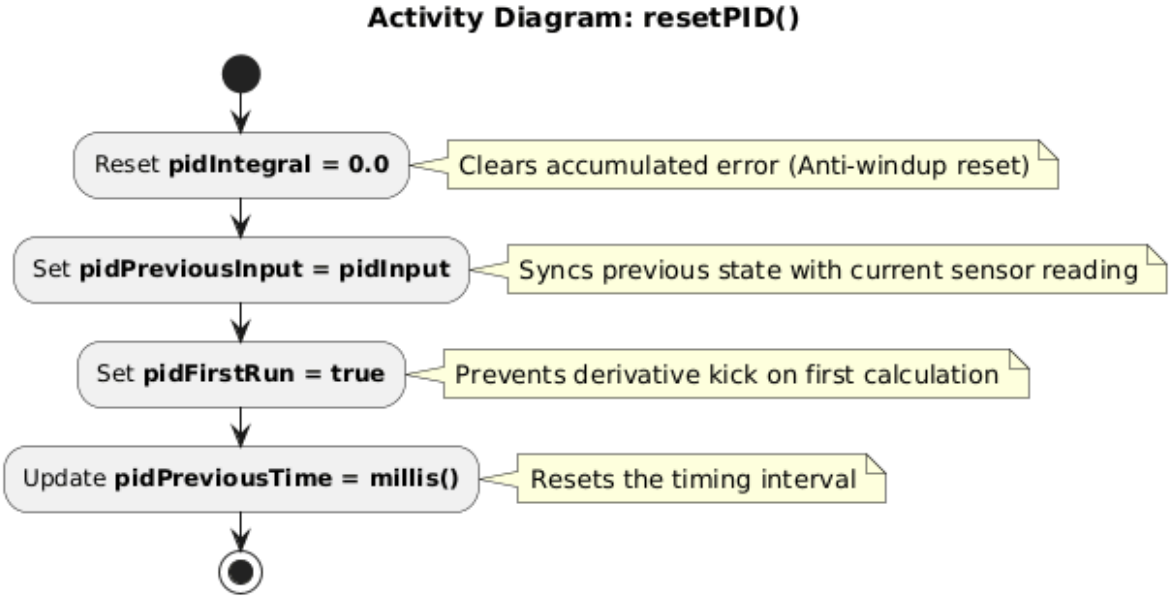

Activity Diagram: manual_mode
manualMode = true
stop()
Serial.println("MANUAL MODE")


Activity Diagram: pidCorrection (PID Control Loop)
Calculate relativeAngle = currentAngle - startingAbsoluteAngleZ
Set pidTargetAngle = targetAngle
Set pidInput = relativeAngle
Calculate dt (time delta in seconds)
dt >= 0.001?
Yes
No
Keep previous PID values
Calculate pidError = pidTargetAngle - pidInput
Integral Term & Anti-Windup
Update pidIntegral += pidError * dt
pidIntegral > 100.0
True
pidIntegral = 100.0
pidIntegral < -100.0
True
pidIntegral = -100.0
Derivative Term
pidFirstRun?
No
Yes
Calculate pidDerivative = (pidInput - pidPreviousInput) / dt
Set pidDerivative = 0.0
Set pidFirstRun = false
PID Output Calculation
proportionalTerm = pidKp * pidError
integralTerm = pidKi * pidIntegral
derivativeTerm = pidKd * pidDerivative
Calculate pidOutput = P + I - D
Derivative is subtracted because it uses Input instead of Error
Constrain pidOutput within [-255, 255]
Update pidPreviousInput = pidInput
Update pidPreviousTime = now
Motor Speed Control
Calculate rightMotor = baseSpeed + pidOutput
Calculate leftMotor = baseSpeed - pidOutput
Constrain motor speeds to [0, 255]
analogWrite to PWM pins

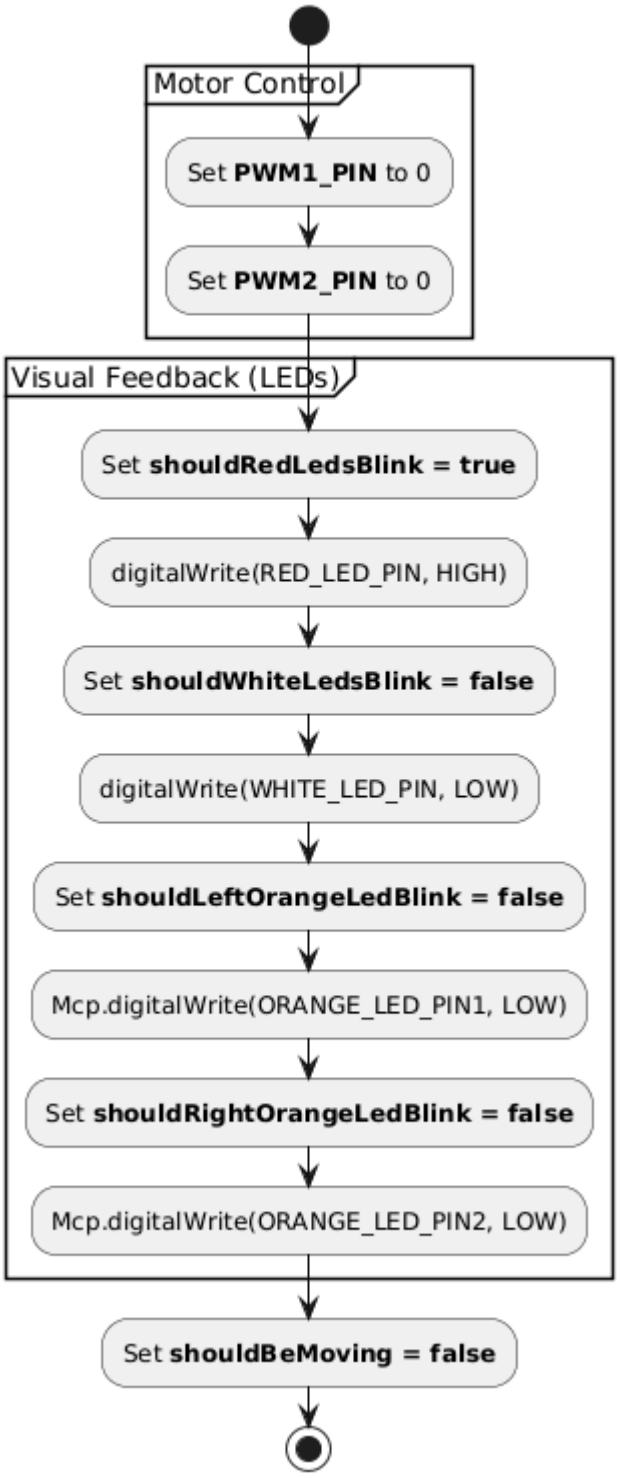
Activity Diagram: readAccelZ()

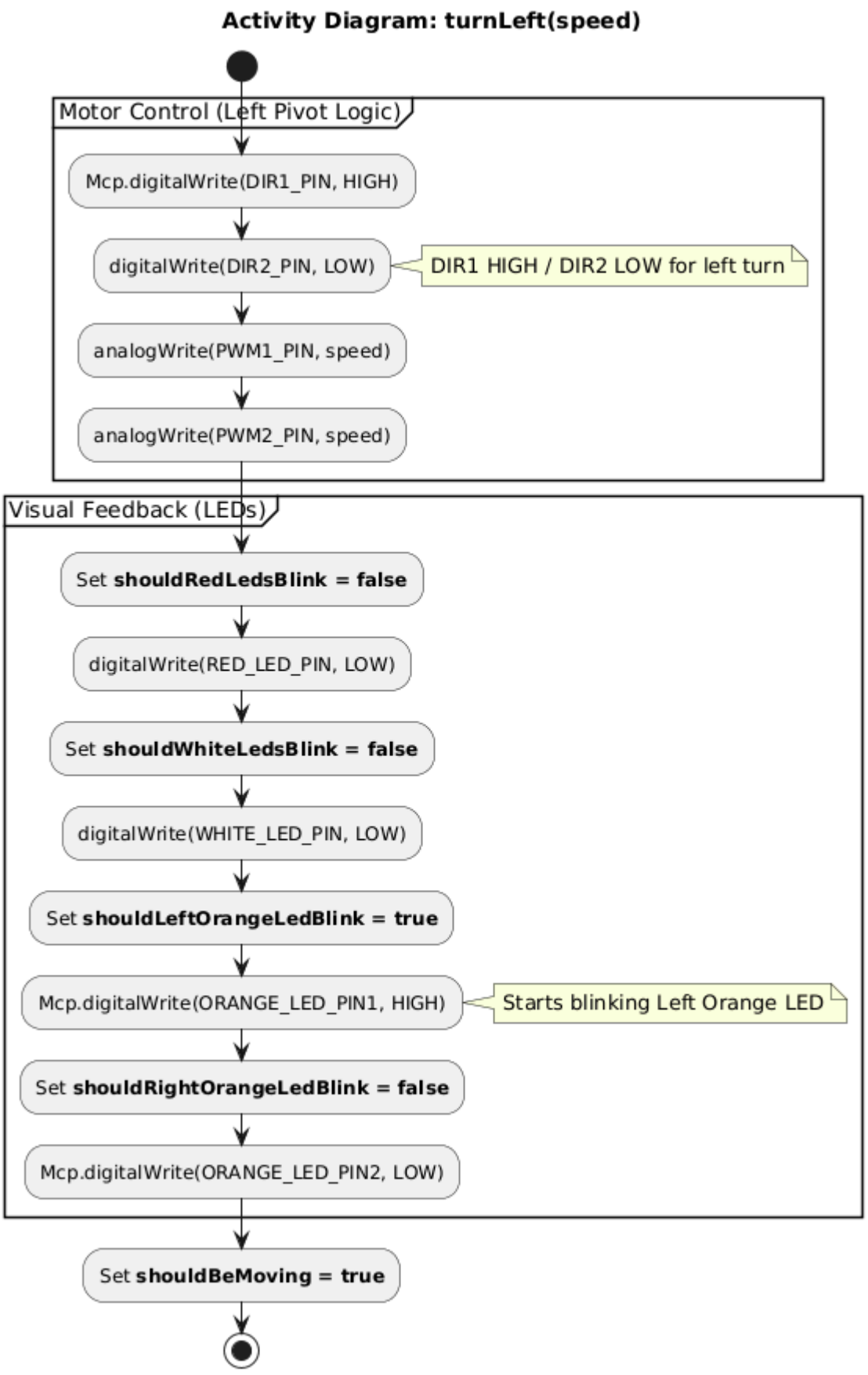


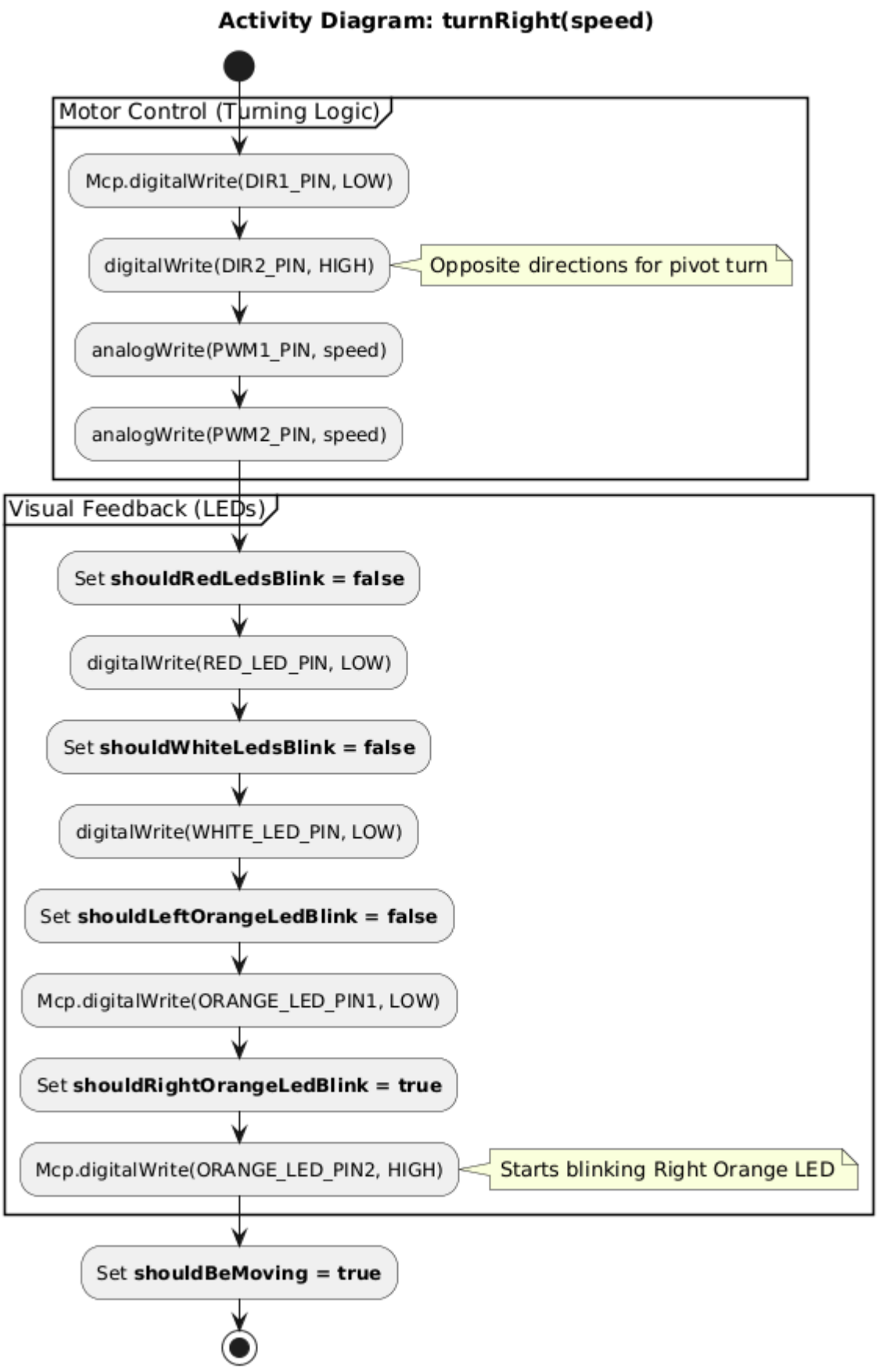

Activity Diagram: readGyro() (Angle Integration)
Calculate elapsed time since last reading
now - previousTimeGyro >= GYRO_SAMPLE_RATE?
Yes
No
Skip reading (Sample rate not reached)
I2C Data Acquisition
Request 6 bytes from GYRO_XOUT_H
Read and combine Raw X, Y, Z values
Signal Processing
Subtract gyroBias from raw values
Removes the static drift
Divide by GYRO_SENSITIVITY
Converts raw bits to deg/sec
Integration (Euler Method)
Calculate dt (time delta in seconds)
Update angleX += gyroX * dt
Update angleY += gyroY * dt
Accumulates angular change over time
Update angleZ += gyroZ * dt
Update previousTimeGyro = now


Activity Diagram: resetPID()
Reset pidIntegral = 0.0
Clears accumulated error (Anti-windup reset)
Set pidPreviousInput = pidInput
Syncs previous state with current sensor reading
Set pidFirstRun = true
Prevents derivative kick on first calculation
Update pidPreviousTime = millis()
Resets the timing interval

Activity Diagram: stop()




Activity Diagram: turnLeft(speed)
Motor Control (Left Pivot Logic)
Mcp.digitalWrite(DIR1_PIN, HIGH)
digitalWrite(DIR2_PIN, LOW)
DIR1 HIGH / DIR2 LOW for left turn
analogWrite(PWM1_PIN, speed)
analogWrite(PWM2_PIN, speed)
Visual Feedback (LEDs)
Set shouldRedLedsBlink = false
digitalWrite(RED_LED_PIN, LOW)
Set shouldWhiteLedsBlink = false
digitalWrite(WHITE_LED_PIN, LOW)
Set shouldLeftOrangeLedBlink = true
Mcp.digitalWrite(ORANGE_LED_PIN1, HIGH)
Starts blinking Left Orange LED
Set shouldRightOrangeLedBlink = false
Mcp.digitalWrite(ORANGE_LED_PIN2, LOW)
Set shouldBeMoving = true


Activity Diagram: turnRight(speed)
Motor Control (Turning Logic)
Mcp.digitalWrite(DIR1_PIN, LOW)
digitalWrite(DIR2_PIN, HIGH)
Opposite directions for pivot turn
analogWrite(PWM1_PIN, speed)
analogWrite(PWM2_PIN, speed)
Visual Feedback (LEDs)
Set shouldRedLedsBlink = false
digitalWrite(RED_LED_PIN, LOW)
Set shouldWhiteLedsBlink = false
digitalWrite(WHITE_LED_PIN, LOW)
Set shouldLeftOrangeLedBlink = false
Mcp.digitalWrite(ORANGE_LED_PIN1, LOW)
Set shouldRightOrangeLedBlink = true
Mcp.digitalWrite(ORANGE_LED_PIN2, HIGH)
Starts blinking Right Orange LED
Set shouldBeMoving = true

Ενσωμάτωση Τεχνητής Νοημοσύνης και Μηχανική Όραση

Η πλοήγηση ενός αυτόνομου οχήματος απαιτεί την ικανότητα αντίληψης και κατανόησης του περιβάλλοντος χώρου. Για την επίτευξη αυτού του στόχου, η εφαρμογή μας ενσωματώνει τεχνολογίες Τεχνητής Νοημοσύνης (Artificial Intelligence – AI), και συγκεκριμένα Μηχανικής Όρασης (Computer Vision), σε συνδυασμό με αισθητήρες υπερύθρων (IR) και υπερήχων (Ultrasonic).Ο μικροελεγκτής δεν εκτελεί απλώς προκαθορισμένες εντολές, αλλά «αντιλαμβάνεται» το περιβάλλον μέσω των αισθητήρων και λαμβάνει αποφάσεις σε πραγματικό χρόνο, όπως η αποφυγή εμποδίων, η ακολούθηση της διαγράμμισης και η αναγνώριση σημάτων.

Η επιλογή χρήσης τεχνητής νοημοσύνης αντί συμβατικού προγραμματισμού βασίστηκε κυρίως στην ανάγκη για προσαρμοστικότητα και στη μεγάλη πολυπλοκότητα του περιβάλλοντος. Οι παραδοσιακοί αλγόριθμοι επεξεργασίας εικόνας, όπως η ανίχνευση χρώματος ή τα φίλτρα γραμμών, είναι σχετικά άκαμπτοι. Μικρές μεταβολές στον φωτισμό, η παρουσία σκιάς ή παραμορφώσεις στο αντικείμενο μπορούν να οδηγήσουν σε αποτυχία αναγνώρισης. Αντίθετα, τα συστήματα AI, μέσω της διαδικασίας εκπαίδευσης, έχουν τη δυνατότητα να γενικεύουν και να αναγνωρίζουν αντικείμενα υπό διαφορετικές συνθήκες.

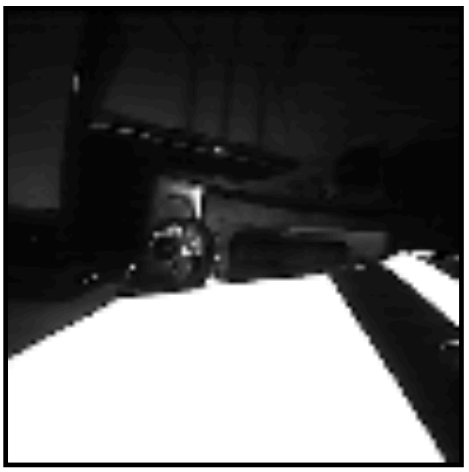
Επιπλέον, σε ένα δυναμικό περιβάλλον, οι κανόνες και οι εναλλακτικές καταστάσεις που θα έπρεπε να περιγραφούν μέσω συμβατικού προγραμματισμού θα ήταν πρακτικά άπειροι, καθιστώντας την προσέγγιση αυτή μη αποδοτική.

Στην αρχική σχεδίαση του συστήματος έγινε προσπάθεια χρήσης της κάμερας για την ακολούθηση της διαγράμμισης. Ωστόσο, λόγω μεταβαλλόμενων συνθηκών λειτουργίας και μετά από εκτεταμένο πειραματισμό, κρίθηκε ότι η χρήση αισθητήρων υπερύθρων (IR) αποτελεί πιο αξιόπιστη και αποδοτική λύση για την υλοποίηση της συγκεκριμένης λειτουργίας.

Η κύρια προσέγγιση που επιλέχθηκε ήταν η δυαδικοποίηση (binarization) της εικόνας με thresholding (δοκιμάστηκαν global, row-based και adaptive) που δρα ως το φίλτρο (μόνο στο region of interest που συναντώνται οι γραμμές), και υπολογισμός του deviation (πόσα μαύρα/άσπρα pixels βρίσκονται σε κάθε μεριά της εικόνας) για την εύρεση της γωνίας. Αυτή η απλουστευμένη προσέγγιση που τρέχει σε πραγματικό χρόνο αποδείχθηκε ευεπίφορη σε αλλαγές φωτισμού καθώς απαιτείται ένα ελεγχόμενο περιβάλλον στο οποίο η κάμερα να μπορεί να βλέπει συνεχώς και τις δύο γραμμές για να οριοθετεί σωστά.

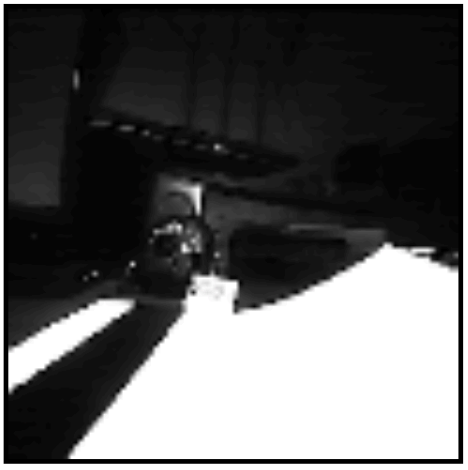
Λόγω των συνθηκών φωτισμού στη πίστα (πλάγιος φωτισμός από παράθυρα συνδυασμένος με μόνο μία ορατή γραμμή), η ανάπτυξη κάποιου πιο ανεπτυγμένου φίλτρου (Sobel, Canny) θα ήταν αναγκαία για οδομετρία με χρήση κάμερας.

Target Angle: -15.95 degrees





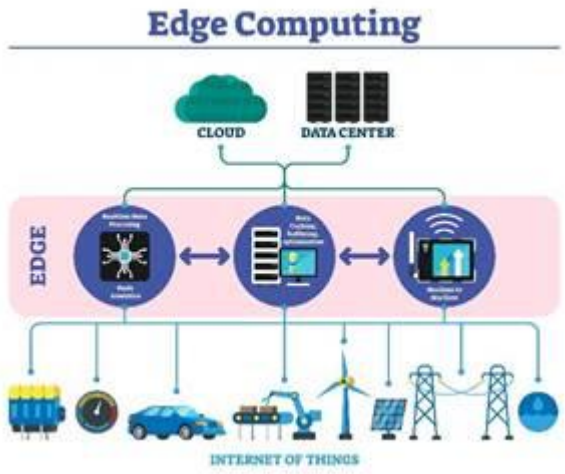
Target Angle: 16.13 degrees



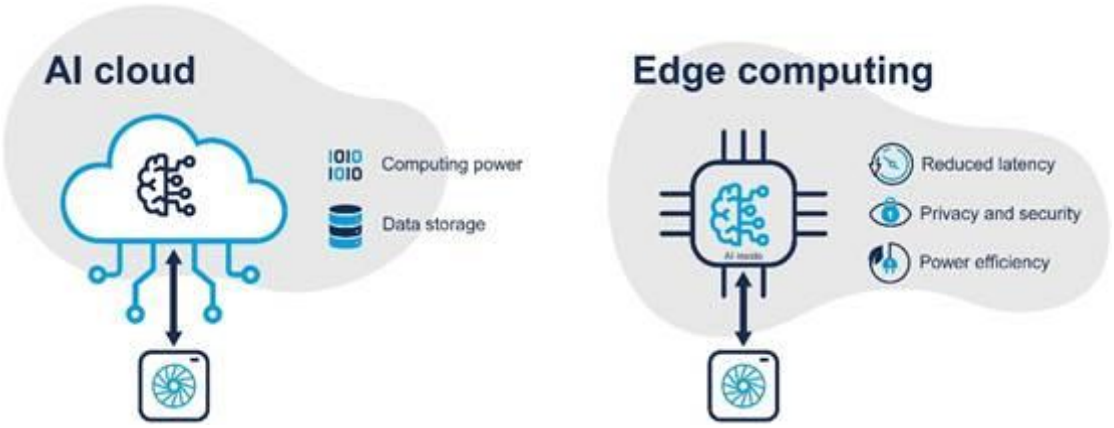


Edge AI

Για τη υλοποίηση του συστήματος επιλέχθηκε η προσέγγιση Edge AI: οι αλγόριθμοι μηχανικής μάθησης εκτελούνται τοπικά απευθείας στον μικροελεγκτή του οχήματος και όχι σε κάποιον απομακρυσμένο διακομιστή (Cloud).



Εικόνα 10: Δομή Cloud Edge Computing



Εικόνα 11: Δομή Edge AI Computing

Η προσέγγιση αυτή επιλέχθηκε για 3 κύριους λόγους:

1. Μηδενικός Χρόνος Απόκρισης (Low Latency):
   Σε ένα κινούμενο όχημα, ο χρόνος είναι ύψιστης σημασίας. Η αποστολή εικόνας στο Cloud και η αναμονή απάντησης θα δημιουργούσε καθυστερήσεις που θα οδηγούσαν σε συγκρούσεις. Το Edge AI προσφέρει άμεση απόκριση.

2. Αυτονομία & Συνδεσιμότητα:
   Το όχημα λειτουργεί ακόμη και χωρίς σύνδεση στο διαδίκτυο (offline).

3. Ιδιωτικότητα (Privacy):
   Τα δεδομένα εικόνας επεξεργάζονται στη συσκευή και δεν μεταδίδονται σε τρίτους.

4. Εκπαιδευτικός χαρακτήρας και ανάπτυξη μοντέλου:
   Η προσέγγιση αυτή επιλέχθηκε επίσης λόγω της ανάγκης εκπαίδευσης και κατανόησης της διαδικασίας δημιουργίας, εκπαίδευσης και ενσωμάτωσης ενός μοντέλου μηχανικής μάθησης από τους ίδιους τους δημιουργούς του συστήματος.

Χρήση Πλατφόρμας Edge Impulse

Για την ανάπτυξη, εκπαίδευση και ενσωμάτωση του μοντέλου Edge AI, χρησιμοποιήθηκε η πλατφόρμα Edge Impulse, το κορυφαίο περιβάλλον ανάπτυξης για Embedded Machine Learning (TinyML).



Εικόνα 12: Edge Impulse Logo

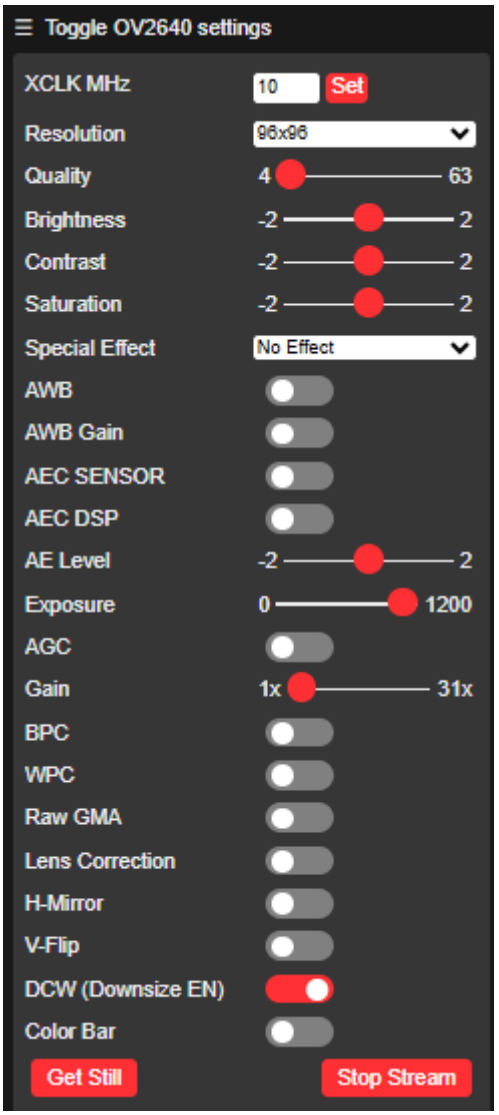
Η Edge Impulse παρέχει όλα τα απαραίτητα εργαλεία σε μία πλατφόρμα (End–to-End Workflow), δίνοντας μας τη δυνατότητα να ολοκληρώσουμε τη συλλογή δεδομένων, την επεξεργασία της εικόνας, την εκπαίδευση του μοντέλου και την μετατροπή του σε βελτιστοποιημένη βιβλιοθήκη, συμβατή με το Arduino IDE. Επίσης η βελτιστοποίηση των πόρων επιτρέπει τη δημιουργία ελαφριών μοντέλων που μπορούν να χρησιμοποιηθούν σε μικροελεγκτές με περιορισμένη μνήμη (RAM, SRAM) και επεξεργαστική ισχύ.

Διαδικασία Εκπαίδευσης Μοντέλου για Αναγνώριση Σήμανσης

Η ανάπτυξη του συστήματος μηχανικής όρασης μέσω της πλατφόρμας Edge Impulse ακολούθησε μια γραμμική διαδικασία τεσσάρων σταδίων, η οποία μετατρέπει τα ακατέργαστα δεδομένα (εικόνες) σε εκτελέσιμο κώδικα για τον μικροελεγκτή.

Πρωταρχικός στόχος ήταν το σύστημα να μπορεί να ανταποκριθεί σε πραγματικό χρόνο με τον βέλτιστο τρόπο. Για τον λόγο αυτό, οι ρυθμίσεις της κάμερας (OV2640) προσαρμόστηκαν

κατάλληλα ώστε να επιτευχθεί η χρυσή τομή μεταξύ ποιότητας εικόνας και ταχύτητας επεξεργασίας.



Εικόνα 13: Ρυθμίσεις κάμερας κατά τη διάρκεια λήψης δεδομένων

Συλλογή Δεδομένων (Data Acquisition)

Το πρώτο στάδιο αποτέλεσε η δημιουργία του συνόλου δεδομένων (Dataset). Λήφθηκαν φωτογραφίες από την κάμερα του οχήματος σε διάφορες γωνίες και συνθήκες φωτισμού και στη συνέχεια οι οποίες στη συνέχεια εισήχθησαν στο περιβάλλον του Edge Impulse.

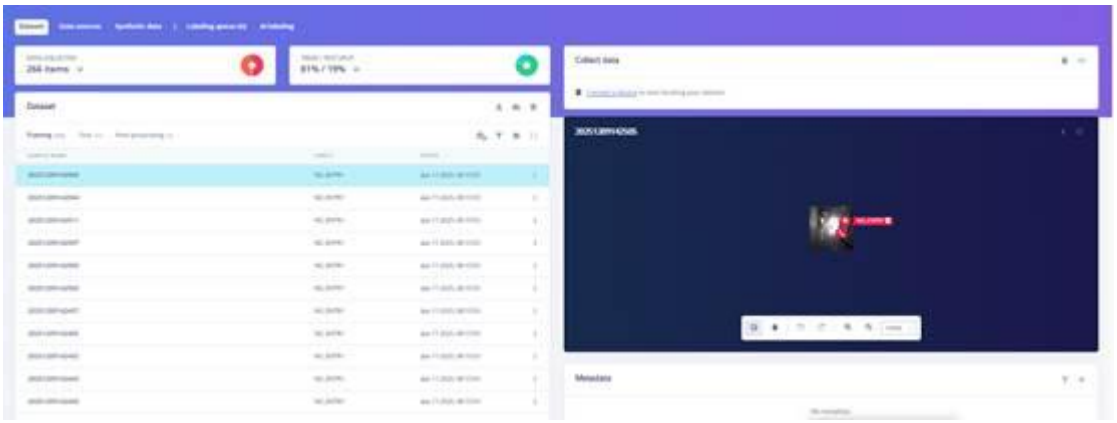


Εικόνα 14: Ενδεικτική εικόνα σήμανσης



Εικόνα 15: Ενδεικτική εικόνα Σήμανσης

Στη συνέχεια, μέσω του εργαλείου "Labeling Queue" της πλατφόρμας, ορίστηκαν τα πλαίσια (bounding boxes) γύρω από τα αντικείμενα ενδιαφέροντος και τους αποδόθηκαν οι αντίστοιχες ετικέτες (labels). Τα δεδομένα διαχωρίστηκαν αυτόματα σε σύνολο εκπαίδευσης (Training set - 81%) και σύνολο δοκιμής (Testing set - 19%).



Εικόνα 16: Dataset



Εικόνα 17:Labeling



Εικόνα 18: Labeling

Σχεδιασμός του Impulse (Create Impulse)

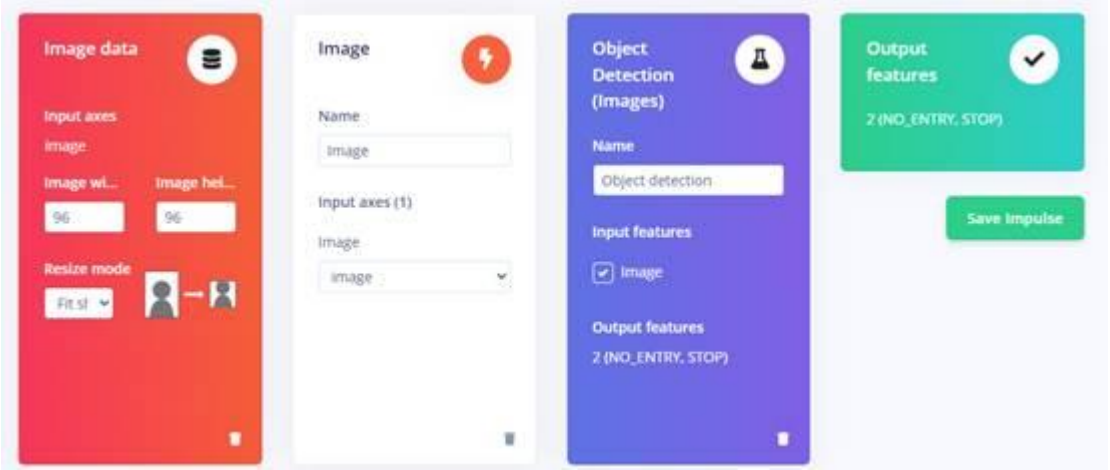
Στο στάδιο αυτό καθορίστηκε η αρχιτεκτονική της επεξεργασίας, γνωστή ως "Impulse". Η ροή αποτελείται από τρία επιμέρους μπλοκ:

1. Input Block:
   Οι εικόνες εισόδου μπορούν να υποβληθούν σε σμίκρυνση (resizing) και να γίνει μετατροπή τους σε κλίμακα του γκρι (Grayscale) για εξοικονόμηση μνήμης και υπολογιστικών πόρων.

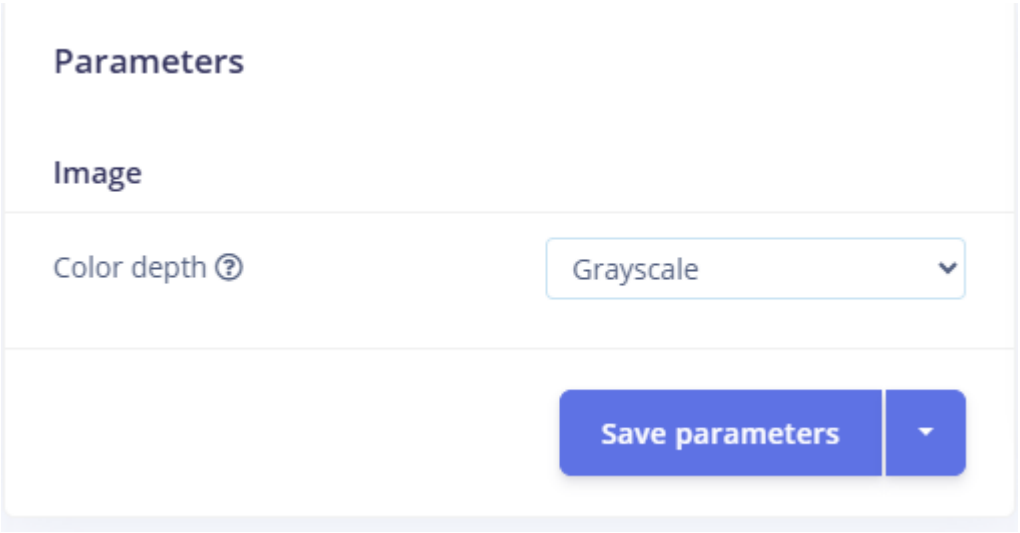
   _Στην περίπτωσή μας, παρόλο που οι φωτογραφίες είχαν ληφθεί εξαρχής σε ανάλυση 96x96 pixels και σε μορφή Grayscale, οι ρυθμίσεις επιβεβαιώθηκαν εκ νέου στο Input Block ώστε να διασφαλιστεί η απόλυτη συμβατότητα με τις βιβλιοθήκες και την αρχιτεκτονική του νευρωνικού δικτύου που παρέχει το Edge Impulse.

2. Processing Block (Image):
   Πραγματοποιείται ψηφιακή επεξεργασία σήματος (DSP) και εφαρμογή φίλτρων για την ανάδειξη των χαρακτηριστικών της εικόνας και την εξομάλυνση του θορύβου.

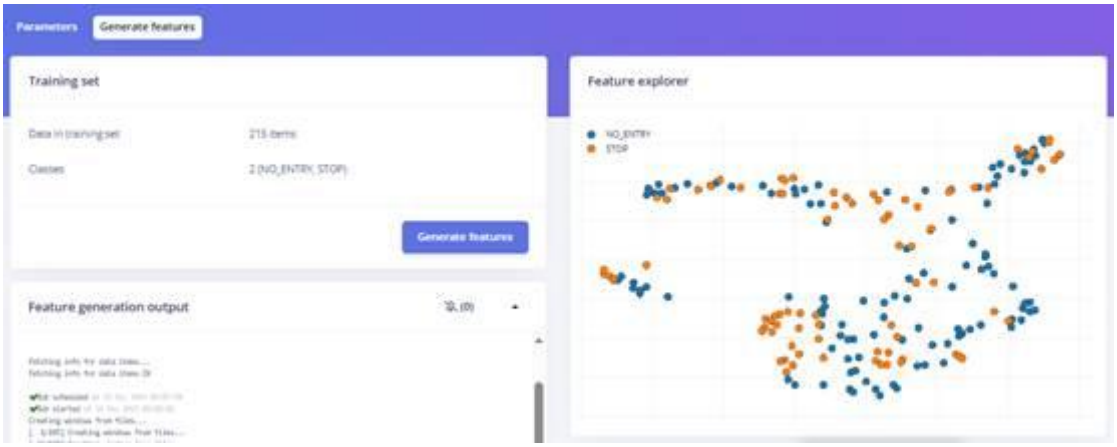
3. Learning Block (Object Detection):
   Εδώ επιλέχθηκε ο αλγόριθμος εκμάθησης. Αντί για κλασικά νευρωνικά δίκτυα ταξινόμησης (classification), επιλέχθηκε η μέθοδος εντοπισμού αντικειμένων (Object Detection) που είναι απαραίτητη για τον προσδιορισμό της θέσης της σήμανσης στον χώρο.



Εικόνα 19: Create impulse

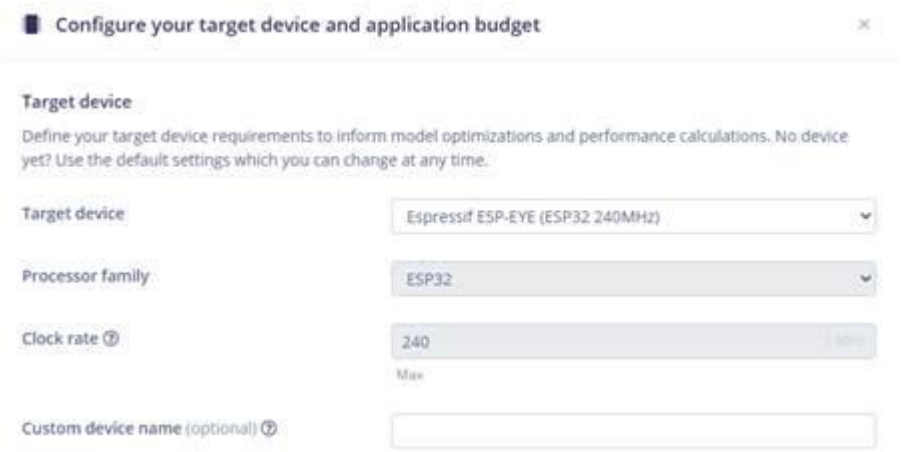


Εικόνα 20: Create Impulse



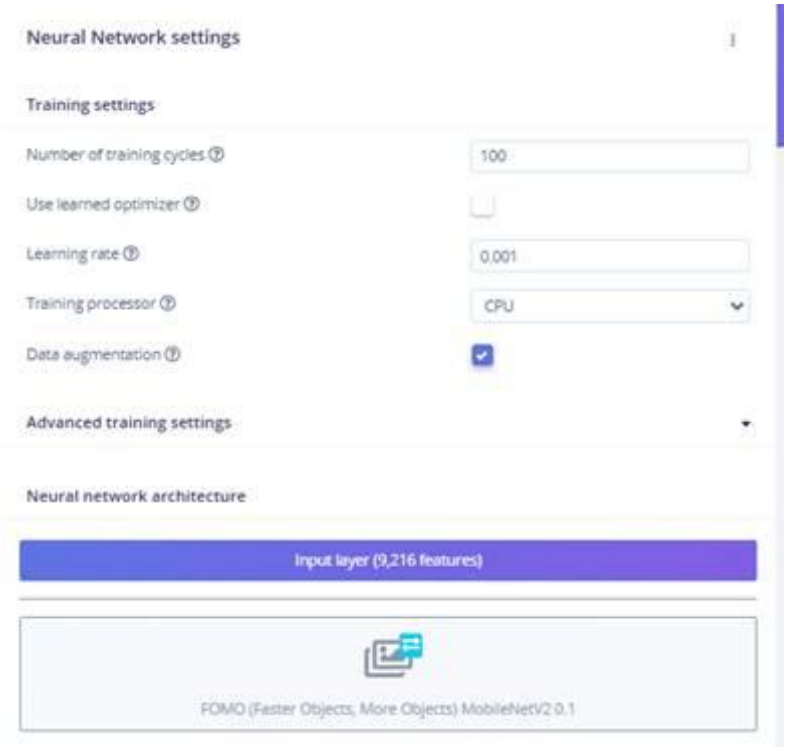
Εικόνα 21: Create Impulse

Επιπρόσθετα, στο πεδίο Target Device (Συσκευή Στόχος) επιλέχθηκε ο μικροελεγκτής που προσεγγίζει περισσότερο τα χαρακτηριστικά του δικού μας hardware, προκειμένου να έχουμε ακριβή εκτίμηση της καθυστέρησης (latency) και της κατανάλωσης μνήμης.



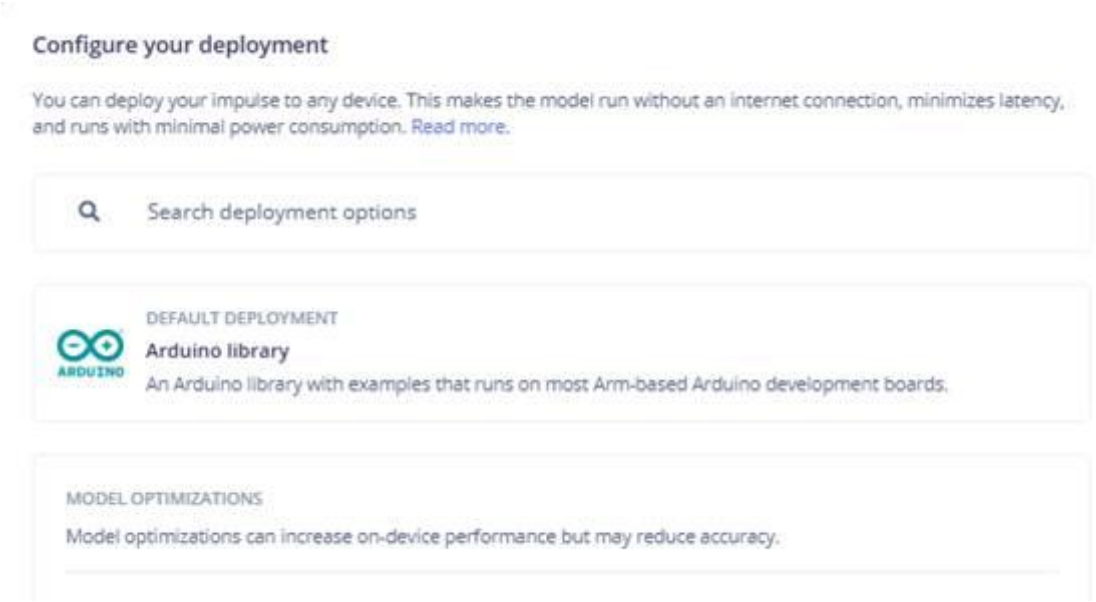
Εικόνα 22: Create Impulse

Ως μοντέλο εκπαίδευσης επιλέχθηκε το FOMO (Faster Objects, More Objects) MobileNetV2 0.1, ενώ οι κύκλοι εκπαίδευσης (epochs) ορίστηκαν στους 100, αριθμός που κρίθηκε επαρκής για την ελαχιστοποίηση του σφάλματος χωρίς να προκαλέσει υπερ-προσαρμογή (overfitting).



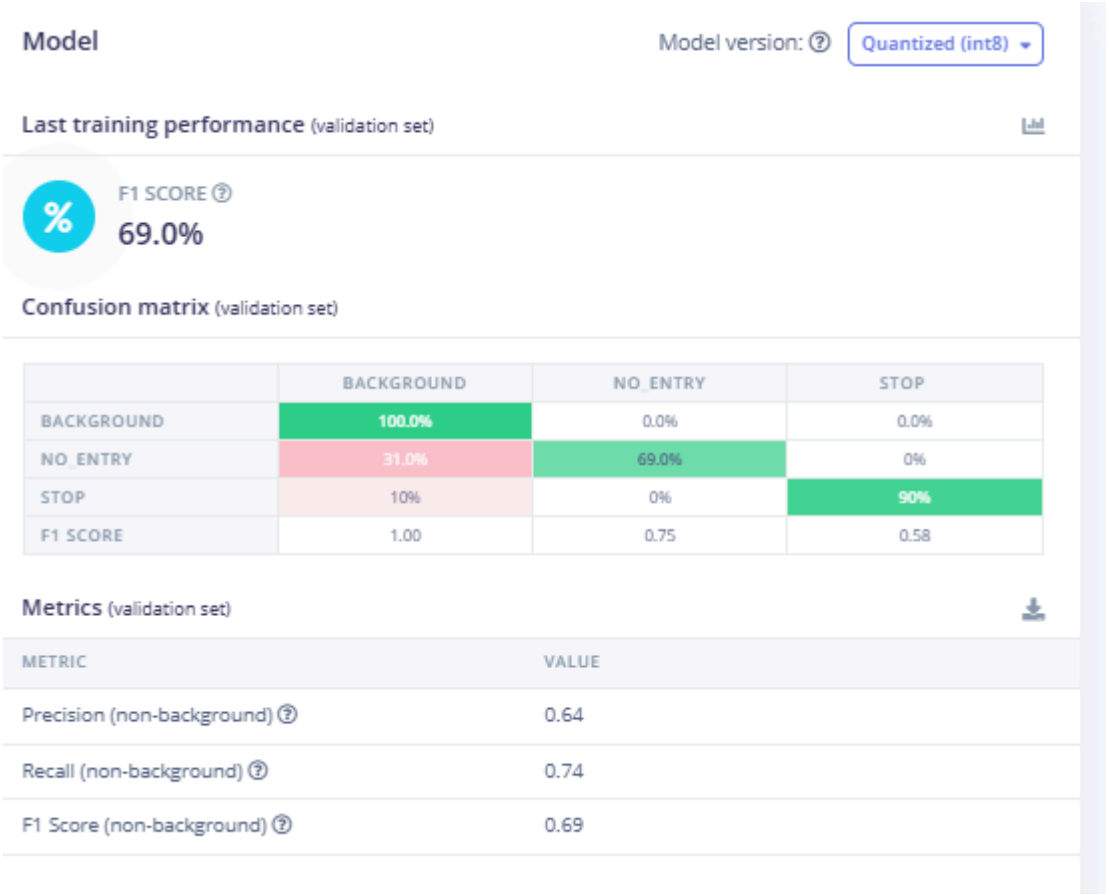
Εικόνα 23: Create Impulse

Τέλος έγινε Build και λήψη του μοντέλου ως Arduino Library.



Εικόνα 25: Deployment

Αξιολόγηση Μοντέλου



Model Model version: Quantized (int8)

Last training performance (validation set)

F1 SCORE
69.0%

Confusion matrix (validation set)

	BACKGROUND	NO_ENTRY	STOP
BACKGROUND	100.0%	0.0%	0.0%
NO_ENTRY	31.0%	69.0%	0%
STOP	10%	0%	90%
F1 SCORE	1.00	0.75	0.58

Metrics (validation set)

METRIC	VALUE
Precision (non-background)	0.64
Recall (non-background)	0.74
F1 Score (non-background)	0.69

Εικόνα 24: Τελικό Μοντέλο

Μετά την ολοκλήρωση της εκπαίδευσης του νευρωνικού δικτύου, η πλατφόρμα Edge Impulse παρείχε τα αποτελέσματα αξιολόγησης (Validation Set) με το μοντέλο να πετυχαίνει συνολικό F1 Score 69.0%.

Η βαθμολογία αυτή είναι ικανοποιητική για ένα σύστημα Edge AI που τρέχει σε μικροελεγκτή με περιορισμένους πόρους, καθώς αποδεικνύει ότι το δίκτυο έχει μάθει να διαχωρίζει τις κλάσεις, αν και υπάρχουν περιθώρια βελτίωσης. Αξίζει να σημειωθεί ότι χρησιμοποιήθηκε η έκδοση Quantized (int8) του μοντέλου. Αυτή η επιλογή μειώνει ελαφρώς την ακρίβεια σε σχέση με την έκδοση float32, αλλά είναι απαραίτητη για την ταχύτητα εκτέλεσης και την εξοικονόμηση μνήμης στο Arduino.

BACKGROUND (Φόντο): Ακρίβεια 100%
_Το μοντέλο αναγνωρίζει άψογα το "άδειο" περιβάλλον. Δεν υπήρξε καμία περίπτωση όπου

το φόντο να μπερδεύτηκε με πινακίδα (False Positive = 0%). Το όχημα δεν θα σταματάει αναίτια όταν δεν υπάρχει σήμανση στο δρόμο.

STOP: Ακρίβεια 90%
Η πινακίδα STOP αναγνωρίζεται σωστά στο 90% των περιπτώσεων. Μόνο ένα 10% των περιπτώσεων χαρακτηρίστηκε λανθασμένα ως Background. Το όχημα θα σταματάει σχεδόν πάντα όταν πρέπει.

NO_ENTRY (Απαγορευτικό): Ακρίβεια 69%
Εδώ παρατηρείται η χαμηλότερη απόδοση. Το μοντέλο αναγνωρίζει σωστά την πινακίδα στο 69% των περιπτώσεων, ενώ στο 31% την μπερδεύει με το Background (False Negative). Αυτό το σφάλμα πιθανώς οφείλεται σε παρόμοια χρώματα με το περιβάλλον ή σε λήψεις από μακρινή απόσταση όπου τα χαρακτηριστικά της πινακίδας δεν ήταν ξεκάθαρα. Στην πράξη, αυτό σημαίνει ότι το όχημα μπορεί να χρειαστεί να πλησιάσει λίγο περισσότερο την πινακίδα για να την αναγνωρίσει έγκυρα.

Ένας σημαντικός παράγοντας που αιτιολογεί την αισθητά υψηλότερη ακρίβεια της πινακίδας STOP (90%) έναντι του NO_ENTRY (69%) είναι η γεωμετρική τους ιδιαιτερότητα:

Η πινακίδα **STOP** έχει σχήμα οκταγώνου. Οι έντονες γωνίες και οι πολλαπλές ευθείες ακμές αποτελούν ισχυρά χαρακτηριστικά (features) που είναι εύκολα αναγνωρίσιμα από τα συνελικτικά φίλτρα του νευρωνικού δικτύου και σπάνια συναντώνται τυχαία στο φυσικό περιβάλλον.

Αντίθετα, η πινακίδα **NO_ENTRY** είναι κυκλική. Το κυκλικό σχήμα είναι πολύ πιο κοινό στο περιβάλλον, γεγονός που αυξάνει την πιθανότητα το μοντέλο να δυσκολευτεί να τη διαχωρίσει από το φόντο ή να την μπερδέψει με θόρυβο, οδηγώντας στο χαμηλότερο ποσοστό επιτυχίας.

Πραγματικές Επιδόσεις Μοντέλου

Παρά τα ικανοποιητικά αποτελέσματα του μοντέλου σε θεωρητικό επίπεδο (κατά τη φάση εκπαίδευσης στην αντίστοιχη πλατφόρμα), τα αποτελέσματα σε πραγματικές συνθήκες λειτουργίας παρουσίασαν σημαντικές αποκλίσεις. Η διαφορά αυτή αποδίδεται κυρίως στη διαφοροποίηση των συνθηκών μεταξύ του περιβάλλοντος εκπαίδευσης και της πρακτικής εφαρμογής. Εκτιμάται ότι η χρήση μεγαλύτερου όγκου δεδομένων, συλλεγμένων σε πραγματικές συνθήκες, θα οδηγούσε σε πιο αξιόπιστα και ικανοποιητικά αποτελέσματα.
Ωστόσο η ταχύτητα του Inference στον μικροελεγκτή κυμαίνεται μεταξύ των 100ms και 150ms χωρίς περεταίρω optimizations (GCC flags, vectorization, acceleration από το ειδικό hardware του S3 για 128-bit operations), που είναι μία ικανοποιητική αρχή για ενσωματωμένα μοντέλα AI.
Το εύρος του απαιτούμενου dataset που εκπαιδεύεται το μοντέλο καθιστά τη κύρια πρόκληση για την υλοποίηση ενός custom image recognition model, το οποίο να τρέχει σε πραγματικό χρόνο σε μικροελεγκτή. Το δυναμικό περιβάλλον που συναντά ένα όχημα κάνει αυτό το βήμα ακόμη πιο δύσκολο συγκριτικά με ένα σταθερό σύστημα.
Η χρήση ενός προεκπαιδευμένου μοντέλου (YOLOv4, MobileNetV3) δεν δίνει ικανοποιητικά αποτελέσματα όταν τρέχει πάνω στον μικροελεγκτή για δύο βασικούς λόγους:

-Το μοντέλο έχει εκπαιδευτεί με “περιττά” δεδομένα (σήματα που δεν θα συναντήσει το σύστημα κατά τη λειτουργία), που καθιστούν το μοντέλο πιο απαιτητικό στη μνήμη και στο inference χωρίς κάποιο τρόπο να αφαιρεθούν για να απομονωθούν internally κατά το inference μόνο τα ζητούμενα labels. Ένα custom μοντέλο που θα είχε μόνο δύο labels θα ήταν πολύ πιο ελαφρύ υπολογιστικά γιατί το neural network του θα ήταν πολλές φορές μικρότερο.
-Ο τύπος δεδομένων που εκπαιδεύτηκε το μοντέλο δεν είναι ίδιος με του συστήματος, συνήθως είναι RGB και σχετικά μεγάλες αναλύσεις άρα στο παρόν σύστημα που η ανάλυση είναι 96*96 και η εικόνα ασπρόμαυρη θα χρειαζόταν casting (μετατροπή από ένα τύπο δεδομένων σε έναν άλλον) και αλλαγή εύρους τιμών σε τουλαχιστον 9,216 bytes ανά frame capture και inference. Σε ένα custom μοντέλο που εκπαιδεύτηκε με αυτά τα δεδομένα μπορεί να γίνει inference απευθείας στο frame της κάμερας χωρίς καμία αναπροσαρμογή τους.



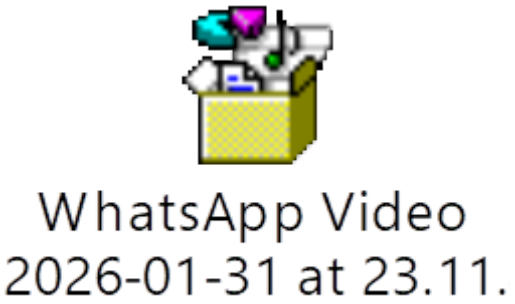
Εικόνα 26: Αποτέλεσμα testing μοντέλου στις συνθήκες στις οποίες γράφηκε ο κώδικας



Εικόνα 27: Αποτέλεσμα testing μοντέλου στις συνθήκες στις οποίες γράφηκε ο κώδικας



Εικόνα 28: Αποτέλεσμα testing μοντέλου στις συνθήκες στις οποίες γράφηκε ο κώδικας



Εικόνα 29: Αποτέλεσμα testing μοντέλου στις συνθήκες στις οποίες γράφηκε ο κώδικας

Αρχιτεκτονική FOMO και Μοντέλο MobileNetV2 0.1

Για την υλοποίηση της μηχανικής όρασης επιλέχθηκε η αρχιτεκτονική FOMO, μια καινοτόμος προσέγγιση ανίχνευσης αντικειμένων (Object Detection) σχεδιασμένη ειδικά για ενσωματωμένα συστήματα με περιορισμένους πόρους (constrained devices).

Σε αντίθεση με τα παραδοσιακά μοντέλα ανίχνευσης, όπως το YOLO (You Only Look Once) ή το MobileNet-SSD, τα οποία προσπαθούν να προβλέψουν το ακριβές μέγεθος και το σχήμα ενός πλαισίου οριοθέτησης (bounding box) γύρω από το αντικείμενο, το FOMO υιοθετεί μια απλούστερη και αποδοτικότερη στρατηγική:

1. Spatial Reduction (Από - χωρικοποίηση):
   Το μοντέλο δεν επεξεργάζεται την εικόνα ως ένα ενιαίο σύνολο για να βγάλει κουτιά, αλλά τη διαιρεί σε ένα πλέγμα (grid).

2. Per-cell Classification (Ταξινόμηση ανά Κελί):
   Για κάθε κελί του πλέγματος, το δίκτυο υπολογίζει την πιθανότητα να περιέχει το κέντρο ενός αντικειμένου.

3. Αποτέλεσμα:
   Η έξοδος του μοντέλου δεν είναι τετράγωνα με διαστάσεις (ύψος/πλάτος), αλλά συντεταγμένες κέντρου (centroids) πάνω στο πλέγμα.

Αυτή η προσέγγιση μειώνει δραστικά τον όγκο των υπολογισμών, καθώς μετατρέπει το πρόβλημα από "ανίχνευση μεγέθους και θέσης" σε ένα πρόβλημα "τοπικής ταξινόμησης", επιτρέποντας στο μοντέλο να είναι ταχύτερο από τα συμβατικά μοντέλα SSD.

FOMO MobileNetV2 0.1

Το FOMO βασίζεται στη μέθοδο της Μεταφοράς Μάθησης (Transfer Learning). Χρησιμοποιεί ως "κορμό" (backbone) ένα προ-εκπαιδευμένο μοντέλο, το οποίο στην περίπτωσή μας είναι το MobileNetV2.

Το MobileNetV2 είναι ένα Συνελικτικό Νευρωνικό Δίκτυο (CNN) βελτιστοποιημένο για κινητές συσκευές. Ωστόσο, η παράμετρος "0.1" που συνοδεύει την επιλογή μας είναι κρίσιμης σημασίας:

Η Παράμετρος 0.1 αναφέρεται στον πολλαπλασιαστή πλάτους (width multiplier) του δικτύου.Επιλέγοντας την τιμή 0.1, ουσιαστικά μειώνουμε τον αριθμό των φίλτρων (filters) σε κάθε επίπεδο του νευρωνικού δικτύου στο 10% του αρχικού μοντέλου έχοντας έτσι τη βέλτιστη ισορροπία μεταξύ ακρίβειας και απόδοσης υλικού.

Η μείωση των φίλτρων οδηγεί σε δραματική μείωση των εκπαιδευόμενων παραμέτρων (weights). Αυτό επιτρέπει στο μοντέλο να χωράει άνετα στη μνήμη (RAM) του μικροελεγκτή, αφήνοντας χώρο για τον κώδικα οδήγησης και τους υπόλοιπους αισθητήρες.

Με λιγότερους υπολογισμούς ανά κύκλο, ο χρόνος που απαιτείται για την επεξεργασία μιας εικόνας μειώνεται σημαντικά (low latency). Αυτό είναι ζωτικής σημασίας για το όχημά μας, καθώς πρέπει να αντιδρά ακαριαία στη σήμανση ενώ κινείται.

Δεδομένου ότι τα αντικείμενα που καλούμαστε να αναγνωρίσουμε (σήμανση, εμπόδια) έχουν διακριτά σχήματα και χαρακτηριστικά, η μειωμένη πολυπλοκότητα του μοντέλου 0.1 είναι επαρκής για την επίτευξη υψηλής ακρίβειας, χωρίς να απαιτείται η "βαριά" έκδοση του MobileNet (π.χ. 1.0 ή 0.35).

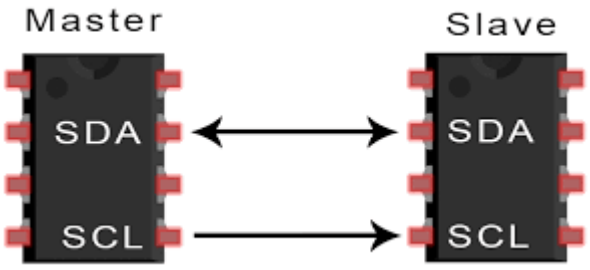
Συνοψίζοντας, ο συνδυασμός της αρχιτεκτονικής FOMO με το ελαφρύ δίκτυο MobileNetV2 0.1 δημιουργεί ένα σύστημα μηχανικής όρασης που λειτουργεί σε πραγματικό χρόνο (Real-time), με ελάχιστη κατανάλωση ενέργειας και μνήμης.

Πρωτόκολλο Επικοινωνίας I2C

Το I2C (Inter-Integrated Circuit) είναι ένα ευρέως διαδεδομένο σύγχρονο σειριακό πρωτόκολλο επικοινωνίας, το οποίο επιτρέπει την ανταλλαγή δεδομένων μεταξύ ενός μικροελεγκτή (Master) και πολλαπλών περιφερειακών συσκευών (Slaves), χρησιμοποιώντας μόνο δύο καλώδια.

Η επικοινωνία I2C βασίζεται σε δίαυλο (bus) δύο γραμμών:

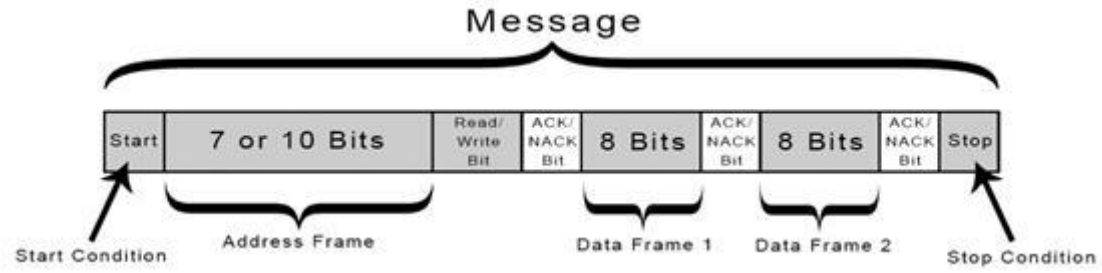
1. SDA (Serial Data Line): Η γραμμή μεταφοράς των δεδομένων (αμφίδρομη).
2. SCL (Serial Clock Line): Η γραμμή χρονισμού που συγχρονίζει τη μεταφορά των bits (ελεγχόμενη από τον Master).



Εικόνα 26: I2C Communication

Η βασική αρχή λειτουργίας στηρίζεται στη σχέση Master - Slave:

1. Ο μικροελεγκτής (Master) ξεκινά την επικοινωνία στέλνοντας ένα σήμα εκκίνησης (Start condition).
2. Στέλνει τη μοναδική διεύθυνση (Address - συνήθως 7-bit) της συσκευής με την οποία θέλει να επικοινωνήσει.
3. Όλες οι συσκευές στο κύκλωμα «ακούν», αλλά απαντά μόνο η συσκευή (Slave) που αναγνωρίζει τη διεύθυνσή της, στέλνοντας ένα σήμα επιβεβαίωσης (ACK).
4. Ακολουθεί η ανταλλαγή δεδομένων και ο τερματισμός της επικοινωνίας (Stop condition).



Εικόνα 27: I2C Communication

Οι κύριοι λόγοι που οδήγησαν στη χρήση του στην εφαρμογή μας είναι η οικονομία των πόρων, η επεκτασιμότητα και η απλότητα καλωδίωσης:

Για κάθε σύνδεση απαιτούνται μόνο δύο ακίδες (Pins) του μικροελεγκτή, αφήνοντας ελεύθερες τις υπόλοιπες ψηφιακές και αναλογικές εισόδους για τους κινητήρες και άλλους αισθητήρες.

Στις ίδιες δύο γραμμές μπορούν να συνδεθούν παράλληλα πολλαπλές συσκευές (π.χ. οθόνη OLED, γυροσκόπιο, επιπλέον αισθητήρες απόστασης), αρκεί να έχουν διαφορετική διεύθυνση.

Μειώνεται την πολυπλοκότητα των καλωδιώσεων πάνω στο σασί του οχήματος.

I2C στην κατασκευή μας

Στο αυτόνομο μας όχημα, το I2C είναι ζωτικής σημασίας καθώς διαχειρίζεται τρία κεντρικά υποσυστήματα ταυτόχρονα:

Συσκευή (Device)	**Τύπος**	**I2C Address (Hex)**	**Ρόλος**
ESP32/Arduino	Master	-	Ελέγχει το δίαυλο και συγχρονίζει τις συσκευές.
LCD Screen	Slave	0x27	Προβολή μηνυμάτων (IP, Mode).
MPU6050 Gyro	Slave	0x6B	Δεδομένα προσανατολισμού & επιτάχυνσης.

OV2640	Slave	??	Ρυθμίσεις αισθητήρα εικόνας.

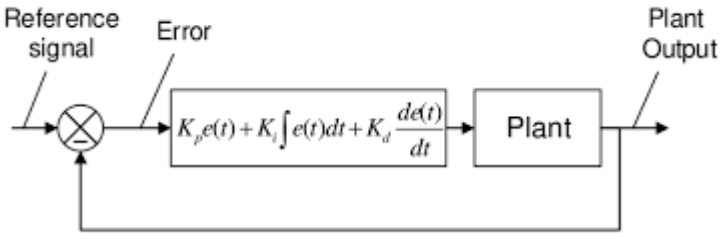
Η κάμερα OV2640 χρησιμοποιεί το πρωτόκολλο SCCB (Serial Camera Control Bus), μια ειδική παραλλαγή του I2C, για τη διαχείριση των ρυθμίσεών της. Ο δίαυλος αυτός δεν μεταφέρει την ίδια την εικόνα —διαδικασία που γίνεται παράλληλα για λόγους ταχύτητας— αλλά μόνο τις εντολές ελέγχου. Μέσω του SCCB, ο μικροελεγκτής διαμορφώνει τη λειτουργία της κάμερας, καθορίζοντας παραμέτρους όπως η ανάλυση της εικόνας, τα επίπεδα φωτεινότητας και αντίθεσης (contrast), καθώς και το format των δεδομένων (JPEG).

Το γυροσκόπιο/επιταχυνσιόμετρο MPU6050 αποτελεί βασικό αισθητήρα πλοήγησης. Στέλνει συνεχώς δεδομένα στον μικροελεγκτή σχετικά με την επιτάχυνση στους 3 άξονες (X, Y, Z) και την γωνιακή ταχύτητα. Τα δεδομένα αυτά είναι απαραίτητα για να αντιλαμβάνεται το ρομπότ την κλίση του ή αν έχει εκτραπεί από την πορεία του.

Η οθόνη LCD χρησιμοποιείται για την οπτική επικοινωνία με τον χρήστη. Ο μικροελεγκτής στέλνει αλφαριθμητικούς χαρακτήρες για την εμφάνιση κρίσιμων πληροφοριών, όπως η διεύθυνση IP του οχήματος, η κατάσταση σύνδεσης (Wi-Fi status) ή μηνύματα σφάλματος κατά την αναγνώριση αντικειμένων.

Αλγόριθμος Ελέγχου PID (Proportional – Integral - Derivative)

Ο ελεγκτής PID (Αναλογικός - Ολοκληρωτικός - Διαφορικός) είναι ο πιο διαδεδομένος αλγόριθμος ελέγχου κλειστού βρόχου (closed-loop control) στη βιομηχανία και τη ρομποτική. Βασικός σκοπός του είναι η διατήρηση μιας μετρήσιμης μεταβλητής (π.χ. ταχύτητα τροχού ή γωνία κλίσης) σε μια επιθυμητή τιμή (Setpoint), διορθώνοντας συνεχώς το σφάλμα μεταξύ της επιθυμητής και της πραγματικής κατάστασης.



Εικόνα 28: PID

Η μαθηματική λειτουργία του αλγορίθμου βασίζεται στο άθροισμα τριών όρων:

$$u(t) = K_\mathrm{p} e(t) + K_\mathrm{i} \int_0^t e(\tau)\,\mathrm{d}\tau + K_\mathrm{d} \frac{\mathrm{d}e(t)}{\mathrm{d}t}$$

Εικόνα 29: PID

Όπου e(t) είναι το σφάλμα (Error = Setpoint - Actual Value).

Αναλογικός Όρος (P - Proportional): Εξετάζει το παρόν σφάλμα. Παράγει διόρθωση ανάλογη του τρέχοντος σφάλματος. Αν το σφάλμα είναι μεγάλο, η διόρθωση είναι μεγάλη.

Ολοκληρωτικός Όρος (I - Integral): Εξετάζει το συναθροισμένο παρελθοντικό σφάλμα. Αθροίζει τα σφάλματα που έχουν συσσωρευτεί με τον χρόνο. Χρησιμοποιείται για να εξαλείψει το μόνιμο σφάλμα (steady-state error) που ο όρος P δεν μπορεί να διορθώσει (π.χ. όταν το όχημα θέλει λίγο παραπάνω ισχύ για να υπερνικήσει την τριβή).

Διαφορικός Όρος (D - Derivative): Προβλέπει το μελλοντικό σφάλμα. Υπολογίζει τον ρυθμό μεταβολής του σφάλματος και προσπαθεί να αποτρέψει την υπερβολική διόρθωση (overshoot), προσφέροντας σταθερότητα και απόσβεση των ταλαντώσεων.

Οι κύριοι λόγοι που οδήγησαν στη χρήση του στην εφαρμογή μας είναι:

1. Το σύστημα δεν επηρεάζεται από τη πτώση τάσης μπαταρίας. Αντίθετα σε ένα σύστημα με απλό έλεγχο η μπαταρία αποφορτίζεται, η ίδια τιμή PWM αποδίδει χαμηλότερη τάση και άρα χαμηλότερη ταχύτητα. Το PID ανιχνεύει την πτώση των στροφών και αυξάνει αυτόματα την ισχύ για να διατηρήσει την διαφορά ταχυτήτων των μοτέρ ανάλογη στα νέα δεδομένα του συστήματος.

2. Το όχημα μπορεί να προσαρμόζεται άμεσα σε αλλαγές του περιβάλλοντος, όπως η κίνηση σε επιφάνειες με αυξημένη τριβή (π.χ. χαλί) ή σε ανηφορική κλίση, όπου απαιτείται περισσότερη ροπή για την ίδια ταχύτητα.

3. Το σύστημα μας επηρεάζεται λιγότερο από την ανομοιομορφία κινητήρων. Λόγω κατασκευαστικών ανοχών, κανένας κινητήρας DC δεν είναι απόλυτα

πανομοιότυπος με τον άλλον. Με την ίδια εντολή PWM, ο ένας κινητήρας ενδέχεται να περιστρέφεται ταχύτερα από τον άλλον, προκαλώντας εκτροπή από την ευθεία πορεία. Ο αλγόριθμος PID εξισορροπεί δυναμικά την ταχύτητα των δύο τροχών, εξασφαλίζοντας ευθυγράμμιση.

Ο αλγόριθμος PID λύνει αυτά τα προβλήματα προσαρμόζοντας δυναμικά την ισχύ που παρέχεται στους κινητήρες, ώστε η πραγματική ταχύτητα να ταυτίζεται πάντα με την επιθυμητή, ανεξάρτητα από τις εξωτερικές συνθήκες.

Πρωτόκολλο Επικοινωνίας UART

Το **UART** είναι η συντομογραφία του *universal asynchronous receiver/transmitter*, και είναι ένα κύκλωμα των υπολογιστών το οποίο διαμεσολαβεί στην σειριακή επικοινωνία υπολογιστών ή υπολογιστών με συσκευές (ή και Ενσωματωμένων υπολογιστικών συστημάτων - Embedded Computer Systems όπως μικροελεγκτές). Η ταχύτητα επικοινωνίας (ρυθμός μετάδοσης) μπορεί να παραμετροποιηθεί και μετριέται σε baud rate (bits/sec). Σήμερα το κύκλωμα UART είναι ενσωματωμένο ως κύκλωμα τσιπ μέσα στον υπολογιστή ή στην περιφερειακή συσκευή.

Μία συσκευή που χρησιμοποιεί UART παίρνει bytes δεδομένων και τα στέλνει σε σειριακή μορφή ως bits. Στον προορισμό ένα δεύτερο UART συλλέγει τα bits και δημιουργεί τα bytes δεδομένων που αποστάλθηκαν. Κάθε UART περιέχει ένα καταχωρητή ολίσθησης (*shift register*) ο οποίος χρησιμοποιείται για την μετατροπή της σειριακής σε παράλληλη μορφή (εγγραφή/διάβασμα των δεδομένων μέσα ένα καταχωρητή από λήψη ή για αποστολή - τα δεδομένα που λαμβάνονται/στέλνονται σε ένα UART θα πρέπει να αποθηκευτούν μέσα σε ένα καταχωρητή του συστήματος με παράλληλο τρόπο). Στην επικοινωνία με UART υπάρχουν εξωτερικά κυκλώματα που μετατρέπουν τα επίπεδα των volt (λογικά επίπεδα) του σήματος του καλωδίου σε αυτό που χρησιμοποιεί το UART.

Επικοινωνία Εφαρμογής με το Μικροελεγκτή

Οι μετρήσεις των αισθητήρων στέλνονται μεταξύ του ESP και της εφαρμογής σε πραγματικό χρόνο χρησιμοποιώντας WebSocket, ένα πρωτόκολλο επικοινωνίας υπολογιστών το οποίο παρέχει κανάλια πλήρους αμφίδρομης επικοινωνίας μέσω μίας TCP σύνδεσης.

Τα δεδομένα αποστέλλονται σε μορφή JSON (JavaScript Object Notation) για την εύκολη διαχείριση τους από την εφαρμογή.
JSON είναι ένα ανοικτό μορφότυπο το οποίο χρησιμοποιεί κείμενο, που μπορεί να διαβαστεί από τον άνθρωπο, για τη μετάδοση αντικειμένων δεδομένων που αποτελούνται από ζεύγη χαρακτηριστικών-τιμών και τύπου δεδομένων συστοιχιών (ή οποιασδήποτε άλλης σειριοποίησιμης τιμής). Πρόκειται για ένα πολύ κοινό μορφότυπο δεδομένων που χρησιμοποιείται για την ασύγχρονη επικοινωνία.

Η μετάδοση του video από τη κάμερα καθιστά ειδική περίπτωση και λόγω της κλίμακας των δεδομένων και της απαιτούμενης ταχύτητας μεταφοράς τους επιλέχθηκε διαφορετική προσέγγιση. Η κάμερα συμπιέζει την εικόνα σε μορφή JPEG και ο μικροελεγκτής ξεκινά έναν HTTP Web Server στον οποίο μεταδίδεται συνεχόμενα εικόνα αν είναι συνδεδεμένος κάποιος client.
Για να μην περιορίζεται ο server σε ένα συγκεκριμένο IP address χρησιμοποιήθηκε mDNS με όνομα esp32s3-data για το server που περιέχει το stream.

Ανάλυση Κώδικα

Συμπεριλαμβανόμενες Βιβλιοθήκες

Εικόνα 34: Κώδικας

Για την εύρυθμη λειτουργία του αυτόνομου οχήματος και τη διαχείριση του πολύπλοκου υλικού (hardware), ο κώδικας κάνει χρήση ενός εκτενούς συνόλου βιβλιοθηκών. Οι βιβλιοθήκες αυτές καλύπτουν τέσσερις βασικούς πυλώνες: τη διαχείριση υλικού, τη δικτύωση, το πρωτόκολλο επικοινωνίας και την Τεχνητή Νοημοσύνη (AI).

Διαχείριση Κάμερας και Τροφοδοσίας

- **DFRobot_AXP313A.h**: Χρησιμοποιείται για την επικοινωνία με το ολοκληρωμένο διαχείρισης ενέργειας (PMIC) AXP313A. Είναι απολύτως απαραίτητο για την ενεργοποίηση της τροφοδοσίας της κάμερας OV2640.
- **esp_camera.h**: Η επίσημη βιβλιοθήκη οδηγών (drivers) της Espressif για τη λήψη δεδομένων (καρέ) από την κάμερα.
- **esp32-hal-i2c.h**: Βιβλιοθήκη χαμηλού επιπέδου (low-level) για τον δίαυλο I2C. Χρησιμοποιείται στρατηγικά για να παρακάμψει τις προεπιλεγμένες ρυθμίσεις της βιβλιοθήκης της κάμερας, αποτρέποντας διενέξεις (conflicts) με τους υπόλοιπους αισθητήρες που μοιράζονται τον ίδιο δίαυλο I2C.

Δικτύωση και Διακομιστής Ιστού

- `WiFi.h` & `WiFiMulti.h`: Αναλαμβάνουν τη σύνδεση του μικροελεγκτή στο τοπικό δίκτυο Wi-Fi. Η `WiFiMulti` επιτρέπει την καταχώρηση πολλαπλών δικτύων, ώστε το όχημα να συνδέεται αυτόματα σε όποιο είναι διαθέσιμο (fallback functionality)

- `WebServer.h`: Παρέχει τη δυνατότητα δημιουργίας ενός τοπικού διακομιστή (HTTP Server) για την εξυπηρέτηση της ιστοσελίδας, τη μετάδοση της ροής βίντεο (stream) και την αποστολή των δεδομένων του AI (σε μορφή JSON) στον χρήστη.

- `ESPmDNS.h`: Επιτρέπει την επίλυση ονομάτων τοπικού δικτύου (mDNS), δίνοντας πρόσβαση στο όχημα μέσω ενός φιλικού URL (`esp32-data.local`) αντί για τη χρήση δυναμικής IP.

Τεχνητή Νοημοσύνη (Edge AI)

- **a0912_inferencing.h**: Η εξαγόμενη βιβλιοθήκη του εκπαιδευμένου μοντέλου μηχανικής μάθησης, η οποία δημιουργήθηκε μέσω της πλατφόρμας Edge Impulse.

- **edge-impulse-sdk/tensorflow/lite/c/common.h**: Αναλαμβάνει την τοπική εκτέλεση συμπερασμάτων (on-device inference) για την αναγνώριση αντικειμένων μέσω της κάμερας.

Αισθητήρες, Επικοινωνία και Διεπαφή Χρήστη

- **Wire.h**: Η βασική βιβλιοθήκη του Arduino/ESP32 για την επικοινωνία μέσω του πρωτοκόλλου I2C, απαραίτητη για το γυροσκόπιο (MPU6050), την οθόνη και το I/O expander.

- **TinyGPS++.h**: Σχεδιασμένη για την αποδοτική ανάλυση (parsing) των NMEA δεδομένων που λαμβάνονται από τη μονάδα GPS μέσω σειριακής (UART) επικοινωνίας.

- **Adafruit_MCP23X17.h**: Χρησιμοποιείται για τον έλεγχο του ολοκληρωμένου MCP23017, το οποίο επεκτείνει τις ψηφιακές θύρες του ESP32-S3 (I/O

Expander). Μέσω αυτού ελέγχονται τα LED, ο βομβητής και διαβάζονται οι αισθητήρες γραμμής (IR), καπνού και φωτιάς.

- **LiquidCrystal_I2C.h**: Οδηγεί την οθόνη υγρών κρυστάλλων (LCD 16x2), στην οποία προβάλλονται μηνύματα κατάστασης και προειδοποιήσεις συστήματος.

- **IRremote.h**: Υπεύθυνη για τη λήψη και αποκωδικοποίηση των σημάτων υπερύθρων, επιτρέποντας τον χειροκίνητο έλεγχο (manual mode) του οχήματος.

Pin Configuration, Global Variables and System State

Ο κώδικας ορίζει με σαφήνεια την αρχιτεκτονική του υλικού, κατανέμοντας τους πόρους του μικροελεγκτή ESP32-S3. Ένα βασικό χαρακτηριστικό της σχεδίασης είναι η υβριδική χρήση απευθείας ακροδεκτών (GPIOs) του ESP32 και η χρήση του επεκτάτη θυρών (I/O Expander) MCP23017, προκειμένου να εξοικονομηθούν πολύτιμες θύρες.

Σύστημα Όρασης (Camera OV2640): Οι ακροδέκτες της κάμερας είναι αυστηρά καθορισμένοι (π.χ. CAM_PIN_XCLK στο 45, ακροδέκτες δεδομένων Y2 έως Y9). Πραγματοποιήθηκε απενεργοποίηση των προεπιλεγμένων I2C ακροδεκτών της κάμερας (CAM_PIN_SIOD και CAM_PIN_SIOC ορισμένα σε -1), προκειμένου ο δίαυλος I2C να διατηρηθεί ελεύθερος για τους υπόλοιπους αισθητήρες.

Εικόνα 35: Κώδικας - Camera Pins Configuration

Πλοήγηση και Επικοινωνία (GPS & Gyro): Η μονάδα GPS συνδέεται στη σειριακή θύρα UART0 (RX: 44, TX: 43) με ταχύτητα επικοινωνίας 9600 baud. Το γυροσκόπιο/επιταχυνσιόμετρο MPU6050, καθώς και η οθόνη LCD, επικοινωνούν μέσω της διεύθυνσης I2C (0x68 και 0x27 αντίστοιχα).

Εικόνα 36: Κώδικας - GPS Pins Configuration

Κινητήρες (Motor Driver): Για τον έλεγχο των κινητήρων χρησιμοποιείται η τεχνική της Διαφορικής Οδήγησης (Differential Steering). Τα σήματα κατεύθυνσης (Direction) μοιράζονται μεταξύ του I/O Expander (DIR1_PIN = 15) και του ESP32 (DIR2_PIN = D7), ενώ η ταχύτητα (PWM) ελέγχεται απευθείας από το ESP32 (D3 και D6).

Αισθητήρες Περιβάλλοντος και Αποφυγής Εμποδίων: Ο αισθητήρας υπερήχων (Ultrasonic) συνδέεται στις θύρες D11 (Trigger) και D12 (Echo). Αντίθετα, οι αναλογικοί και ψηφιακοί αισθητήρες ασφαλείας, όπως της φλόγας (Pin 3), του καπνού (Pin 4) και οι υπέρυθροι ανιχνευτές γραμμής (IR Line Sensors, Pins 1 & 2), διαχειρίζονται αποκλειστικά από το κύκλωμα MCP23017

Οπτικοακουστικές Ενδείξεις: Τα κύρια LED (Κόκκινο, Λευκό) ελέγχονται από το ESP32 (Pins 16, 17), ενώ τα πορτοκαλί φλας αλλαγής πορείας και ο βομβητής (Buzzer) ανήκουν στο κύκλωμα του MCP23017.

Εικόνα 37: Κώδικας

Εικόνα 38: Κώδικας

Εικόνα 39: Κώδικας

Εικόνα 40: Κώδικας

Διαχείριση Μνήμης και Τεχνητής Νοημοσύνης: Η μεταβλητή currentRawImageCopy αποτελεί έναν ενδιάμεσο χώρο μνήμης (buffer) μεγέθους 96x96 pixels. Για την αποφυγή συγκρούσεων (race conditions) κατά την ταυτόχρονη ανάγνωση της εικόνας από τον Web Server και την ανάλυσή της από το μοντέλο AI, χρησιμοποιείται ένας μηχανισμός συγχρονισμού του FreeRTOS, ονόματι FrameMutex (τύπου SemaphoreHandle_t). Οι μεταβλητές previousLabelX, previousLabelY και previousLabel αποθηκεύουν τα αποτελέσματα (συντεταγμένες και ετικέτα) της τελευταίας επιτυχούς ταξινόμησης (inference).

Κινηματική Κατάσταση και Ελεγκτής PID: Οι μεταβλητές gyroX, Y, Z και οι γωνίες angleX, Y, Z διατηρούν την τρέχουσα θέση του οχήματος στον χώρο. Γίνεται χρήση συστήματος PID: οι pidKp, pidKi, pidKd και pidError, pidIntegral, pidDerivative (μεταβλητές σφάλματος), χρησιμοποιούνται για τον δυναμικό υπολογισμό της διόρθωσης της πορείας (pidOutput), εξασφαλίζοντας ομαλή και ευθύγραμμη κίνηση.

Τηλεμετρία και Σημαίες (Flags) Συστήματος: Μια σειρά από μεταβλητές τύπου boolean (π.χ. smoke, flame, obstacleDetected, isPickedUp, shouldBeMoving) λειτουργούν ως «σημαίες» (flags). Αυτές καθορίζουν τη λογική ροή του προγράμματος, επιτρέποντας στο όχημα να λαμβάνει άμεσες αποφάσεις — όπως η διακοπή της κίνησης σε περίπτωση ανίχνευσης εμποδίου κάτω από τα 5 εκατοστά (OBSTACLE_THRESHOLD_CM). Τέλος, μεταβλητές όπως latitude και longitude συλλέγουν τα δεδομένα του GPS για τη μετάδοσή τους (telemetry) στον απομακρυσμένο χρήστη.

Δημιουργία Αντικειμένων (Object Instantiation)

Εικόνα 41: Κώδικας

Για να μπορέσει το μικροελεγκτής να αλληλεπιδράσει πραγματικά με τα περιφερειακά του, απαιτείται η δημιουργία συγκεκριμένων objects των κλάσεων.
Τα αντικείμενα αυτά δηλώνονται σκόπιμα σε **καθολική εμβέλεια (global scope)**, δηλαδή στην αρχή του προγράμματος και έξω από τις βασικές συναρτήσεις setup() και loop(). Αυτό

γίνεται διότι τα συγκεκριμένα αντικείμενα αντιπροσωπεύουν κρίσιμους πόρους του συστήματος που πρέπει να είναι συνεχώς διαθέσιμοι και προσβάσιμοι από οποιαδήποτε υπο-συνάρτηση ή διεργασία (Task) του FreeRTOS καθ' όλη τη διάρκεια λειτουργίας του οχήματος.

WebServer Server(80);: Δημιουργεί το αντικείμενο του διακομιστή ιστού και το δεσμεύει (binds) στην προεπιλεγμένη θύρα επικοινωνίας HTTP (Θύρα 80). Μέσω αυτού του αντικειμένου φιλοξενείται η ιστοσελίδα διεπαφής και δρομολογούνται τα αιτήματα του χρήστη (π.χ. λήψη εικόνας, JSON δεδομένων).

DFRobot_AXP313A Axp;: Εγκαθιδρύει την επικοινωνία με το ολοκληρωμένο διαχείρισης ενέργειας (PMIC). Η κλήση αυτού του αντικειμένου είναι επιβεβλημένη, καθώς είναι το μοναδικό υπεύθυνο για την παροχή τάσης στην κάμερα OV2640. Χωρίς αυτό, η κάμερα δεν μπορεί να αρχικοποιηθεί.

WiFiMulti WifiMulti;: Δημιουργεί τον διαχειριστή δικτύων. Σε αντίθεση με την απλή σύνδεση σε ένα δίκτυο, το αντικείμενο αυτό επιτρέπει την αποθήκευση πολλαπλών δικτύων Wi-Fi (π.χ. κύριο δίκτυο και εφεδρικά hotspots) και τη δυναμική εναλλαγή σε αυτά σε περίπτωση απώλειας σήματος.

HardwareSerial GpsSerial(0); και TinyGPSPlus Gps;: Λειτουργούν συνεργατικά για τον εντοπισμό θέσης.

- Το GpsSerial(0) δεσμεύει το υποσύστημα υλικού UART0 (Σειριακή θύρα) του ESP32-S3 για την ασύγχρονη λήψη των πρωτογενών δεδομένων του GPS.
- Το αντικείμενο Gps της βιβλιοθήκης TinyGPSPlus αναλαμβάνει να αποκωδικοποιήσει (parse) αυτή τη ροή ασυνάρτητων χαρακτήρων (NMEA sentences) και να την μετατρέψει σε αξιοποιήσιμες μεταβλητές (συντεταγμένες lat, lng).

Adafruit_MCP23X17 Mcp;: Αντιπροσωπεύει τον I/O Expander. Η καθολική δήλωσή του είναι ζωτικής σημασίας, καθώς χρησιμοποιείται σχεδόν από κάθε συνάρτηση του προγράμματος, είτε για την κίνηση των κινητήρων (σήματα Direction), είτε για την ανάγνωση αισθητήρων (φλόγας, καπνού, IR), είτε για την οδήγηση των ενδεικτικών LED.

LiquidCrystal_I2C Lcd(LCD_I2C_ADRESS, 16, 2);: Αρχικοποιεί την οθόνη υγρών κρυστάλλων. Κατά τη δημιουργία του αντικειμένου, περνιούνται ως ορίσματα οι φυσικές παράμετροι του υλικού: η διεύθυνση I2C (π.χ. 0x27) και η γεωμετρία της οθόνης (16 στήλες, 2 γραμμές).

Υποσύστημα Ελέγχου Κίνησης: Συναρτήσεις Ελεγκτή PID

Για την ακριβή πλοήγηση του οχήματος και τη διόρθωση της πορείας του σε πραγματικό χρόνο, το σύστημα υλοποιεί έναν ελεγκτή PID (Αναλογικό - Ολοκληρωτικό - Παραγωγικό). Ο ελεγκτής αυτός αναλαμβάνει να συγκρίνει την επιθυμητή γωνία πορείας (Target Angle) με την τρέχουσα μετρούμενη γωνία (Current Angle) από το γυροσκόπιο και να υπολογίσει την απαραίτητη διόρθωση στις ταχύτητες των τροχών μέσω διαφορικής οδήγησης (Differential Steering).

Το υποσύστημα ελέγχεται από δύο βασικές συναρτήσεις:

Εικόνα 42: Κώδιακας - resetPID()

Η συνάρτηση resetPID() εκτελεί μια "εκκαθάριση" του ελεγκτή:

- Μηδενίζει τον Ολοκληρωτικό όρο (pidIntegral = 0.0), αποτρέποντας απότομες και ανεξέλεγκτες κινήσεις που θα προέκυπταν από παλιά συσσωρευμένα σφάλματα.
- Θέτει τη σημαία pidFirstRun = true ώστε η παράγωγος να υπολογιστεί σωστά στον επόμενο κύκλο.
- Συγχρονίζει τον χρόνο (pidPreviousTime = millis()).

Εικόνα 43:Κώδικας - pidCorrection

Εικόνα 44: Κώδικας - pidCorrection

Η συνάρτηση pidCorrection() λαμβάνει τρία ορίσματα: την επιθυμητή γωνία (targetAngle), την τρέχουσα γωνία (currentAngle) και τη βασική ταχύτητα κίνησης (baseSpeed). Η εκτέλεσή της ακολουθεί τα εξής μαθηματικά και λογικά βήματα:

1. **Υπολογισμός Σφάλματος και Χρόνου:** Αρχικά, υπολογίζεται η σχετική γωνία του οχήματος σε σχέση με τη γωνία εκκίνησης και ο χρόνος dt (σε δευτερόλεπτα) που έχει παρέλθει από τον προηγούμενο υπολογισμό. Το σφάλμα ορίζεται ως η διαφορά: e(t) = Target – Input

2. **Υπολογισμός των τριών Όρων (P, I, D):**

   - **Αναλογικός Όρος (Proportional - P):** Πολλαπλασιάζει το τρέχον σφάλμα με τη σταθερά Kp. Είναι υπεύθυνος για την άμεση αντίδραση του συστήματος.
   - **Ολοκληρωτικός Όρος (Integral - I):** Συσσωρεύει το σφάλμα στο χρόνο (edt). Στον κώδικα περιλαμβάνεται ένας κρίσιμος μηχανισμός Anti-Windup, ο οποίος περιορίζει την τιμή του pidIntegral στο εύρος [-100.0, 100.0]. Αυτό προστατεύει το όχημα από το να αντιδράσει υπερβολικά αν εμποδιστεί μηχανικά να στρίψει για ώρα.
   - **Παραγωγικός Όρος (Derivative - D):** Υπολογίζεται η παράγωγος της pidInput. Αυτό γίνεται για να αποφευχθεί το φαινόμενο του "Derivative Kick" — ένα απότομο και βίαιο τίναγμα των κινητήρων όταν αλλάζει απότομα η επιθυμητή γωνία (targetAngle) μέσω τηλεχειριστηρίου ή των αισθητήρων γραμμής.

3. **Έξοδος και Διαφορική Οδήγηση:** Υπολογίζεται η τελική έξοδος**.**
   Στη συνέχεια, η έξοδος pidOutput (περιορισμένη στο εύρος [-255, 255] εφαρμόζεται στους δύο κινητήρες. Προστίθεται στον έναν και αφαιρείται από τον άλλον (baseSpeed + pidOutput και baseSpeed - pidOutput), αναγκάζοντας το όχημα να διαγράψει καμπύλη και να επιστρέψει στην επιθυμητή πορεία, διατηρώντας τη μέση ταχύτητά του σταθερή.

Γυροσκόπιο και το Επιταχυνσιόμετρο (MPU6050)

Αυτό το υποσύστημα είναι τα «μάτια» του οχήματος όσον αφορά την αντίληψη του χώρου (spatial awareness). Χωρίς αυτό, το όχημα δεν θα ήξερε προς τα πού κοιτάζει και το PID που αναλύσαμε πριν θα ήταν εντελώς άχρηστο.

Οι αισθητήρες τεχνολογίας MEMS (Μικρο-Ηλεκτρο-Μηχανικά Συστήματα), όπως το MPU6050, παρουσιάζουν εγγενώς ένα μικρό σφάλμα μέτρησης (Zero-Rate Offset ή Bias). Αυτό σημαίνει ότι, ακόμα και όταν το όχημα είναι εντελώς ακίνητο, το γυροσκόπιο μπορεί να αναφέρει μια μικρή, ψευδή περιστροφή.

Εικόνα 45: Κώδικας - gyroBegin()

Η συνάρτηση gyroBegin(): Είναι υπεύθυνη για την «αφύπνιση» του αισθητήρα, καθώς από προεπιλογή το MPU6050 εκκινεί σε κατάσταση αναμονής (sleep mode) για εξοικονόμηση ενέργειας. Η συνάρτηση γράφει την τιμή 0x00 στο register 0x6B, ενεργοποιώντας το τσιπ και τελειώνει την μετάδοση.

Εικόνα 46: Κώδικας - calibrateGyro()

Η συνάρτηση calibrateGyro(): Εκτελείται κατά την εκκίνηση του συστήματος (setup()). Λαμβάνει 100 διαδοχικές μετρήσεις από το γυροσκόπιο όσο το όχημα βρίσκεται σε απόλυτη ακινησία. Στη συνέχεια, υπολογίζει τον μέσο όρο αυτών των μετρήσεων για κάθε άξονα (X, Y, Z) και τον αποθηκεύει στις μεταβλητές gyroBiasX, gyroBiasY και gyroBiasZ. Αυτό το στατικό σφάλμα θα αφαιρείται αργότερα από κάθε πραγματική μέτρηση, αποτρέποντας τη σταδιακή συσσώρευση λάθους (Sensor Drift).

Εικόνα 47: Κώδικας - readGyro()

Λήψη Δεδομένων και Αριθμητική Ολοκλήρωση (readGyro): Η συγκεκριμένη συνάρτηση εκτελείται συνεχώς στην κύρια λούπα (loop) σε σταθερά χρονικά διαστήματα, τα οποία ορίζονται από τη σταθερά GYRO_SAMPLE_RATE (κάθε 10ms). Τα βήματα που ακολουθεί είναι κρίσιμα για τη μετατροπή των ακατέργαστων ηλεκτρικών σημάτων σε ωφέλιμα γεωμετρικά δεδομένα:

1. **Ανάγνωση:** Διαβάζει τα ακατέργαστα 16-bit δεδομένα των αξόνων απευθείας από τους καταχωρητές του αισθητήρα (ξεκινώντας από τον GYRO_XOUT_H).
2. **Αφαίρεση Σφάλματος & Κλιμάκωση:** Αφαιρεί τις τιμές Bias (που υπολογίστηκαν στην calibrateGyro()). Έπειτα, διαιρεί το αποτέλεσμα με τη σταθερά ευαισθησίας GYRO_SENSITIVITY (τιμή 131 LSB/deg/s. Πλέον, οι μεταβλητές αντιπροσωπεύουν Γωνιακή Ταχύτητα (ω) σε μοίρες ανά δευτερόλεπτο.
3. **Αριθμητική Ολοκλήρωση (Numerical Integration):** Το γυροσκόπιο μετράει πόσο γρήγορα περιστρέφεται το όχημα, όχι προς τα πού κοιτάζει. Για να βρεθεί η απόλυτη

γωνία (Angle), η συνάρτηση υπολογίζει τον χρόνο dt που πέρασε από την προηγούμενη μέτρηση (σε δευτερόλεπτα) .

Εικόνα 48: Κώδικας - readAccelZ()

Η συνάρτηση **readAccelZ()** διαβάζει την επιτάχυνση στον κατακόρυφο άξονα Z και τη μετατρέπει σε μονάδες βαρύτητας (g). Όταν το όχημα ακουμπά οριζόντια στο έδαφος, δέχεται σταθερά την επιτάχυνση της βαρύτητας, οπότε η μέτρηση είναι περίπου 1.0g.

Εικόνα 49: Κώδικας - isPickedUp()

Η συνάρτηση **isPickedUp()** ελέγχει αν αυτή η τιμή βγει εκτός των φυσιολογικών ορίων (> 1.5g ή < 0.5g). Αν συμβεί αυτό, στην πραγματικότητα υποδηλώνει ότι το όχημα σηκώθηκε από το έδαφος από κάποιον άνθρωπο ή έπεσε αλλά στην εφαρμογή μας χρησιμοποιείται για αν προσομοιάσομε σύγκρουση. Σε αυτή την περίπτωση επιστρέφει true, ενεργοποιώντας τη σημαία ασφαλείας (speedLimit = true) και ενημερώνοντας το κέντρο ελέγχου μέσω τηλεμετρίας.

Έλεγχος Κατεύθυνσης και Ταχύτητας

Προκειμένου να εξοικονομηθούν θύρες στο ESP32-S3 γίνεται υβριδικός έλεγχος των ακροδεκτών των κινητήρων:
Έλεγχος Κατεύθυνσης (Direction): Ο ακροδέκτης του αριστερού κινητήρα (DIR1_PIN) ελέγχεται ψηφιακά μέσω του I/O Expander (Mcp.digitalWrite), ενώ του δεξιού (DIR2_PIN) απευθείας από το ESP32 (digitalWrite). Ο συνδυασμός της λογικής κατάστασης (LOW ή HIGH) στους δύο αυτούς ακροδέκτες καθορίζει τη φορά περιστροφής των τροχών.

Έλεγχος Ταχύτητας (PWM): Η ταχύτητα ρυθμίζεται μέσω Διαμόρφωσης Εύρους Παλμού (Pulse Width Modulation - PWM) χρησιμοποιώντας την εντολή analogWrite στις θύρες PWM1_PIN και PWM2_PIN.

Εικόνα 50: Κώδικας – forward()

Εικόνα 51: Κώδικας - backward()

Ευθύγραμμη Κίνηση (forward & backward): Για την κίνηση προς τα εμπρός, αμφότεροι οι ακροδέκτες κατεύθυνσης τίθενται σε κατάσταση LOW, ενώ για την οπισθοπορεία τίθενται σε HIGH. Το σήμα PWM εφαρμόζεται ομοιόμορφα (βάσει του ορίσματος speed), εξασφαλίζοντας την ταυτόχρονη και ισόποση περιστροφή των τροχών.

Εικόνα 52: Κώδικας - turnLeft()

Εικόνα 53: Κώδικας - turnRight()

Περιστροφή στον Άξονα (turnLeft & turnRight): Εδώ αξιοποιείται πλήρως η Διαφορική Οδήγηση (Differential Steering). Για να στρίψει το όχημα, ο ένας κινητήρας τίθεται σε κατάσταση κίνησης προς τα εμπρός (LOW) και ο άλλος σε όπισθεν (HIGH). Αυτό δημιουργεί μια αντίρροπη ροπή που αναγκάζει το όχημα να περιστραφεί επιτόπου γύρω από τον κατακόρυφο άξονά του (Spin Turn), προσφέροντας μέγιστη ευελιξία σε περιορισμένους χώρους .

Εικόνα 54; Κώδικας - stop()

Πέδηση (stop): Η συνάρτηση μηδενίζει το σήμα εξόδου (analogWrite(PWM_, 0)), διακόπτοντας την παροχή ισχύος στους κινητήρες και προκαλώντας την άμεση στάση του οχήματος. Επίσης, αλλάζει την καθολική μεταβλητή shouldBeMoving = false, ενημερώνοντας το υπόλοιπο σύστημα (όπως το σύστημα αποφυγής εμποδίων) ότι το όχημα είναι πλέον ακίνητο.

Συγχρονισμός με το Σύστημα Οπτικής και Ηχητικής Σήμανσης (LEDs & Buzzer)

Κάθε φορά που καλείται μια συνάρτηση κίνησης, αναλαμβάνει αυτόματα να διαμορφώσει τόσο το σύστημα φωτισμού όσο και την ακουστική ανατροφοδότηση, προσομοιώνοντας πλήρως τη συμπεριφορά ενός πραγματικού οχήματος:

- **Στη συνάρτηση stop():** Ενεργοποιείται το κόκκινο LED (λειτουργία φώτων πέδησης/φρένων) και τίθεται σε λειτουργία ο βομβητής (Buzzer), λειτουργώντας ως άμεση οπτικοακουστική ένδειξη ακινητοποίησης του οχήματος.
- **Στη συνάρτηση backward():** Ενεργοποιείται το λευκό LED (λειτουργία φώτων όπισθεν). Παράλληλα, ο βομβητής χρησιμοποιείται ως προειδοποιητικός ήχος οπισθοπορείας (backup beeper), ενισχύοντας την ασφάλεια κατά την κίνηση προς τα πίσω, όπως ακριβώς στα βαρέα οχήματα.
- **Στις συναρτήσεις turnLeft() και turnRight():** Γίνονται true οι αντίστοιχες σημαίες (shouldLeftOrangeLedBlink ή shouldRightOrangeLedBlink). Αυτές οι σημαίες επιτρέπουν στις παράλληλες συναρτήσεις ελέγχου (π.χ. checkLeftOrangeLed) να αναβοσβήνουν τα πορτοκαλί LED στον ρυθμό της σταθεράς BLINK_PERIOD (660ms), λειτουργώντας ως φλας (Turn Indicators). Ταυτόχρονα, η κλήση του βομβητή πλαισιώνει την κίνηση με τον χαρακτηριστικό ήχο διακοπτόμενης ειδοποίησης (clicker/beeper) που συνοδεύει κάθε αλλαγή πορείας.
- Όταν το όχημα κινείται κανονικά προς τα εμπρός μέσω της συνάρτησης **forward()**, γίνεται επαναφορά του συστήματος σήμανσης: όλα τα προειδοποιητικά φώτα (κόκκινο, λευκό, πορτοκαλί) απενεργοποιούνται και ο βομβητής επανέρχεται σε κατάσταση σίγασης.

Διαχείριση Καταστάσεων Λειτουργίας (Operating Modes)

Το αυτόνομο όχημα έχει σχεδιαστεί να λειτουργεί σε δύο διακριτές καταστάσεις: τη Χειροκίνητη (Manual Mode) και την Αυτόματη (Automatic Mode). Η μετάβαση μεταξύ αυτών των δύο καταστάσεων δεν είναι μια απλή αλλαγή μιας μεταβλητής, αλλά απαιτεί την προσεκτική αρχικοποίηση και επαναφορά (reset) διαφόρων υποσυστημάτων για τη διασφάλιση της ασφάλειας και της ομαλής λειτουργίας.

Η διαχείριση αυτών των καταστάσεων υλοποιείται μέσω δύο αποκλειστικών συναρτήσεων: manual_mode() και automaticMode().

Εικόνα 55: Κώδικας - manualMode()

Η συνάρτηση manual_mode() καλείται όταν ο χρήστης επιθυμεί να αναλάβει τον πλήρη έλεγχο του οχήματος (π.χ. μέσω του τηλεχειριστηρίου υπερύθρων). Τα βήματα που εκτελούνται είναι άμεσα:

1. **Ενημέρωση Σημαίας:** Η καθολική (global) μεταβλητή manualMode τίθεται σε true. Αυτό ειδοποιεί την κύρια λούπα (loop()) να σταματήσει να εκτελεί τον αλγόριθμο αυτόνομης πλοήγησης PID και παρακολούθησης γραμμής (Line Tracking).
2. **Ασφαλής Ακινητοποίηση:** Καλείται άμεσα η συνάρτηση stop(), η οποία μηδενίζει το σήμα PWM στους κινητήρες και ενεργοποιεί τα κόκκινα LED πέδησης και το buzzer. Αυτό αποτρέπει το όχημα από το να συνεχίσει ανεξέλεγκτα την τελευταία αυτόματη εντολή που είχε λάβει.
3. **Επιβεβαίωση:** Εκτυπώνεται το μήνυμα "MANUAL MODE" στη σειριακή θύρα (Serial Monitor) για λόγους εντοπισμού σφαλμάτων (debugging).

Εικόνα 56:Κώδικας - automaticMore()

Η μετάβαση στην αυτόματη λειτουργία μέσω της συνάρτησης automaticMode() είναι σημαντικά πιο περίπλοκη, καθώς ο μικροελεγκτής πρέπει να προετοιμάσει τους αισθητήρες του για να αναλάβουν την οδήγηση. Η διαδικασία περιλαμβάνει τα εξής στάδια:

1. **Απεμπλοκή Χρήστη:** Η μεταβλητή manualMode τίθεται σε false, επιτρέποντας πλέον στον κώδικα της loop() να εκτελεί τις ρουτίνες αυτόνομου ελέγχου (όπως την αποφυγή εμποδίων και τη διόρθωση PID).
2. **Εκκαθάριση Κατάστασης Οχήματος:** Καλείται η συνάρτηση stop() για να διασφαλιστεί ότι το όχημα ξεκινά την αυτόνομη πορεία του από στάση.

   Όλες οι σημαίες οπτικής σήμανσης (LEDs) και τα αντίστοιχα ψηφιακά pins (RED_LED_PIN, WHITE_LED_PIN, ORANGE_LED_PIN1/2) τίθενται υποχρεωτικά σε LOW. Αυτό σβήνει τυχόν φλας ή φώτα πορείας που είχαν μείνει αναμμένα από την προηγούμενη χειροκίνητη χρήση.

3. **Αρχικοποίηση Συστήματος Αναφοράς (Heading Reference):** Αυτό είναι το πιο κρίσιμο βήμα. Το όχημα πρέπει να γνωρίζει προς τα πού κοιτάζει τη στιγμή που ξεκινάει η αυτόματη λειτουργία.

   Αποθηκεύεται η τρέχουσα απόλυτη γωνία του γυροσκοπίου (angleZ) στη μεταβλητή startingAbsoluteAngleZ. Αυτή η γωνία αποτελεί πλέον το «Σημείο Μηδέν» (Reference Point) για τον υπολογισμό όλων των μελλοντικών στροφών.

   Η επιθυμητή γωνία (targetAngle) μηδενίζεται (0.0), δίνοντας την εντολή στο όχημα να κινηθεί αρχικά σε ευθεία γραμμή.

4. **Επαναφορά Ελεγκτή PID:** Καλείται η συνάρτηση resetPID(). Όπως αναλύθηκε στο προηγούμενο υποσύστημα, αυτό διαγράφει το ιστορικό σφαλμάτων (Integral Windup) και τους προηγούμενους υπολογισμούς, επιτρέποντας στον ελεγκτή να ξεκινήσει τους υπολογισμούς διόρθωσης πορείας από «καθαρή» βάση.

Non-Blocking LED Control

Σε συστήματα πραγματικού χρόνου (Real-Time Systems) όπως ένα αυτόνομο όχημα, ο έλεγχος των ενδεικτικών λυχνιών (LED) για τη σήμανση της πορείας (π.χ. φλας) δεν μπορεί να υλοποιηθεί με παραδοσιακές συναρτήσεις καθυστέρησης, όπως η delay(). Η χρήση της delay() "παγώνει" την εκτέλεση του μικροελεγκτή, καθιστώντας τον ανίκανο να διαβάσει το γυροσκόπιο, να υπολογίσει το PID ή να λάβει εικόνα από την κάμερα.

Η διαχείριση της κατάστασης των LED (Κόκκινα, Λευκά, Πορτοκαλί Αριστερά/Δεξιά) γίνεται μέσω τεσσάρων διακριτών συναρτήσεων: checkRedLeds(), checkWhiteLeds(), checkLeftOrangeLed(), και checkRightOrangeLed().

Εικόνα 57: Κώδικας - checkRedLeds()

Ο αλγόριθμος αξιοποιεί τον ενσωματωμένο χρονομετρητή υλικού μέσω της συνάρτησης millis(). Η λογική λειτουργίας συνοψίζεται στα εξής τρία στάδια:

1. **Έλεγχος Ενεργοποίησης (Flag Checking):** Ελέγχεται η λογική σημαία (boolean flag) shouldRedLedsBlink, η οποία ορίζεται δυναμικά από το υποσύστημα κίνησης (π.χ. όταν καλείται η συνάρτηση stop()).
2. **Ασύγχρονη Χρονομέτρηση (Asynchronous Timing):** Εφόσον η σημαία είναι αληθής, το σύστημα υπολογίζει τη διαφορά του τρέχοντος χρόνου (now) από τον χρόνο της τελευταίας αλλαγής κατάστασης (previousTimeRED). Αν αυτή η διαφορά υπερβαίνει την προκαθορισμένη περίοδο αναλαμπής (BLINK_PERIOD, 660ms), η λογική κατάσταση του LED (HIGH/LOW) αντιστρέφεται και η μεταβλητή του χρόνου ανανεώνεται.
3. **Ασφαλής Απενεργοποίηση (Fail-Safe Reset):** Στην περίπτωση που η σημαία ενεργοποίησης είναι ψευδής (το όχημα δεν πρέπει να αναβοσβήνει τα φώτα), το πρόγραμμα παρακάμπτει τον έλεγχο χρόνου και επιβάλλει άμεσα λογική κατάσταση LOW στον αντίστοιχο ακροδέκτη, διασφαλίζοντας ότι το LED θα παραμείνει κλειστό.

Αυτή η αρχιτεκτονική επιτρέπει την ακριβή και συνεπή οπτική ανατροφοδότηση του συστήματος, διατηρώντας παράλληλα την αδιάλειπτη και ταχύτατη εκτέλεση του κύριου βρόχου (loop()).

Οι checkWhiteLeds(), checkLeftOrangeLed(), και checkRightOrangeLed() λειτουργούν με το ίδιο τρόπο.

Υποσύστημα Αντίληψης και Αποφυγής Εμποδίων (Obstacle Detection)

Η προστασία του αυτόνομου οχήματος από μετωπικές συγκρούσεις διαχειρίζεται σε υψηλό επίπεδο μέσω της συνάρτησης handleObstacle(). Σε αντίθεση με τις συναρτήσεις κίνησης που εκτελούν «τυφλές» εντολές, η συγκεκριμένη συνάρτηση λειτουργεί ως κόμβος λήψης αποφάσεων, ελέγχοντας δυναμικά το περιβάλλον.

Εικόνα 58: Κώδικας - handleObstacle()

Αξιολόγηση Απόστασης: Ελέγχεται η καθολική μεταβλητή ultrasonicDistance. Αν η απόσταση είναι μικρότερη ή ίση με το κρίσιμο όριο OBSTACLE_THRESHOLD_CM (καθορισμένο στα 5.0 εκατοστά), η συνθήκη ικανοποιείται.

Έλεγχος Κατάστασης Λειτουργίας (Mode Checking): Η συνάρτηση ελέγχει ταυτόχρονα τη σημαία !manualMode. Αυτή είναι μια κρίσιμη σχεδιαστική επιλογή: ο αυτόματος έλεγχος αποφυγής εμποδίων ενεργοποιείται μόνο όταν το όχημα πλοηγείται αυτόνομα. Όταν βρίσκεται σε χειροκίνητη λειτουργία (τηλεκατεύθυνση), το σύστημα παραχωρεί τον πλήρη έλεγχο (και την ευθύνη) στον χειριστή, επιτρέποντάς του να πλησιάσει εμπόδια χωρίς την παρεμβολή του λογισμικού.

Όταν ικανοποιούνται ταυτόχρονα και οι δύο συνθήκες (Εμπόδιο < 5cm ΚΑΙ Αυτόματη Λειτουργία), η συνάρτηση εγείρει τη σημαία (boolean flag) obstacleDetected = true. Διαφορετικά, η σημαία παραμένει ή επιστρέφει στην κατάσταση false.

Η αλλαγή της obstacleDetected παρακολουθείται από την loop()). Όταν η σημαία γίνει αληθής, ο κύριος βρόχος αναστέλλει (override) τη λειτουργία του Ελεγκτή PID και του αλγορίθμου παρακολούθησης γραμμής. Αντ' αυτού, επιβάλλει στο όχημα έναν ελιγμό αποφυγής, αντιστρέφοντας τη φορά των κινητήρων (όπισθεν με σταθερή ταχύτητα 200 PWM) και ενεργοποιώντας την αντίστοιχη οπτική σήμανση (λευκά LED). Μόλις το εμπόδιο απομακρυνθεί, η σημαία κατεβαίνει και το PID αναλαμβάνει ξανά την εξομάλυνση της πορείας.

Σύστημα Τηλεχειρισμού και Αποκωδικοποίησης Υπερύθρων (IR Communication)

Η συνάρτηση handleIRcommunication() διαχειρίζεται τα εισερχόμενα σήματα του τηλεχειριστηρίου, λειτουργώντας ως η κύρια διεπαφή μεταξύ του χειριστή και της λογικής του συστήματος.

Εικόνα 59: Κώδικας – handleIRcommunication()

Φιλτράρισμα Σήματος (Debouncing): Για την αποφυγή εσφαλμένης πολλαπλής εκτέλεσης εντολών, ο κώδικας απορρίπτει αυτόματα τα επαναλαμβανόμενα σήματα του υλικού (IRDATA_FLAGS_IS_REPEAT). Παράλληλα, εφαρμόζει μια χρονική αποθορυβοποίηση λογισμικού (DEBOUNCE_DELAY 250ms), αγνοώντας ταχύτατα, τυχαία πατήματα του ίδιου πλήκτρου.

Εικόνα 60: Κώδικας – handleIRcommunication()

Εικόνα 61: Κώδικας – handleIRcommunication()

Στη **Χειροκίνητη Λειτουργία**, τα πλήκτρα κατεύθυνσης καλούν απευθείας τις συναρτήσεις κινηματικής (π.χ. forward(), turnLeft()), προσφέροντας απόλυτο, άμεσο έλεγχο στους κινητήρες.

Στην **Αυτόματη Λειτουργία**, η άμεση οδήγηση παρακάμπτεται. Αντ' αυτού, οι εντολές κατεύθυνσης ορίζουν μια νέα επιθυμητή γωνία πορείας (targetAngle), αφήνοντας τον ελεγκτή PID να αναλάβει την ομαλή, αυτόνομη εκτέλεση της στροφής.

Μέσω σγκεκριμένωνν πλήκτρων, η συνάρτηση επιτρέπει την αυξομείωση της βασικής ταχύτητας PWM (currentSpeed), εφαρμόζοντας αυστηρά όρια (70 έως 255) για την προστασία των κινητήρων.

Στο τέλος κάθε κύκλου αποκωδικοποίησης, καλείται η μέθοδος IrReceiver.resume(), προετοιμάζοντας τον δέκτη για το επόμενο σήμα χωρίς να καθυστερεί την εκτέλεση του κύριου προγράμματος.

Παράλληλη Επεξεργασία Δεδομένων Υπερήχων (FreeRTOS Ultrasonic Task)

Η συνεχής μέτρηση της απόστασης για την αποφυγή εμποδίων απαιτεί χρονικές καθυστερήσεις (αναμονή για το echo) που θα μπορούσαν να "παγώσουν" τον κώδικα. Για την εξάλειψη αυτού του προβλήματος, η συλλογή των δεδομένων του αισθητήρα HC-SR04 έχει απομονωθεί σε μια ανεξάρτητη διεργασία (Task) του FreeRTOS, μέσω της συνάρτησης ultrasonicTask().

Εικόνα 62: Κώδικας - ultrasonicTask()

Η λειτουργία της διεργασίας βασίζεται στα ακόλουθα στάδια:

1. **Διέγερση Αισθητήρα (Triggering):** Μέσα σε έναν βρόχο (while(1)), η διεργασία στέλνει έναν ψηφιακό παλμό διάρκειας 10 ms στον ακροδέκτη TRIGGER_PIN, δίνοντας εντολή στον αισθητήρα να εκπέμψει τα υπερηχητικά κύματα.
2. **Μέτρηση Χρόνου:** Μέσω της εντολής pulseIn(), ο μικροελεγκτής μετράει τον χρόνο που παραμένει ενεργός (HIGH) ο ακροδέκτης ECHO_PIN, ο οποίος αντιστοιχεί στον χρόνο που χρειάστηκε ο ήχος για να χτυπήσει στο εμπόδιο και να επιστρέψει. Η προσθήκη ορίου αναμονής (timeout) στα 20.000 μικροδευτερόλεπτα προστατεύει το σύστημα από το να εγκλωβιστεί σε κατάσταση ατέρμονης αναμονής (blocking) αν το σήμα χαθεί.
3. **Κινηματική Μετατροπή:** Εάν ληφθεί έγκυρη μέτρηση, ο χρόνος μετατρέπεται σε απόσταση (εκατοστά). Η μετατροπή γίνεται πολλαπλασιάζοντας τον χρόνο με την ταχύτητα του ήχου στον αέρα (0.034 cm/μs) και διαιρώντας δια δύο (αφού ο ήχος καλύπτει διπλή απόσταση, μετάβαση και επιστροφή). Το αποτέλεσμα εγγράφεται στην καθολική μεταβλητή ultrasonicDistance, καθιστώντας το διαθέσιμο στη συνάρτηση handleObstacle().
4. **Αποδέσμευση Επεξεργαστή (Task Yielding):** Το πιο σημαντικό στοιχείο της διεργασίας είναι η χρήση της συνάρτησης του RTOS vTaskDelay(pdMS_TO_TICKS(100)). Η εντολή αυτή δεν "παγώνει" τον επεξεργαστή όπως η κλασική delay(). Αντίθετα, θέτει τη συγκεκριμένη διεργασία σε "ύπνο" για 100ms, αφήνοντας ελεύθερους άμεσα τους πόρους ώστε να μπορούν να χρησιμοποιηθούν για άλλες διεργασίες. Έτσι, ο επεξεργαστής μένει ελεύθερος να εκτελέσει την Τεχνητή Νοημοσύνη και το PID χωρίς καμία καθυστέρηση.

Διεπαφή Χρήστη και Διακομιστής Ιστού (IoT Web Server)

Για την απομακρυσμένη παρακολούθηση του οχήματος σε πραγματικό χρόνο, ο μικροελεγκτής φιλοξενεί έναν τοπικό διακομιστή ιστού (Web Server). Η αρχιτεκτονική του διακομιστή ακολουθεί το μοντέλο πελάτη-εξυπηρετητή (client-server) και διαχειρίζεται τα αιτήματα μέσω τριών εξειδικευμένων συναρτήσεων:

Εικόνα 63: Κώδικας - handleServerRoot()

Εικόνα 64: Κώδικας - handleServerRoot()

Εικόνα 65: Κώδικας - handleServerRoot()

handleServerRoot(): Αποτελεί τη ρίζα (root endpoint /) του διακομιστή. Επιστρέφει στον περιηγητή (browser) του χρήστη τον κώδικα HTML και JavaScript που χτίζει τη γραφική διεπαφή. Η σελίδα χρησιμοποιεί την τεχνολογία HTML5 Canvas και ασύγχρονες κλήσεις (AJAX/Fetch API) κάθε 300ms για να αντλεί δεδομένα στο παρασκήνιο, χωρίς να απαιτείται ανανέωση (refresh) της σελίδας.

Εικόνα 66: Κώδικας - handleServerImageCapture()

handleServerImageCapture(): Ανταποκρίνεται στο endpoint /capture. Αντί να μεταδώσει μια συμπιεσμένη εικόνα JPEG, η συνάρτηση μεταδίδει απευθείας τα ακατέργαστα δεδομένα της μνήμης (raw buffer 96x96) σε μορφή application/octet-stream. Αυτή η προσέγγιση είναι εξαιρετικά αποδοτική σε ταχύτητα, καθώς απαλλάσσει τον μικροελεγκτή από τον υπολογιστικό φόρτο της συμπίεσης εικόνας.

Εικόνα 67: Κώδικας - handleServerData()

handleServerData(): Ανταποκρίνεται στο endpoint /data. Συλλέγει τις καθολικές μεταβλητές που αφορούν την αναγνώριση αντικειμένων (previousLabelX, previousLabelY, previousLabel) και τις δομεί σε ένα τυποποιημένο αντικείμενο JSON (π.χ. {"x": 45, "y": 50, "label": "Stop Sign"}). Η JavaScript στον υπολογιστή του χρήστη διαβάζει αυτό το JSON και σχεδιάζει δυναμικά το κόκκινο πλαίσιο (bounding box) πάνω στον καμβά του βίντεο.

Μηχανική Όραση και Ενσωματωμένη Τεχνητή Νοημοσύνη (Edge AI)

Η αναγνώριση αντικειμένων εκτελείται τοπικά (on-device) μέσω του μοντέλου μηχανικής μάθησης της Edge Impulse, αξιοποιώντας τη βιβλιοθήκη TensorFlow Lite. Η διαδικασία αυτή είναι υπολογιστικά απαιτητική και χωρίζεται σε δύο φάσεις:

Προεπεξεργασία Δεδομένων (convertBufferForAI):

Εικόνα 68: Κώδικας – convertBufferForAI

Τα νευρωνικά δίκτυα απαιτούν τα δεδομένα εισόδου να βρίσκονται σε συγκεκριμένη μαθηματική μορφή. Η κάμερα παράγει εικονοστοιχεία (pixels) ως ακέραιους αριθμούς 8-bit (από το 0 έως το 255). Η συνάρτηση convertBufferForAI λειτουργεί ως μέθοδος προεπεξεργασίας (callback): σαρώνει τον πίνακα της εικόνας, μετατρέπει κάθε εικονοστοιχείο σε αριθμό κινητής υποδιαστολής (32-bit float) και αφαιρεί την τιμή 128. Η πράξη αυτή (Pixel - 128) κανονικοποιεί τα δεδομένα, κεντράροντάς τα γύρω από το μηδέν (περίπου από -128.0 έως 127.0), κάτι που βελτιώνει ριζικά την ακρίβεια και την ταχύτητα σύγκλισης του μοντέλου AI.

Εκτέλεση Συμπεράσματος (inferenceTask):

Εικόνα 69: Κώδικας – inferenceTask

Εικόνα 70: Κώδικας – inferenceTask

Η αναγνώριση αντικειμένων (Inferencing) εκτελείται ασύγχρονα ως μια ανεξάρτητη διεργασία του λειτουργικού συστήματος FreeRTOS . Η αρχιτεκτονική της διασφαλίζει τη σταθερότητα του συστήματος μέσα από τα εξής βήματα:

1. **Ασφάλεια Μνήμης (Mutex Synchronization):** Επειδή η κάμερα εγγράφει συνεχώς νέες εικόνες στη μνήμη, η διεργασία χρησιμοποιεί τον σηματοφορέα FrameMutex (xSemaphoreTake). Αυτό «κλειδώνει» προσωρινά τη μνήμη της εικόνας, αποτρέποντας την κάμερα από το να αλλοιώσει τα δεδομένα (memory corruption/tearing) την ώρα που το AI τα διαβάζει.

2. **Ταξινόμηση (Classification):** Καλείται η συνάρτηση run_classifier(), η οποία τροφοδοτεί τα κανονικοποιημένα δεδομένα στο εκπαιδευμένο νευρωνικό δίκτυο.

3. **Αξιολόγηση Εμπιστοσύνης (Confidence Thresholding):** Μόλις εξαχθούν τα αποτελέσματα, ο κώδικας σαρώνει τα ανιχνευμένα πλαίσια οριοθέτησης (bounding boxes). Εάν η βεβαιότητα του μοντέλου (bb.value) υπερβαίνει το κατώφλι του 20% (> 0.2f), το σύστημα κατοχυρώνει την ανίχνευση, ενημερώνοντας τις καθολικές συντεταγμένες και την ετικέτα του αντικειμένου. Εάν κανένα αντικείμενο δεν ξεπεράσει αυτό το όριο, η ετικέτα επιστρέφει στην κατάσταση "Searching...".

4. **Αποδέσμευση Πόρων:** Με την ολοκλήρωση της ανάλυσης, η διεργασία ξεκλειδώνει τη μνήμη (xSemaphoreGive) και εισέρχεται σε κατάσταση αναμονής (vTaskDelay), παραχωρώντας τον επεξεργαστή στις υπόλοιπες λειτουργίες του ρομπότ.

Setup()

Η συνάρτηση setup() αποτελεί το σημείο εισόδου (entry point) του προγράμματος και εκτελείται κατά την εκκίνηση ή την επαναφορά (reset) του μικροελεγκτή. Ο ρόλος της είναι η προετοιμασία όλου του υλικού και του λογισμικού, ακολουθώντας μια αυστηρή, διαδοχική σειρά αρχικοποίησης για την αποφυγή διενέξεων (conflicts) μεταξύ των υποσυστημάτων.

Εικόνα 71: Κώδικας - setup()

Η διαδικασία ξεκινά με την εγκαθίδρυση των πρωτοκόλλων επικοινωνίας:

Σειριακή Επικοινωνία (UART): Ενεργοποιείται η κύρια σειριακή θύρα (Serial.begin(115200)) για την αποσφαλμάτωση (debugging) και η δευτερεύουσα (GpsSerial.begin) στα 9600 baud για τη λήψη των δεδομένων NMEA από τη μονάδα GPS.

Δίαυλος I2C: Γίνεται κλήση της Wire.begin() για την εκκίνηση του διαύλου επικοινωνίας.

Αρχικοποίηση Περιφερειακών I2C: Αμέσως μετά, ενεργοποιούνται οι συσκευές που στηρίζονται στο I2C. Το γυροσκόπιο αφυπνίζεται (gyroBegin()) και βαθμονομείται (calibrateGyro()). Παράλληλα, αρχικοποιείται η οθόνη LCD (Lcd.init()) και ο επεκτάτης θυρών MCP23017 (Mcp.begin_I2C()).

Καθορισμός Ακροδεκτών (Pin Mapping): Μέσω των εντολών pinMode και Mcp.pinMode, το σύστημα δηλώνει τη συμπεριφορά (Είσοδος/Έξοδος) όλων των αισθητήρων (υπέρηχοι, φλόγα, καπνός) και των ενεργοποιητών (κινητήρες, LED, βομβητής), κατανέμοντας τους πόρους μεταξύ του ESP32 και του MCP.

Εικόνα 72: Κώδικας - setup()

Εικόνα 73: Κώδικας - setup()

Εικόνα 74: Κώδικας - setup()

Η αρχικοποίηση της κάμερας αποτελεί ένα από τα πιο κρίσιμα στάδια και απαιτεί ειδικό χειρισμό:

Τροφοδοσία (PMIC AXP313A): Στις σύγχρονες πλακέτες ESP32-S3, η κάμερα δεν τροφοδοτείται άμεσα. Ο κώδικας αρχικοποιεί το ολοκληρωμένο AXP313A και του δίνει εντολή να παρέχει ρεύμα στον αισθητήρα OV2640 (Axp.enableCameraPower).

Εικόνα 75: Κώδικας - setup()

Διαμόρφωση Κάμερας: Δημιουργείται μια δομή διαμόρφωσης (camera_config_t) αυστηρά προσαρμοσμένη στις απαιτήσεις της Τεχνητής Νοημοσύνης. Επιλέγεται μορφή PIXFORMAT_GRAYSCALE (ασπρόμαυρο) και ανάλυση FRAMESIZE_96X96 . Αυτή η επιλογή ελαχιστοποιεί δραματικά το μέγεθος του Frame Buffer (μικρότερη κατανάλωση μνήμης PSRAM) και επιταχύνει τον χρόνο συμπεράσματος (inference time) του μοντέλου Edge Impulse.

Αποφυγή Διενέξεων (I2C Conflicts): Μια εξαιρετική πρακτική του κώδικα είναι η απενεργοποίηση του εσωτερικού SCCB I2C της κάμερας (pins ορισμένα σε -1), προκειμένου να μην υπάρξει εμπλοκή με το γυροσκόπιο και το MCP23017. Μόνο μετά τη διασφάλιση αυτών των παραμέτρων, καλείται η esp_camera_init().

Εικόνα 76: Κώδικας - setup()

Το σύστημα συνδέεται στο τοπικό δίκτυο χρησιμοποιώντας τη βιβλιοθήκη WiFiMulti, η οποία διαθέτει λίστα με το κύριο δίκτυο και εφεδρικά δίκτυα (fallback networks), εξασφαλίζοντας υψηλή διαθεσιμότητα. Αφού επιτευχθεί η σύνδεση, ενεργοποιείται το mDNS (για πρόσβαση μέσω της διεύθυνσης esp32-data.local) και ρυθμίζονται τα endpoints του Διακομιστή Ιστού (/, /capture, /data) μέσω της Server.on(). Τέλος, ο διακομιστής τίθεται σε λειτουργία (Server.begin()).

Το τελευταίο βήμα της αρχικοποίησης εισάγει το σύστημα στον κόσμο της παράλληλης επεξεργασίας:

Δημιουργείται ο σηματοφορέας (Mutex) FrameMutex, ο οποίος θα εγγυάται την ασφαλή, εναλλάξ πρόσβαση στη μνήμη της εικόνας.
Δημιουργούνται και εκκινούν οι ανεξάρτητες διεργασίες: Το έργο ανάλυσης της Τεχνητής Νοημοσύνης (inferenceTask) καρφιτσώνεται (pinned) στρατηγικά στον Core 1 του επεξεργαστή, προκειμένου να εκμεταλλευτεί τον διπύρηνο σχεδιασμό του ESP32-S3, ενώ εκκινεί και η διεργασία των υπερήχων (ultrasonicTask).
Το όχημα τυπώνει το αρχικό μήνυμα "Bus Departed" στην οθόνη LCD, σημαίνοντας την επιτυχή ολοκλήρωση της εκκίνησης.

Main Control Loop

Η συνάρτηση loop() αποτελεί τον πυρήνα εκτέλεσης του λειτουργικού συστήματος του οχήματος. Ο ρόλος της δεν είναι η απευθείας εκτέλεση χρονοβόρων υπολογισμών, αλλά ο

ταχύτατος συντονισμός όλων των ασύγχρονων υποσυστημάτων. Κάθε επανάληψη του βρόχου διαρκεί ελάχιστα χιλιοστά του δευτερολέπτου, διασφαλίζοντας την άμεση απόκριση του συστήματος (Real-Time Responsiveness).

Εικόνα 77: Κώδικας - loop()

Ο βρόχος ξεκινά με την εξυπηρέτηση των κρίσιμων εξωτερικών διεπαφών:

1. **Εξυπηρέτηση Πελατών Ιστού:** Καλείται η Server.handleClient(), η οποία ελέγχει εάν υπάρχει νέο αίτημα (HTTP Request) από κάποιον browser.
2. **Λήψη Καρέ (Frame Buffering):** Το σύστημα ζητά ένα νέο καρέ από την κάμερα (esp_camera_fb_get()). Αν το καρέ είναι έγκυρο (διαθέτει το σωστό μέγεθος bufferLength), ο μικροελεγκτής δεσμεύει προσωρινά τον xSemaphoreTake. Στη συνέχεια, αντιγράφει ταχύτατα τα δεδομένα της εικόνας στον καθολικό πίνακα currentRawImageCopy (μέσω της συνάρτησης memcpy) και ελευθερώνει τον σηματοφορέα (xSemaphoreGive). Η διαδικασία αυτή εγγυάται ότι η διεργασία της Τεχνητής Νοημοσύνης (inferenceTask) θα έχει πάντα πρόσβαση στην πιο πρόσφατη εικόνα, χωρίς τον κίνδυνο αλλοίωσης των δεδομένων (memory corruption). Στο τέλος, η μνήμη της κάμερας αποδεσμεύεται (esp_camera_fb_return).
3. **Αποκωδικοποίηση Υπερύθρων & Σειριακής:** Αμέσως μετά, το σύστημα ελέγχει για εντολές τηλεχειρισμού (handleIRcommunication()). Ταυτόχρονα, σαρώνει τη σειριακή θύρα (Serial.available()) για "ψευδείς" εντολές δοκιμής (mock commands, π.χ. "smoke", "arrival"), οι οποίες χρησιμοποιούνται για την προσομοίωση καταστάσεων έκτακτης ανάγκης και την ενημέρωση της οθόνης LCD κατά το στάδιο της ανάπτυξης.

Εικόνα 78: Κώδικας - loop()

Εικόνα 79: Κώδικας - loop()

Ελέγχονται οι ψηφιακοί (φλόγα, καπνός) και αναλογικοί (δόνηση) αισθητήρες. Ανάλογα με τις μετρήσεις, υψώνονται οι αντίστοιχες σημαίες συναγερμού (π.χ. flame = true) και ενημερώνεται η οθόνη LCD. Παράλληλα, καλείται η isPickedUp() για τον έλεγχο ανύψωσης του οχήματος.

Εικόνα 80: Κώδικας - loop()

Μέσω ενός χρονομέτρου (κάθε 5 δευτερόλεπτα), ελέγχεται η διαθεσιμότητα του I/O Expander MCP23017 (Wire.beginTransmission(0x20)). Σε περίπτωση αποσύνδεσης ή ηλεκτρομαγνητικής παρεμβολής, το σύστημα επιχειρεί αυτόματη επανεκκίνηση (Re-initialization) του τσιπ και των ακροδεκτών του, αποτρέποντας την κατάρρευση του συστήματος.

Εικόνα 81: Κώδικας - loop()

Καλούνται οι συναρτήσεις ελέγχου των LED (checkRedLeds(), checkWhiteLeds() κ.λπ.), οι οποίες, όπως προαναφέρθηκε, αναβοσβήνουν τα φώτα χωρίς να μπλοκάρουν την εκτέλεση του κώδικα.

Εικόνα 82: Κώδικας - loop()

Το σύστημα τροφοδοτεί συνεχώς το αντικείμενο Gps με νέους χαρακτήρες από τη σειριακή θύρα (Gps.encode()) και ανανεώνει τις συντεταγμένες (latitude, longitude).

Το πιο κρίσιμο τμήμα του βρόχου ενεργοποιείται όταν το όχημα βρίσκεται σε αυτόματη λειτουργία (!manualMode). Η λογική ελέγχου ακολουθεί αυστηρή ιεραρχία προτεραιοτήτων:

Προτεραιότητα 1: Αποφυγή Εμποδίων. Καλείται η handleObstacle(). Εάν η σημαία obstacleDetected είναι ενεργή (εμπόδιο < 5cm), ο αλγόριθμος πλοήγησης αναστέλλεται. Το όχημα εκτελεί άμεση οπισθοπορεία (PWM 200) και ενεργοποιεί τα λευκά LED, παρακάμπτοντας κάθε άλλη εντολή κίνησης.

Προτεραιότητα 2: Ασφάλεια Υπερήχων. Αν η απόσταση είναι μικρότερη από 10cm, η σημαία shouldBeMoving γίνεται false και το όχημα ακινητοποιείται πλήρως (ανάβοντας τα κόκκινα LED).

Προτεραιότητα 3: Παρακολούθηση Γραμμής (Line Tracking). Εφόσον η διαδρομή είναι ελεύθερη (shouldBeMoving && !obstacleDetected), το σύστημα διαβάζει τους αισθητήρες υπερύθρων (IR_Right, IR_Left). Βάσει της ανακλαστικότητας του εδάφους (HIGH/LOW), τροποποιεί δυναμικά τη γωνία-στόχο (targetAngle ±10.0 μοίρες).

Προτεραιότητα 4: Εξομάλυνση και Διόρθωση (PID). Για την αποφυγή νευρικής κίνησης (ταλάντωση "sine wave") λόγω απότομων αλλαγών στον στόχο, ο κώδικας εφαρμόζει ένα ψηφιακό Φίλτρο Εκθετικού Κινητού Μέσου Όρου (Exponential Moving Average Filter) με συντελεστή $\alpha = 0.2$. Η φιλτραρισμένη γωνία (filteredTargetAngle) "γλιστράει" ομαλά προς τον νέο στόχο. Τέλος, η τιμή αυτή τροφοδοτείται στη συνάρτηση pidCorrection(), η οποία υπολογίζει το σφάλμα σε σχέση με το γυροσκόπιο (angleZ) και ρυθμίζει την ταχύτητα των τροχών (Differential Steering).

Εικόνα 83: Κώδικας - loop()

Εικόνα 84: Κώδικας - loop()

Εικόνα 85: Κώδικας - loop()

Εικόνα 86: Κώδικας - loop()

Εικόνα 87: Κώδικας - loop()

Εικόνα 88: Κώδικας - loop()

Στο τέλος του βρόχου, το σύστημα αξιολογεί τις τελικές ταχύτητες των τροχών (leftMotorSpeed, rightMotorSpeed). Μέσω μιας σειράς συγκρίσεων, το λογισμικό αντιλαμβάνεται τη φυσική συμπεριφορά του οχήματος (στάση, όπισθεν, στροφή) και ενεργοποιεί αυτόματα τα αντίστοιχα LED (Κόκκινα, Λευκά, Πορτοκαλί Φλας), κλείνοντας τον κύκλο αλληλεπίδρασης μεταξύ υλικού και λογισμικού πριν το loop() ξεκινήσει ξανά.

Σχεδιασμός και Υλοποίησης Εφαρμογής

Επιλογή του Περιβάλλοντος Qt Designer

Για την υλοποίηση της διεπαφής επιλέχθηκε το περιβάλλον Qt και συγκεκριμένα η τεχνολογία Qt Quick/QML. Η επιλογή αυτή βασίστηκε στην επένδυση χρόνου σε μια τεχνολογία που αποτελεί βιομηχανικό πρότυπο, προσφέροντας πλούσια τεκμηρίωση και ολοκληρωμένο οικοσύστημα για την ανάπτυξη σύγχρονων εφαρμογών.

Σχεδιασμός Λογοτύπου (Logo Design)

Η οπτική ταυτότητα της εφαρμογής SafeRoute σχεδιάστηκε με χρήση τεχνολογίας Generative AI. Το λογότυπο δημιουργήθηκε βάσει στοχευμένου prompt και αποτυπώνει τη σχολική μεταφορά σε συνδυασμό με την έννοια της ασφάλειας, μέσω απεικόνισης σχολικού λεωφορείου εντός προστατευτικής ασπίδας.

Εικόνα 1: SafeRoute Logo – Light Mode

Σενάριο Εφαρμογής

Το SafeRoute αποτελεί πλατφόρμα παρακολούθησης σχολικών μεταφορών και απευθύνεται σε δύο βασικές ομάδες χρηστών: γονείς/κηδεμόνες και διοίκηση σχολείου. Η πρόσβαση πραγματοποιείται μέσω κοινής οθόνης σύνδεσης, με δυναμική προσαρμογή του περιβάλλοντος βάσει ρόλου χρήστη.

Κοινά Στοιχεία Ελέγχου & Ρυθμίσεις

Η εφαρμογή περιλαμβάνει μόνιμα στοιχεία ελέγχου σε σταθερές θέσεις, εξασφαλίζοντας συνέπεια και άμεση πρόσβαση σε βασικές λειτουργίες.

1. **Επάνω Δεξιά Γωνία - Γενικές Ρυθμίσεις Περιβάλλοντος**
    - **Εναλλαγή Γλώσσας**
    - **Εναλλαγή Θέματος**



Εικόνα 3: Στοιχεία Ελέγχου

2. **Κάτω Δεξιά Γωνία - Υποστήριξη & Απόρρητο**
    - **Υποστήριξη:** Πρόσβαση σε οδηγίες χρήσης και συμβουλές λειτουργίας.
    - **Ρυθμίσεις:** Μενού απορρήτου και τεχνικών παραμέτρων της εφαρμογής.



Εικόνα 4: Στοιχεία Ελέγχου

Σημείωση: Οι επιλογές «Υποστήριξη» και «Ρυθμίσεις» δεν είναι διαθέσιμες στις οθόνες Σύνδεσης και Ανάκτησης Κωδικού, για λόγους ασφάλειας και απλοποίησης της διαδικασίας εισόδου.

3. **Επάνω Αριστερή Γωνία Χάρτη και Camera Feed – Ηχητικές Ειδοποιήσεις**



Εικόνα 5: Στοιχεία Ελέγχου

Κοινή Οθόνη Εισόδου & Ανάκτησης Κωδικού

Η πρόσβαση στην εφαρμογή *SafeRoute* πραγματοποιείται μέσω ενιαίας πύλης εισόδου, κοινής για όλους τους χρήστες (γονείς/κηδεμόνες και διοίκηση). Η σχεδίαση δίνει έμφαση στην απλότητα και την ασφάλεια, εισάγοντας ταυτόχρονα τον χρήστη στην εταιρική ταυτότητα της εφαρμογής.

Οθόνη Εισόδου (Login Screen)

Η οθόνη οργανώνεται σε δύο βασικές ζώνες:

1. **Εταιρική Ταυτότητα:** Στην αριστερή πλευρά προβάλλεται το λογότυπο *SafeRoute*, ενισχύοντας το αίσθημα ασφάλειας και αξιοπιστίας από την πρώτη επαφή με το σύστημα.

2. **Φόρμα Πιστοποίησης:** Στο κεντρικό τμήμα βρίσκεται η φόρμα εισόδου με πεδία «Όνομα χρήστη» και «Κωδικός», συνοδευόμενα από οικεία εικονίδια για άμεση αναγνώριση της λειτουργίας τους.

   Στο πεδίο του κωδικού έχει ενσωματωθεί λειτουργία εμφάνισης/απόκρυψης (eye icon), η οποία μειώνει τα σφάλματα εισαγωγής και ενισχύει την ευχρηστία.



Εικόνα 6: Κοινή Οθόνη Σύνδεσης – Light Mode

Οθόνη Ανάκτησης Κωδικού (Password Recovery)

Η διαδικασία ανάκτησης κωδικού είναι άμεσα προσβάσιμη μέσω σχετικού συνδέσμου από την οθόνη εισόδου και έχει σχεδιαστεί με γνώμονα την απλότητα:

1. Παρέχονται σαφείς οδηγίες για την εισαγωγή της διεύθυνσης email εγγραφής.

2. Ο χρήστης συμπληρώνει ένα μόνο πεδίο, ενώ το κουμπί «Αποστολή Συνδέσμου» ενεργοποιεί τη διαδικασία επαναφοράς.

3. Υπάρχει εμφανής σύνδεσμος «Επιστροφή στην Είσοδο», διευκολύνοντας την πλοήγηση.

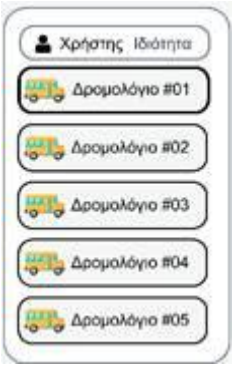


Εικόνα 7: Κοινή Οθόνη Ανάκτησης Κωδικού – Dark Mode

Περιβάλλον Διοίκησης Σχολείου (School App Interface)

Η διοίκηση του σχολείου ασκεί εποπτικό ρόλο μέσω ενός κεντρικού πίνακα ελέγχου (Admin Dashboard). Η διεπαφή οργανώνεται σε δύο βασικά τμήματα:

1. **Πλευρική Στήλη Πλοήγησης**
   Στην αριστερή πλευρά της οθόνης διατηρείται σταθερά η λίστα των ενεργών δρομολογίων (#01–#05), επιτρέποντας γρήγορη εναλλαγή μεταξύ οχημάτων. Στο άνω μέρος της στήλης εμφανίζεται πλαίσιο ταυτοποίησης του συνδεδεμένου χρήστη με το όνομα και την ιδιότητά του.



Εικόνα 8: Διοίκηση Σχολείου - Focused στο Δρομολόγιο #01 – Light Mode

2. **Κεντρική Περιοχή Πληροφοριών:** Το περιεχόμενο της κύριας οθόνης μεταβάλλεται δυναμικά ανάλογα με την επιλογή του χρήστη, προσφέροντας δύο επίπεδα ελέγχου:

    A. **Προβολή Κατάστασης (Προεπιλογή)**
    Ο διαχειριστής έχει πλήρη εποπτεία του δρομολογίου μέσω χάρτη πραγματικού χρόνου. Στο κάτω μέρος εμφανίζονται πληροφορίες θέσης και εκτιμώμενης ώρας άφιξης (ETA).

    Στη δεξιά στήλη προβάλλονται τα στοιχεία του οδηγού, με δυνατότητα άμεσης κλήσης μέσω εικονιδίου τηλεφώνου. Κάτω από αυτά εμφανίζεται η λίστα μαθητών (Student Manifest), με χρωματικές ενδείξεις κατάστασης (εντός/εκτός οχήματος) και δυνατότητα κύλισης για μεγάλες λίστες.



Εικόνα 9: Διοίκηση Σχολείου - Focused στο Δρομολόγιο #01 – Light Mode

B. **Προβολή Επίβλεψης**
Με την ενεργοποίηση του κουμπιού «Επίβλεψη», το σύστημα εισέρχεται σε λειτουργία αυξημένης παρακολούθησης:

- Ο χάρτης αντικαθίσταται από ζωντανή εικόνα (Live Feed) της κάμερας του οχήματος.
- Η δεξιά στήλη διατηρεί τα στοιχεία οδηγού και τη λίστα επιβαινόντων, εξασφαλίζοντας συνέχεια πληροφορίας.
- Η αριστερή στήλη μετατρέπεται σε ημερολόγιο συμβάντων (Event Log), όπου καταγράφονται αποβιβάσεις μαθητών και κρίσιμες ειδοποιήσεις ασφαλείας.

Η επιστροφή στην προεπιλεγμένη προβολή γίνεται άμεσα μέσω του κουμπιού «Κατάσταση».



Εικόνα 10:Διοίκηση Σχολείου - Focused στο Δρομολόγιο #01- Επίβλεψη – Light Mode

Περιβάλλον Γονέων και Κηδεμόνων (Parent App Interface)

Η διεπαφή για γονείς και κηδεμόνες υιοθετεί *mobile-first προσέγγιση*, με έμφαση στην άμεση ενημέρωση και στη μείωση των απαιτούμενων ενεργειών για κρίσιμες λειτουργίες. Η οθόνη οργανώνεται σε τρεις βασικές ζώνες πληροφορίας.

1. **Πλευρική Στήλη Κατάστασης & Επιλογής (Αριστερά)**
Η αριστερή στήλη λειτουργεί ως κέντρο ειδοποιήσεων και διαχείρισης προφίλ, με πλαίσιο ταυτοποίησης του συνδεδεμένου χρήστη.

A. **Ενδείξεις Συμβάντων:** Χρωματικά κωδικοποιημένες ειδοποιήσεις ενημερώνουν για την κατάσταση του δρομολογίου (Κίνδυνος – κόκκινο, Αποβίβαση – πράσινο, Καθυστέρηση – κίτρινο).

B. **Επιλογή Μαθητή:** Πλήκτρα επιλογής επιτρέπουν την εστίαση σε συγκεκριμένο παιδί, διευκολύνοντας οικογένειες με περισσότερους μαθητές.



Εικόνα 11: Γονείς/Κηδεμόνες - Focused στη μαθήτρια "Carpenter K." – Light Mode

2. **Κεντρική Περιοχή Παρακολούθησης**
Αποτελεί τον πυρήνα της διεπαφής και παρέχει πληροφορίες σε πραγματικό χρόνο. Προβάλλεται χάρτης με τη θέση του οχήματος, ενώ στο κάτω μέρος εμφανίζονται η τρέχουσα τοποθεσία και η εκτιμώμενη ώρα άφιξης (ETA).

3. **Πλευρική Στήλη Πληροφοριών & Ενεργειών (Δεξιά)**
Στη δεξιά πλευρά συγκεντρώνονται οι δυναμικές πληροφορίες του ταξιδιού και οι βασικές ενέργειες αλληλεπίδρασης

A. **Κάρτα Κατάστασης (Status Card):** Παρουσιάζει το πρόσφατο ιστορικό ενεργειών του μαθητή με χρονοσήμανση και οπτική επιβεβαίωση.

B. **Στοιχεία Οδηγού:** Εμφανίζονται το όνομα του οδηγού και ο αριθμός λεωφορείου, με δυνατότητα άμεσης κλήσης μέσω εικονιδίου τηλεφώνου.

C. **Δήλωση Απουσίας:** Ευδιάκριτο κουμπί επιτρέπει την άμεση ενημέρωση σχολείου και οδηγού σε περίπτωση απουσίας, βελτιστοποιώντας τη ροή του δρομολογίου.



Εικόνα 12: Γονείς/Κηδεμόνες - Focused στη μαθήτρια "Carpenter K."- Dark Mode

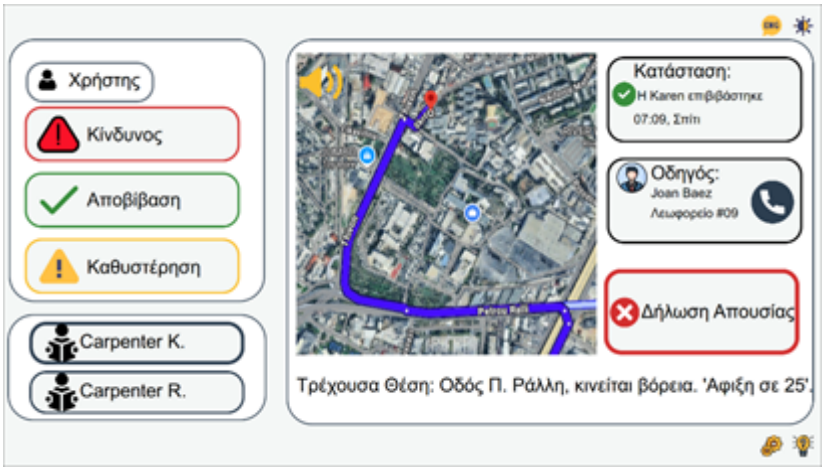
Φωτογραφική Απεικόνιση



Εικόνα 19: Login Screen



Εικόνα 20: Password Recovery Screen



Εικόνα 21: Parents' Screen

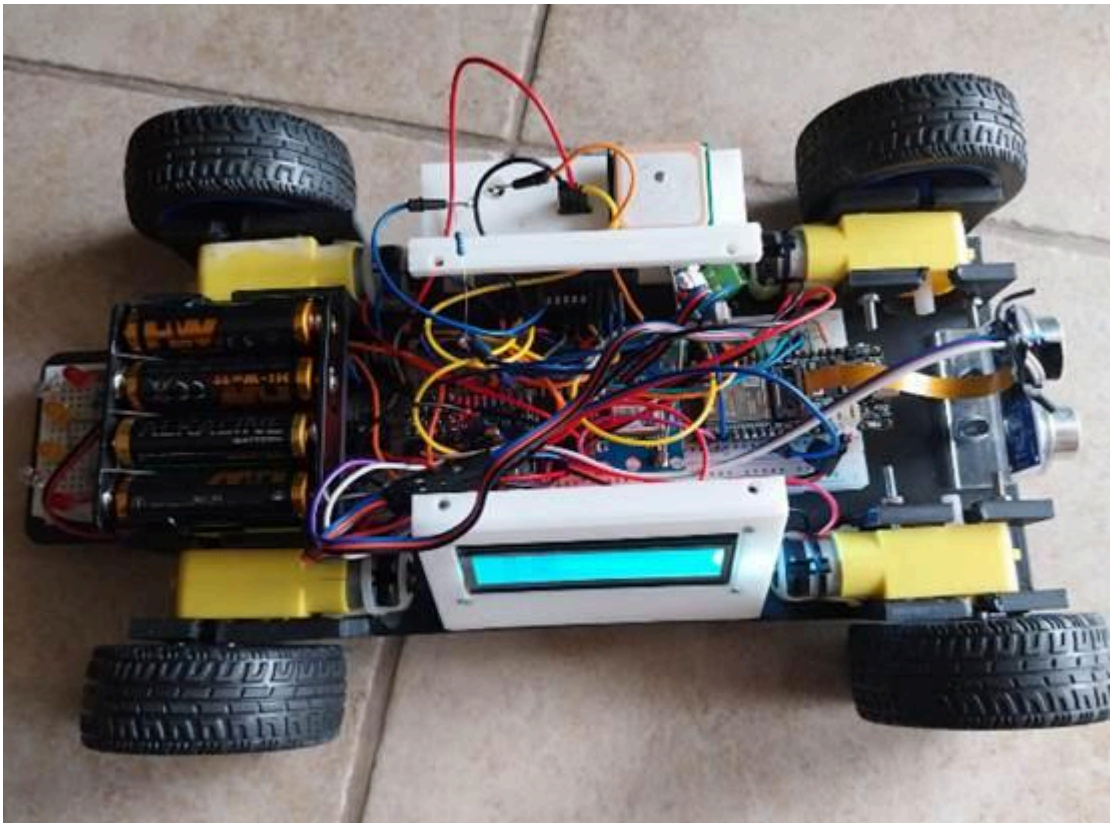


Εικόνα 22: School Screen #01

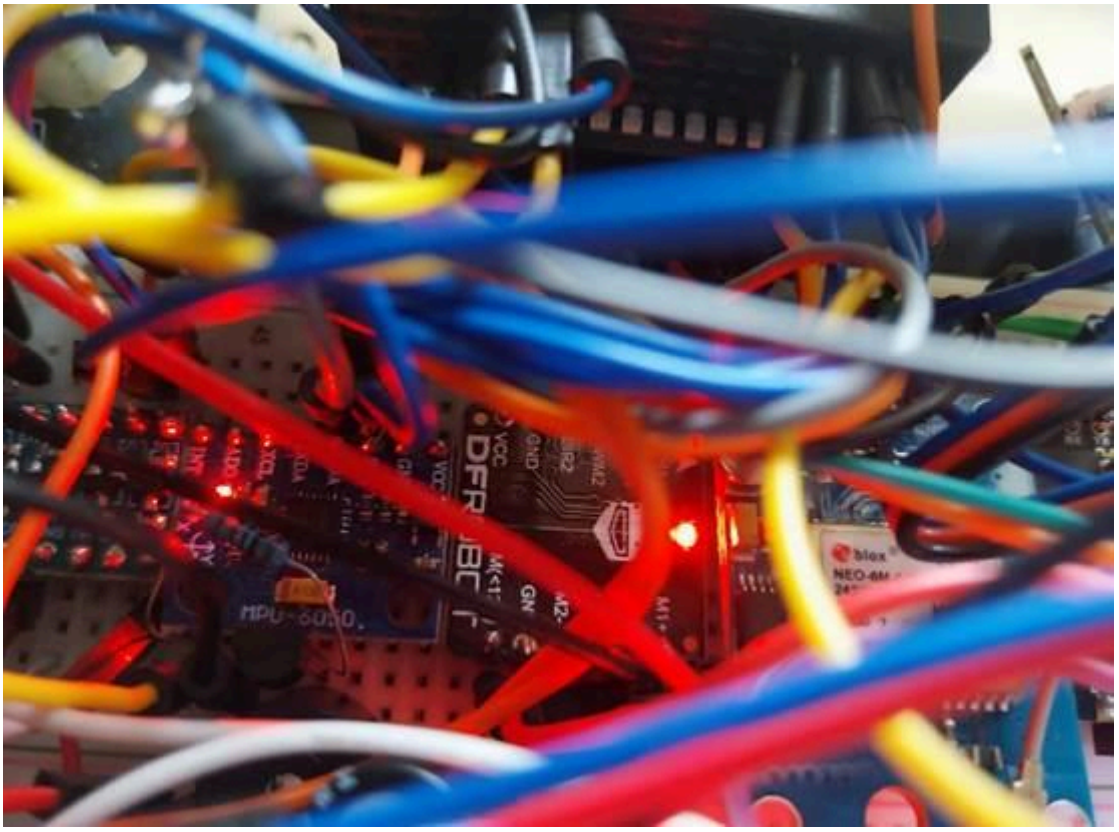


Εικόνα 23: School Screen #02 1

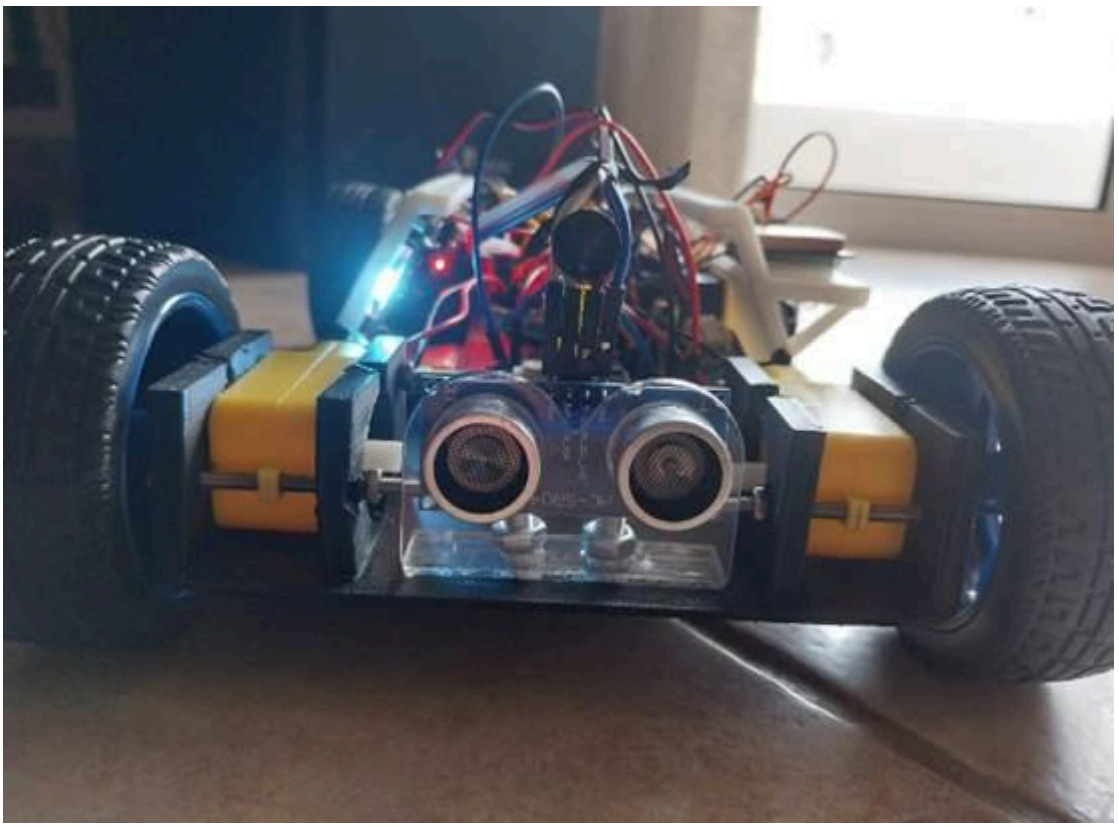
Φωτογραφική Απεικόνιση



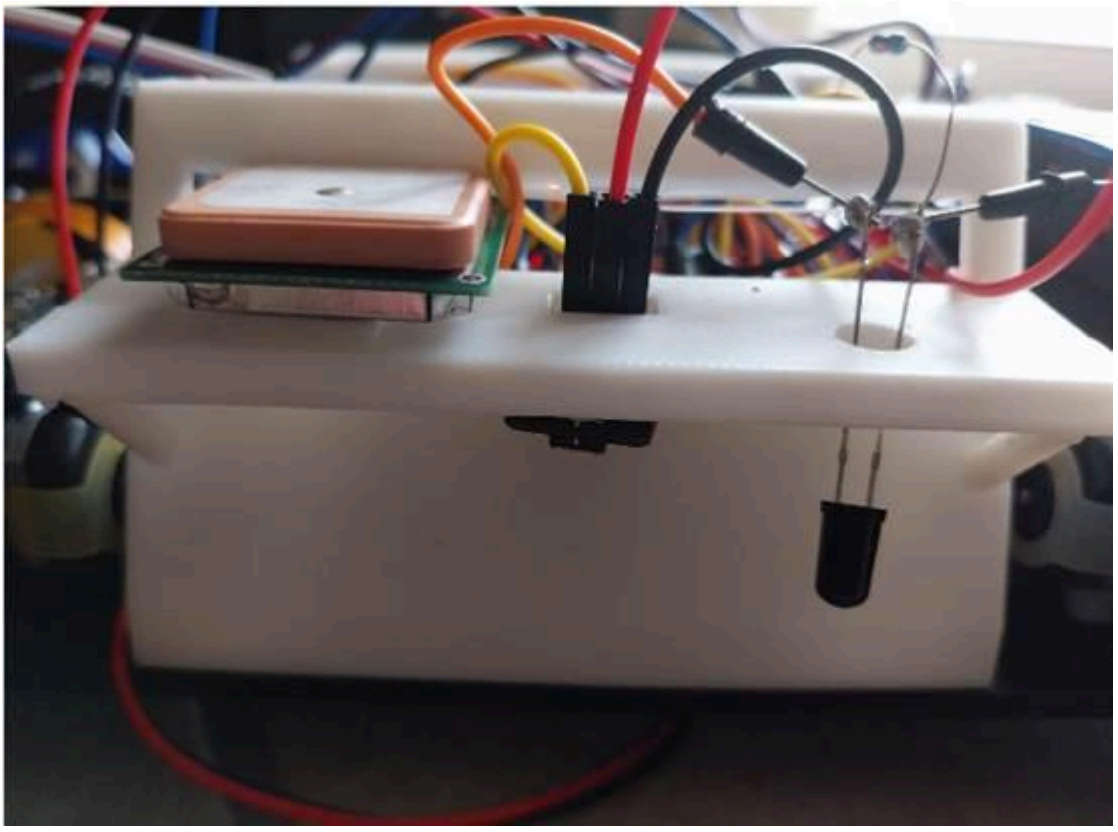
Εικόνα 51: Σχολικό



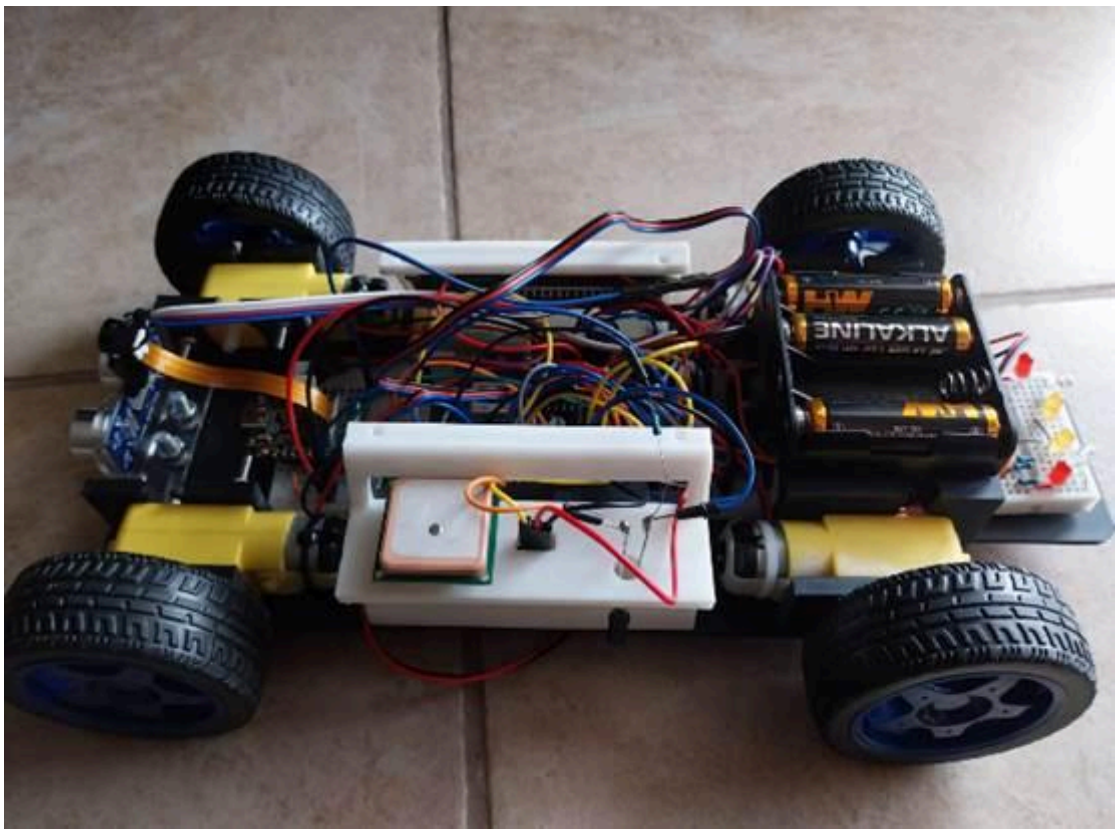
Εικόνα 52: Σχολικό



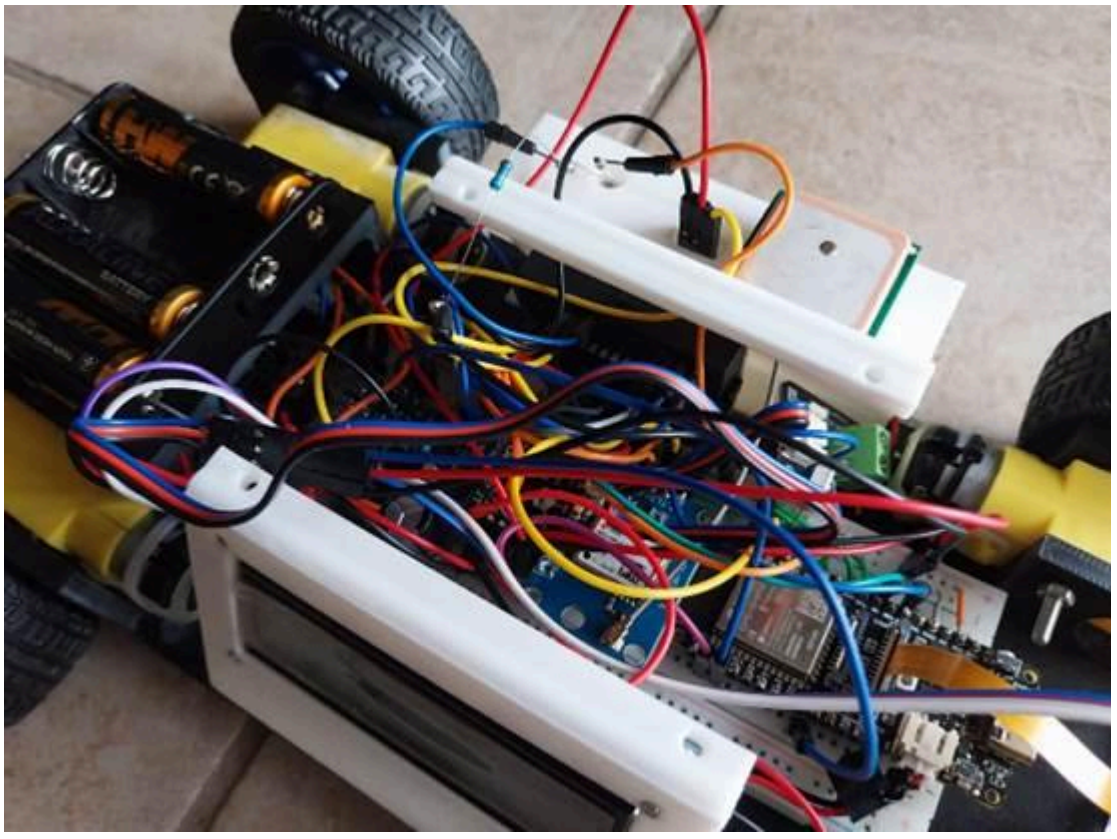
Εικόνα 53: Σχολικό



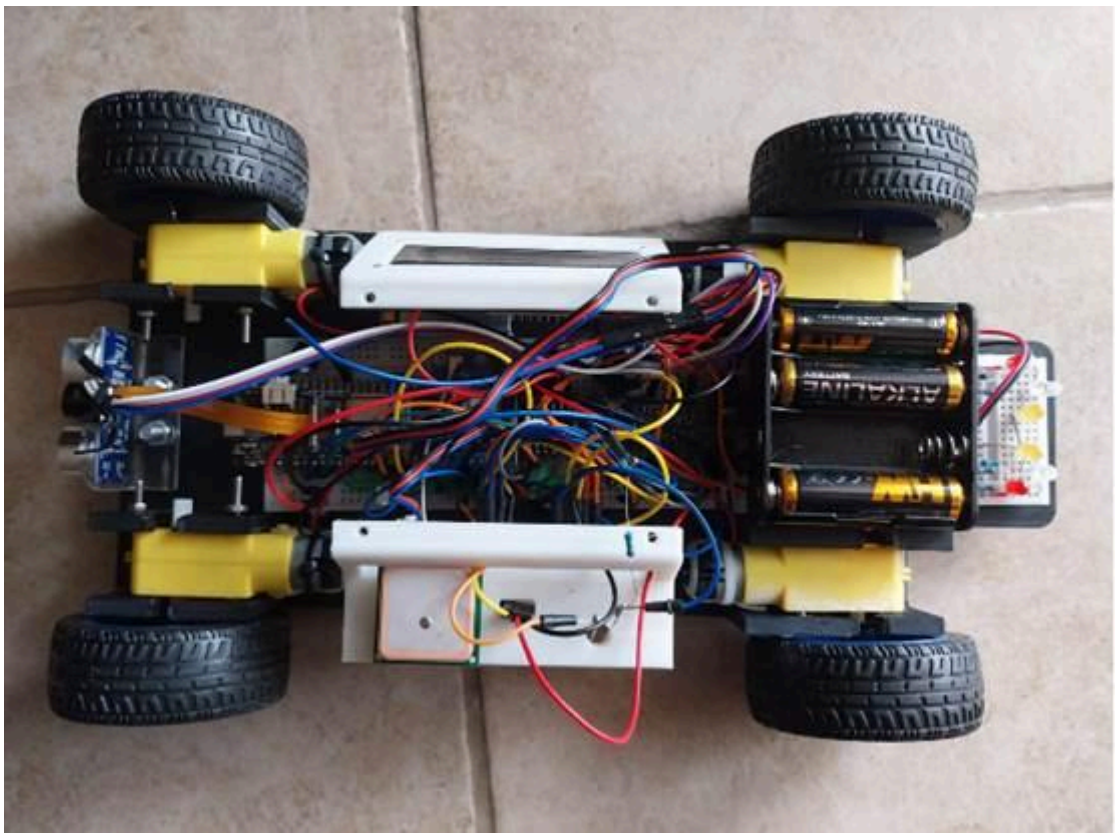
Εικόνα 54: Σχολικό



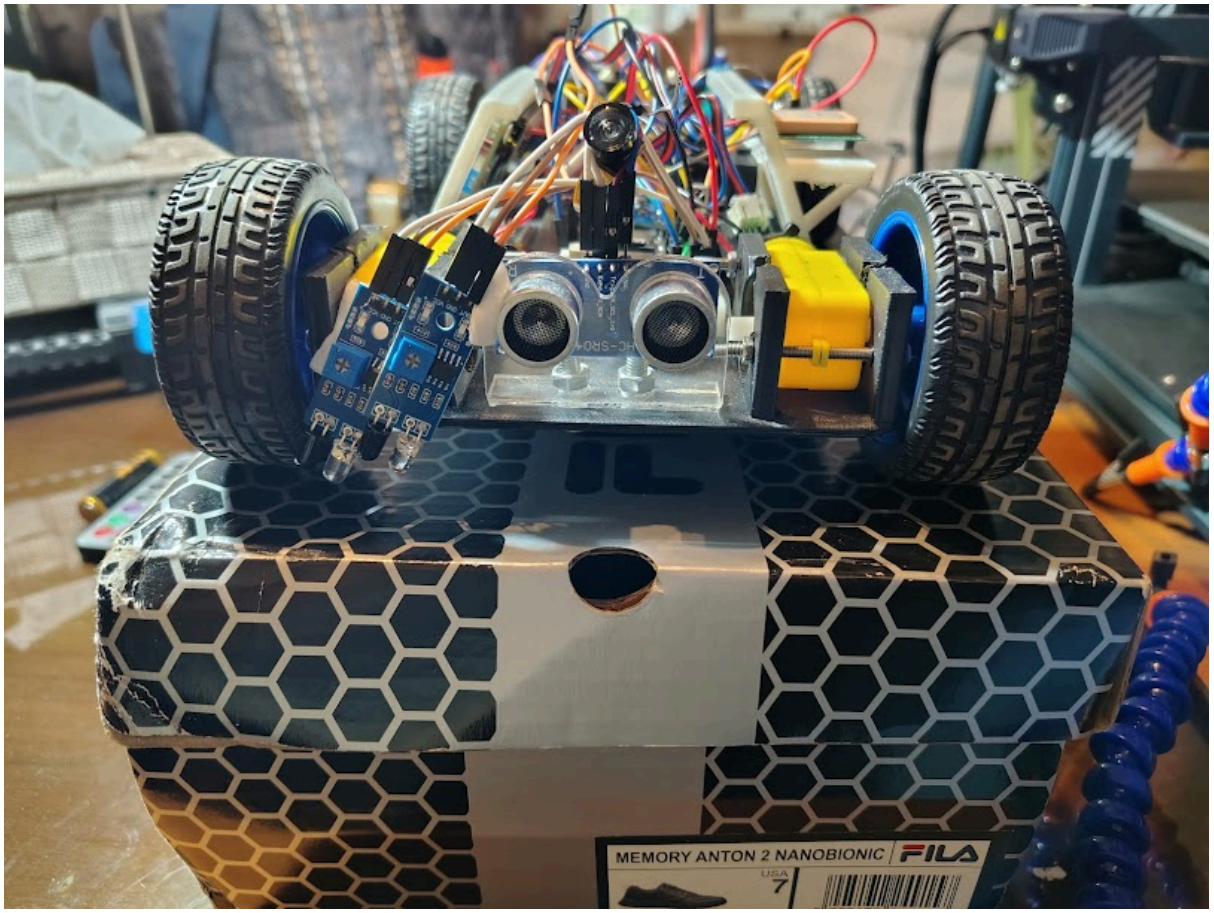
Εικόνα 55: Σχολικό



Εικόνα 56: Σχολικό



Εικόνα 57: Σχολικό



Πηγές & Datasheets

DFRobot FireBeetle 2 ESP32-S3 with Camera – 16MB Flash

DFRobot FireBeetle 2 ESP32-S3 with Camera – 16MB Flash - Schematic

DFRobot FireBeetle 2 ESP32-S3 with Camera – 16MB Flash - Datasheet

SKU DFR0975 FireBeetle 2 Board ESP32 S3

Analog Piezo Disk Vibration Sensor SKU DFR0052

Fermion: MEMS Smoke Sensor (breakout) Wiki - DFRobot

2x1.2A DC Motor Driver (TB6612FNG) wiki- DFRobot

Github - IRRemote

Github - Adafruit-MCP23017-Arduino-Library

How to use the MCP23017 Port Expander with the Raspberry Pi

HC-SR04 Ultrasonic Sensor

Empirical PID gain tuning (Kevin Lynch)

Τι είναι το PID, πόσο σημαντικό είναι και πώς το ρυθμίζουμε

Basic 16x2 Character LCD - White on Blue 5V

Επιταχυνσιόμετρο & Γυροσκόπιο 3 Αξόνων IMU - MPU6050 (Soldered Header)

DC Gear Motor TT - 130 RPM Datasheet

Controlling Self Driving Cars

Announcing FOMO

Simple ESP32 - CAM Object Detection

Understanding I2C

Understanding UART

Γαστεράτος Α. *Μηχατρονική.* Εκδόσεις Τσότρας Ε.Ε.

Καλοφωλιάς Δ. *Προγραμματισμός του Μικροελεγκτή AVR ATMega328. ΕΚΔΟΣΕΙΣ Α. ΤΖΙΟΛΑ & ΥΙΟΙ Α.Ε.*